StayNest Pro

Hotel, Resort and Room Booking Backend
Screenshots Report

50 / 50	153	40	62
Modules done	API endpoints	MySQL tables	Tests passed

Developer	Shaik Khalid
Stack	Python, FastAPI, MySQL, SQLAlchemy, Alembic, Pydantic, JWT
Testing	Swagger UI, Postman, pytest
Payments	Stripe (mock mode)
GitHub	github.com/shaikkhalid7095-shaik/staynest-pro

Contents

Module Coverage Checklist (50 Modules) 4

Part 1 - Environment Setup and MySQL Database 6

Part 2 - Swagger UI: All API Endpoints 10

Part 3 - Module-wise Testing (50 Modules) 16

Module 1. Authentication and RBAC 16
Module 2. Customer Profile Management 22
Module 3. Hotel Registration and Verification 23
Module 4. Hotel Profile Management 24
Module 5. Room Management 25
Module 6. Room Type Management 27
Module 7. Hotel Amenities Management 28
Module 8. Room Availability Management 28
Module 9. Hotel Search System 29
Module 10. Advanced Hotel Filters 29
Module 11. Hotel Details and Gallery 29
Module 12. Room Booking System 30
Module 13. Guest Management 32
Module 14. Booking Management 33
Module 15. Stripe Payment Integration 34
Module 16. Invoice and Receipt Management 37
Module 17. Cancellation and Refund Management 38
Module 18. Coupon and Discount Management 40
Module 19. Reviews and Ratings 41
Module 20. Wishlist and Favorite Hotels 42
Module 21. AI Hotel Recommendation Engine 43
Module 22. AI Travel Assistant 44
Module 23. AI Customer Support Chatbot 44
Module 24. In-App Notifications 45
Module 25. In-App Messaging 45
Module 26. Housekeeping Management 46
Module 27. Hotel Staff Management 46
Module 28. Check-In and Check-Out Management 47
Module 29. Additional Services Management 48
Module 30. Room Service Management 49
Module 31. Hotel Revenue Analytics 50

Module 32. Hotel Occupancy Analytics . . . 51
Module 33. Customer Analytics . . . 52
Module 34. Hotel Owner Dashboard APIs . . . 53
Module 35. Customer Dashboard APIs . . . 54
Module 36. Admin Dashboard APIs . . . 55
Module 37. Hotel Verification and Moderation . . . 56
Module 38. Reports and Export . . . 58
Module 39. Audit Logs and Security Monitoring . . . 60
Module 40. Hotel Subscription Plans . . . 61
Module 41. Hotel Subscription Payments . . . 62
Module 42. Dynamic Pricing Management . . . 63
Module 43. Loyalty and Rewards . . . 64
Module 44. Location Services . . . 65
Module 45. Calendar and Booking Scheduler . . . 66
Module 46. Financial Management . . . 67
Module 47. Business Intelligence APIs . . . 68
Module 48. Fraud Detection and Security . . . 69
Module 49. System Configuration . . . 70
Module 50. Advanced Platform Administration . . . 71

Part 4 - Postman API Testing . . . 74

Part 5 - Module Check and Automated Tests . . . 77

Module Coverage Checklist (50 Modules)

All 50 required modules are implemented, pass check_modules.py (50 / 50) and pytest (62 passed), and every module has live Swagger, Postman or MySQL screenshots in this report.

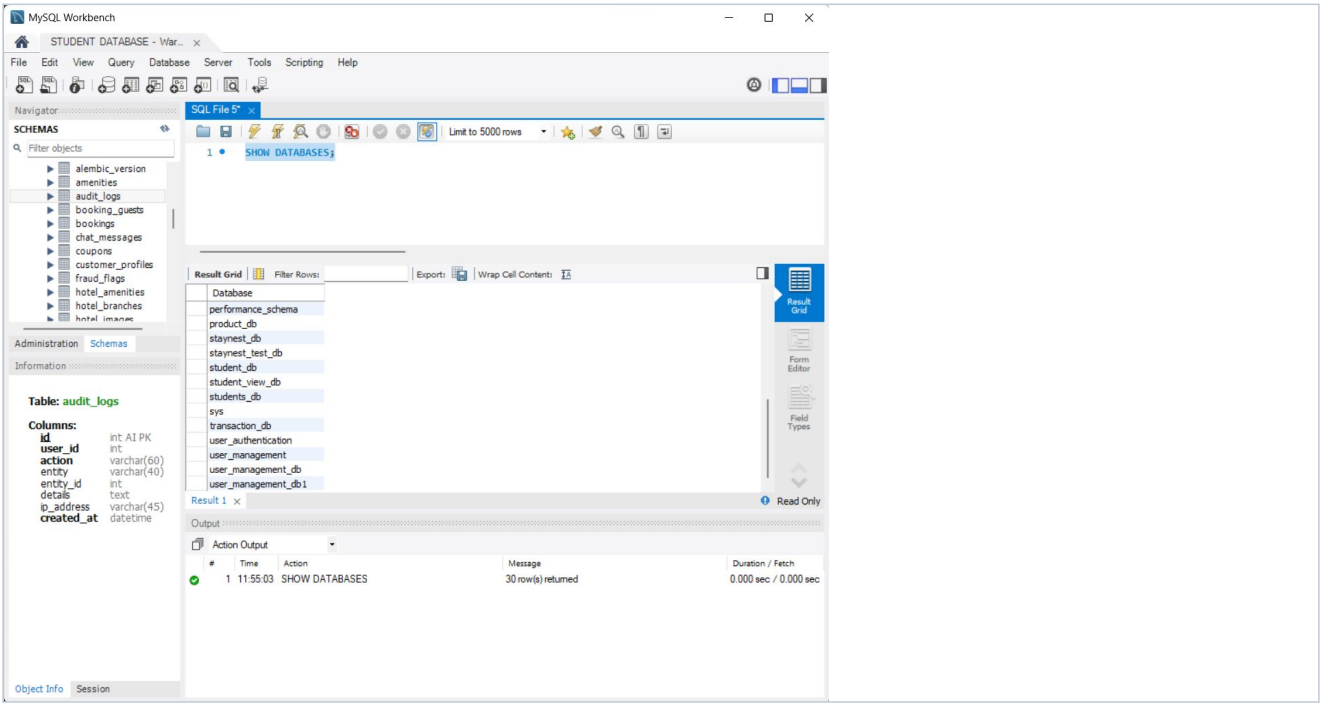
#	Module	Screenshot evidence	Status
1	Authentication and RBAC	Fig 18, 19, 20, 21, 22, 23, 24, 25, 26	Tested
2	Customer Profile Management	Fig 29, 30	Tested
3	Hotel Registration and Verification	Fig 31, 32	Tested
4	Hotel Profile Management	Fig 32, 33	Tested
5	Room Management	Fig 34, 35, 36, 37	Tested
6	Room Type Management	Fig 38, 39	Tested
7	Hotel Amenities Management	Fig 40	Tested
8	Room Availability Management	Fig 41	Tested
9	Hotel Search System	Fig 42	Tested
10	Advanced Hotel Filters	Fig 42	Tested
11	Hotel Details and Gallery	Fig 33	Tested
12	Room Booking System	Fig 43, 44, 45, 46	Tested
13	Guest Management	Fig 47, 48	Tested
14	Booking Management	Fig 47, 49	Tested
15	Stripe Payment Integration	Fig 50, 51, 52, 53	Tested
16	Invoice and Receipt Management	Fig 56	Tested
17	Cancellation and Refund Management	Fig 57, 58, 59, 60	Tested
18	Coupon and Discount Management	Fig 61, 62	Tested
19	Reviews and Ratings	Fig 63, 64, 65	Tested
20	Wishlist and Favorite Hotels	Fig 66, 67	Tested
21	AI Hotel Recommendation Engine	Fig 68	Tested
22	AI Travel Assistant	Fig 69	Tested
23	AI Customer Support Chatbot	Fig 70	Tested
24	In-App Notifications	Fig 71	Tested
25	In-App Messaging	Fig 72	Tested
26	Housekeeping Management	Fig 73	Tested
27	Hotel Staff Management	Fig 74	Tested
28	Check-In and Check-Out Management	Fig 48, 75	Tested
29	Additional Services Management	Fig 76, 77	Tested
30	Room Service Management	Fig 77, 78, 79, 80	Tested
31	Hotel Revenue Analytics	Fig 81	Tested
32	Hotel Occupancy Analytics	Fig 82	Tested
33	Customer Analytics	Fig 83	Tested
34	Hotel Owner Dashboard APIs	Fig 84	Tested
35	Customer Dashboard APIs	Fig 85	Tested
36	Admin Dashboard APIs	Fig 86	Tested
37	Hotel Verification and Moderation	Fig 87, 88, 89, 90	Tested
38	Reports and Export	Fig 91, 92, 93, 94	Tested
39	Audit Logs and Security Monitoring	Fig 95, 96, 97	Tested
40	Hotel Subscription Plans	Fig 84, 98	Tested
41	Hotel Subscription Payments	Fig 98, 99	Tested
42	Dynamic Pricing Management	Fig 100, 101	Tested

#	Module	Screenshot evidence	Status
43	Loyalty and Rewards	Fig 102, 103	Tested
44	Location Services	Fig 104, 105	Tested
45	Calendar and Booking Scheduler	Fig 101, 106	Tested
46	Financial Management	Fig 107, 108, 109	Tested
47	Business Intelligence APIs	Fig 110	Tested
48	Fraud Detection and Security	Fig 111, 112	Tested
49	System Configuration	Fig 113, 114, 115	Tested
50	Advanced Platform Administration	Fig 116, 117, 118, 119, 120	Tested

Part 1 - Environment Setup and MySQL Database

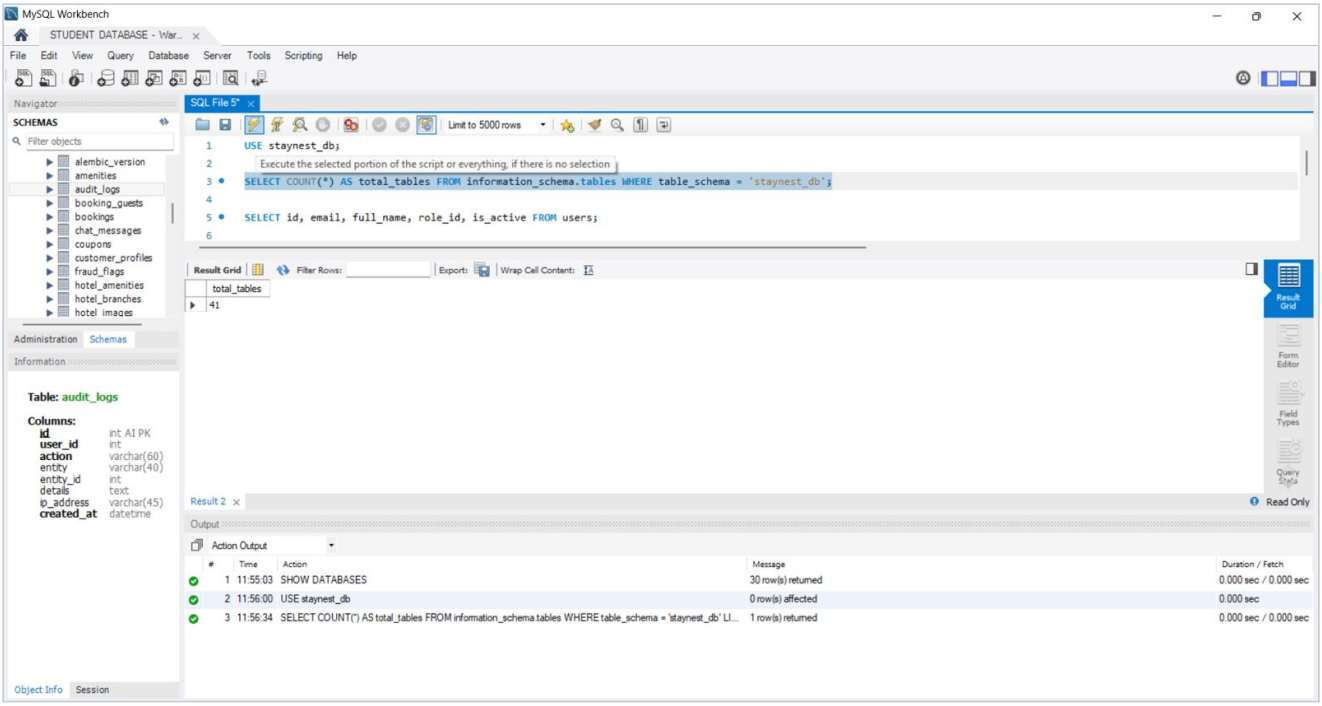
Step 3 of the task: MySQL is configured, staynest_db is created and the tables are verified in Workbench.

Figure 1. SHOW DATABASES in MySQL Workbench



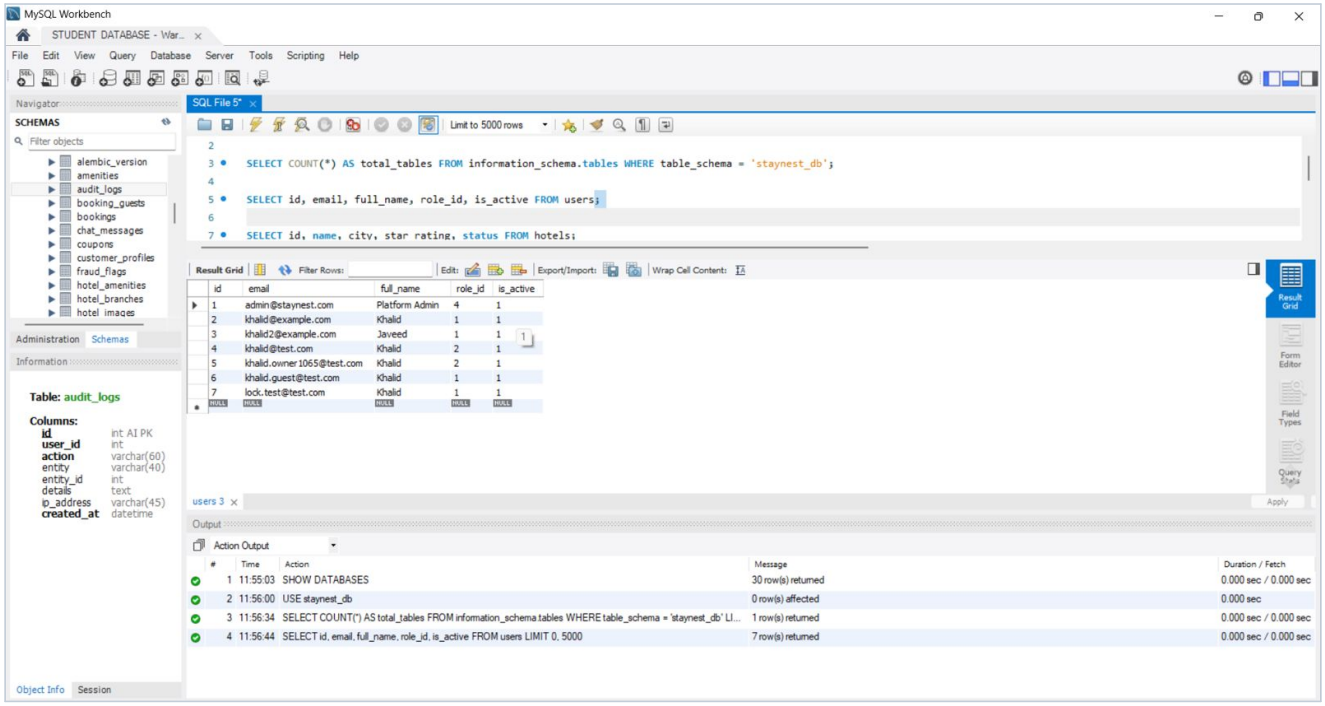
Workbench is connected to the local MySQL 9 server.
staynest_db, the project database, is listed with staynest_test_db next to it.

Figure 2. Count tables in staynest_db



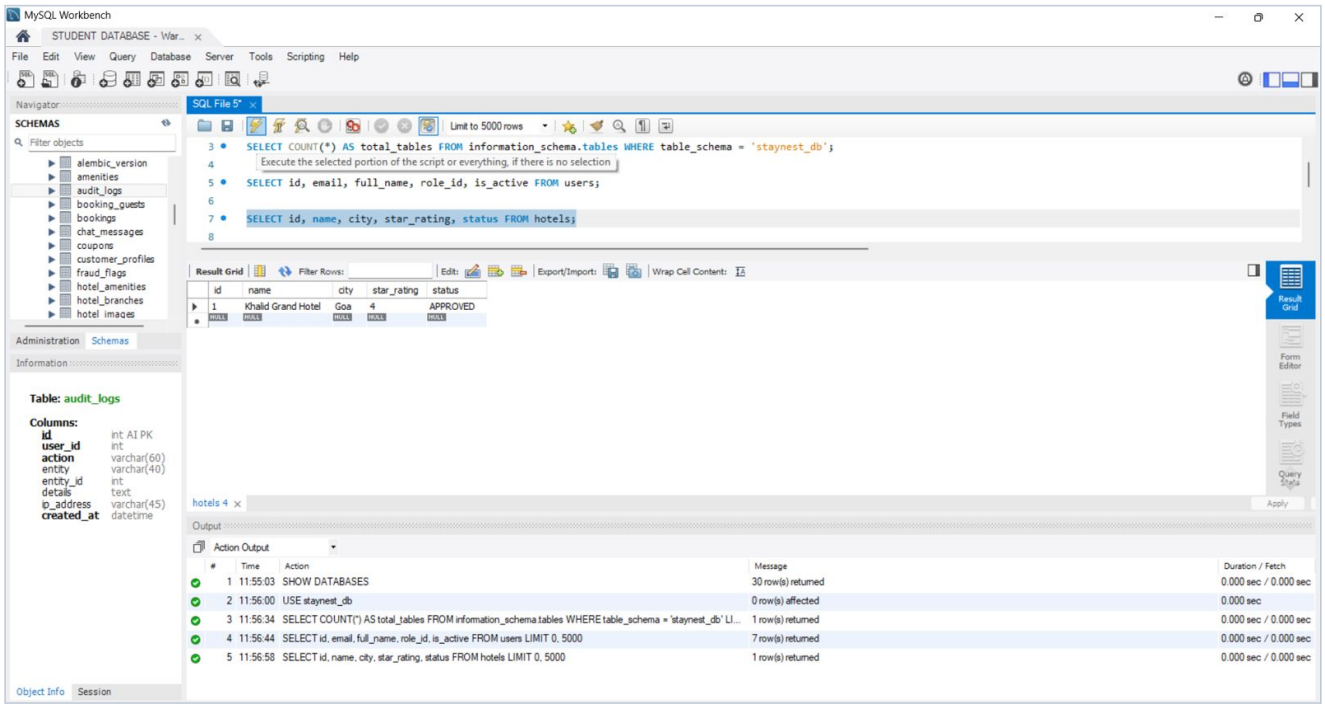
information_schema is queried for the number of tables in staynest_db.
It returns 41: the 40 project tables plus alembic_version used by migrations.

Figure 3. users table



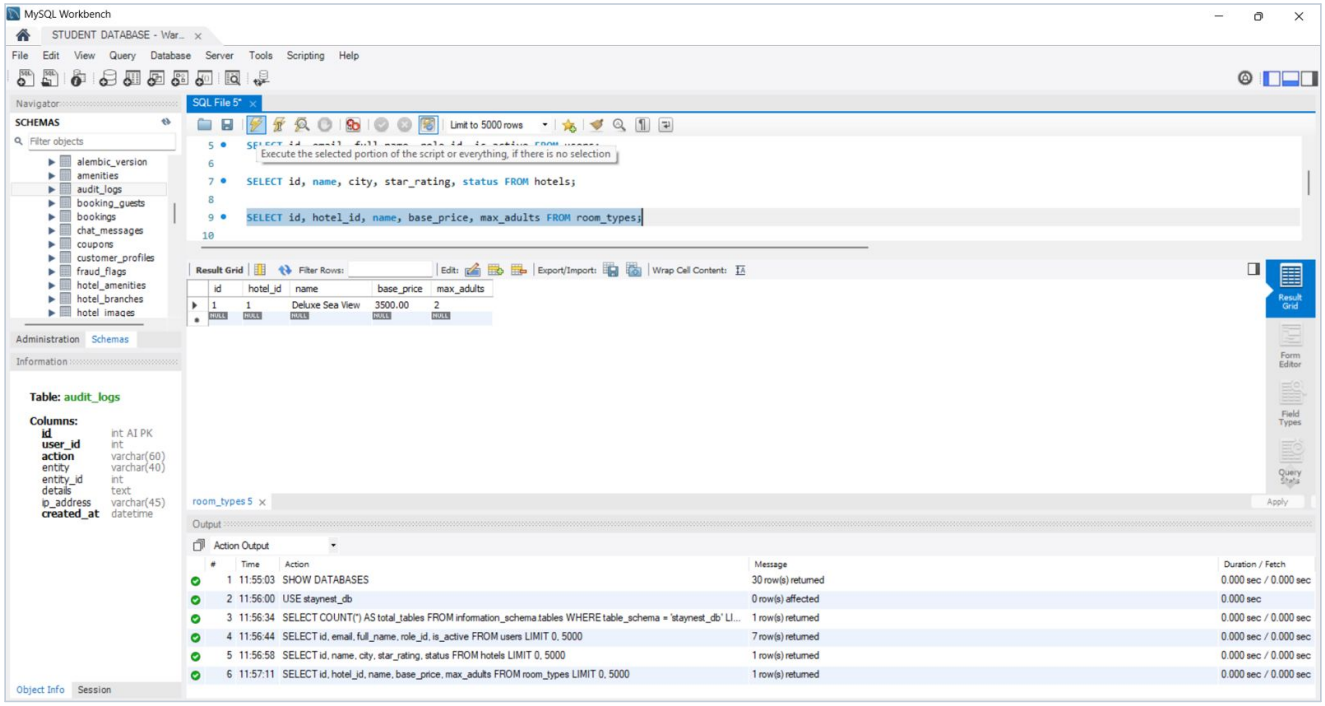
SELECT on users shows every account created during testing.
7 users with role_id 1 CUSTOMER, 2 HOTEL_OWNER and 4 ADMIN, all active.

Figure 4. hotels table



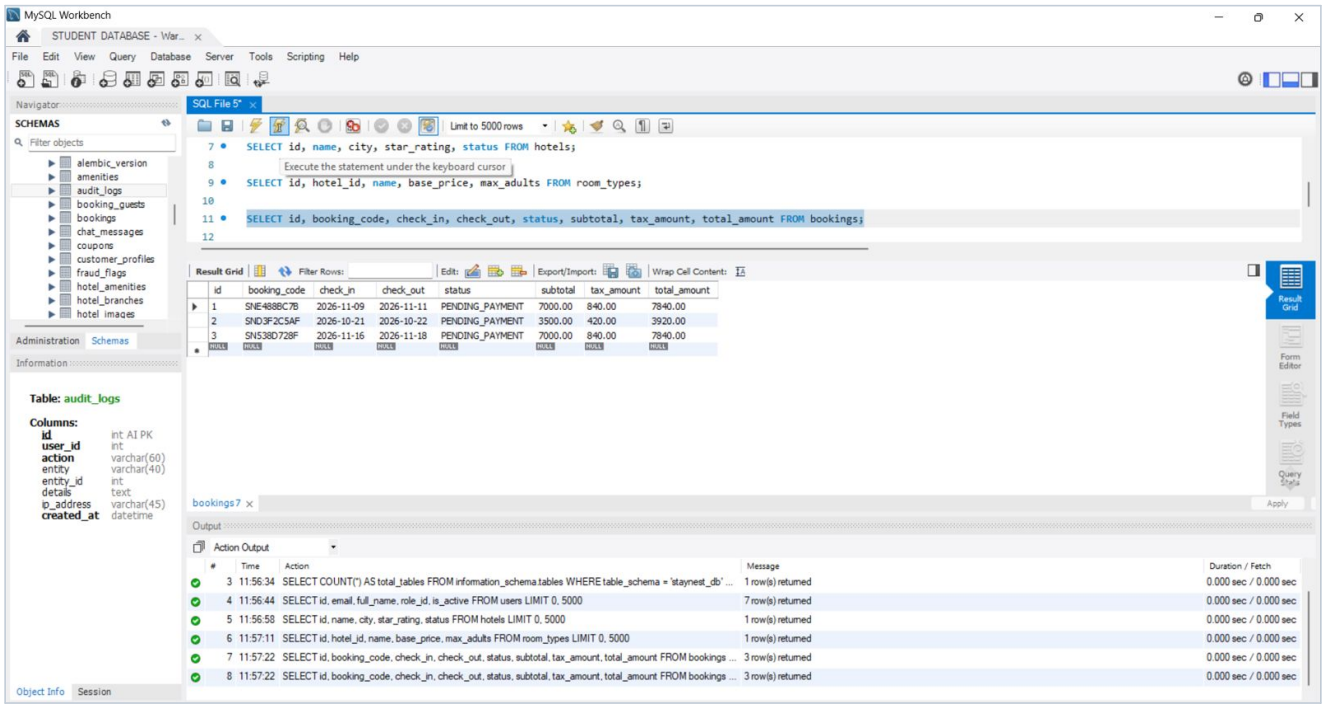
The hotel registered through the API is stored in MySQL.
Khalid Grand Hotel, Goa, 4 stars, status APPROVED.

Figure 5. room_types table



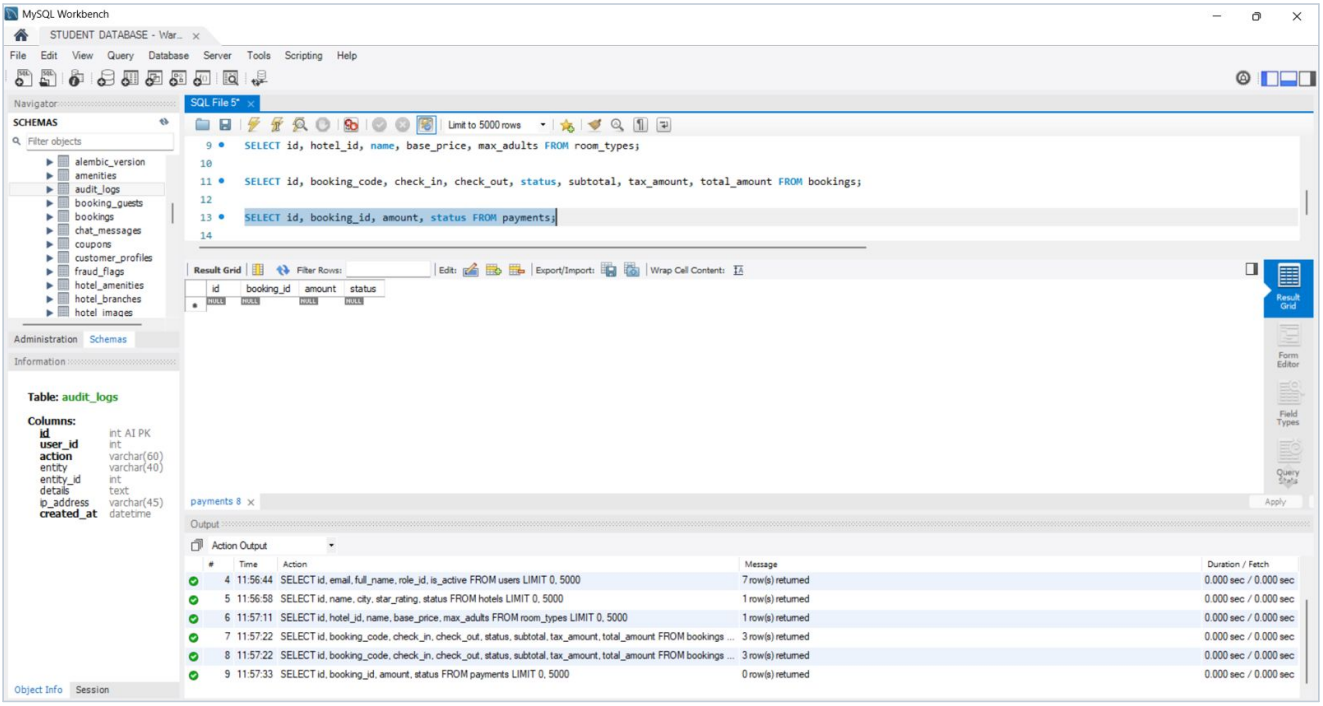
The room type created by the owner is saved against hotel 1.
Deluxe Sea View with base price 3500.00 and max 2 adults.

Figure 6. bookings table



Each booking stores dates, status and the full price break-up.
3 bookings are PENDING_PAYMENT, with subtotal, 12% GST and total amounts.

Figure 7. payments table before paying

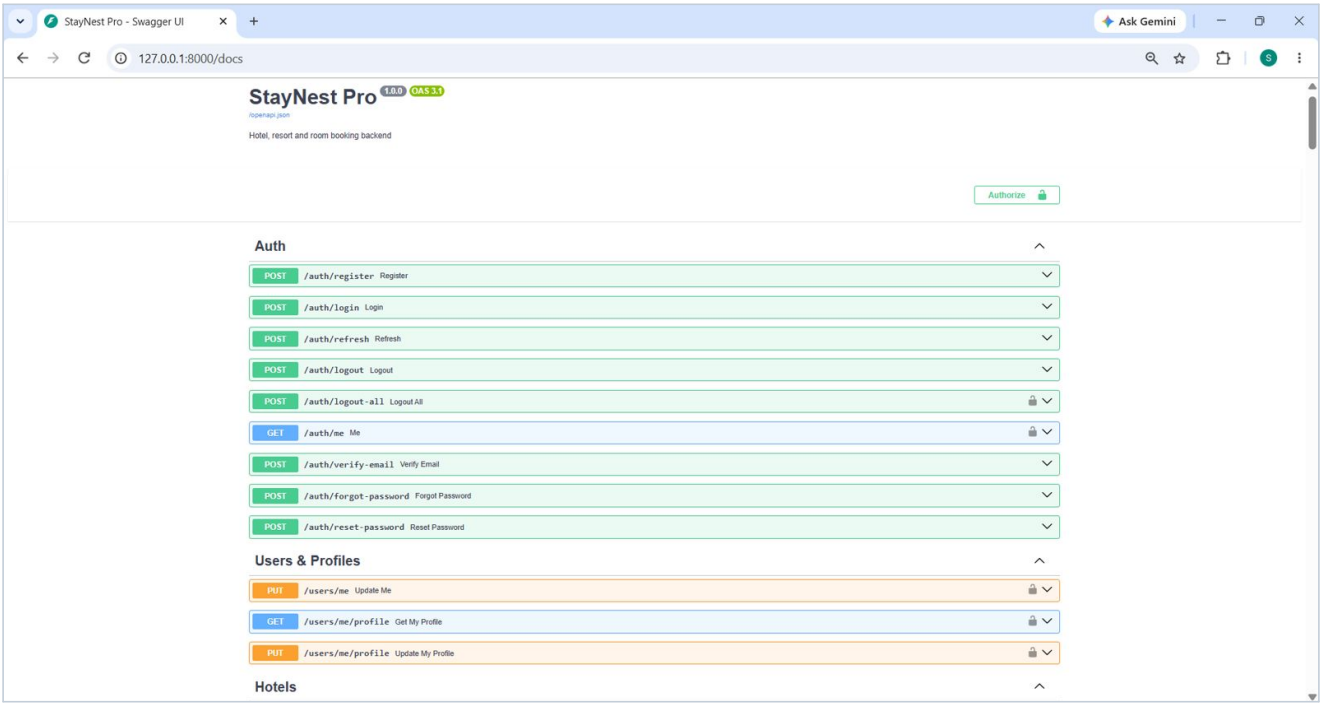


The payments table is checked before any payment is made.
It is empty (0 rows), so payments are only created when the customer starts to pay.

Part 2 - Swagger UI: All API Endpoints

The FastAPI app runs at 127.0.0.1:8000/docs and lists all 153 endpoints, grouped by feature.

Figure 8. Swagger UI home - Auth and Users & Profiles



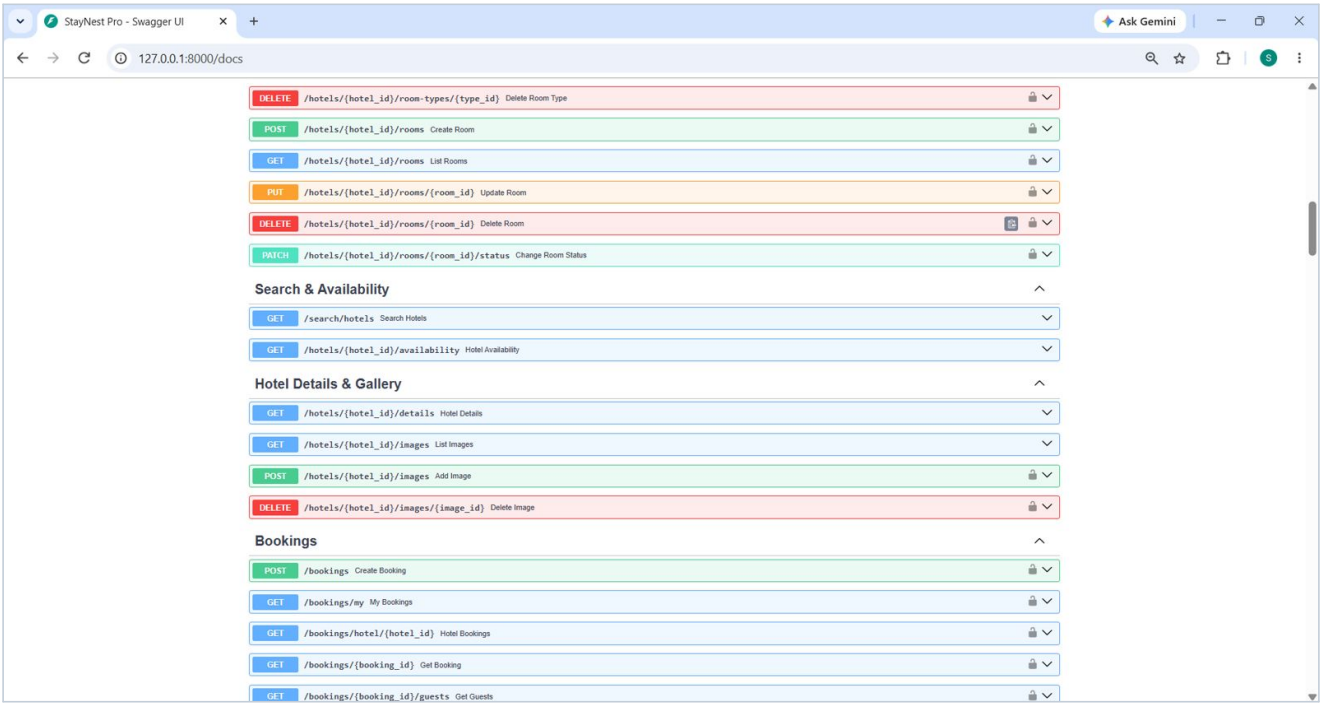
StayNest Pro 1.0.0 is running at 127.0.0.1:8000/docs with the Authorize button for JWT login.
Auth covers register, login, refresh, logout, verify-email and forgot / reset password; Users & Profiles lets a customer view and update their profile.

Figure 9. Hotels, Amenities and Room Types



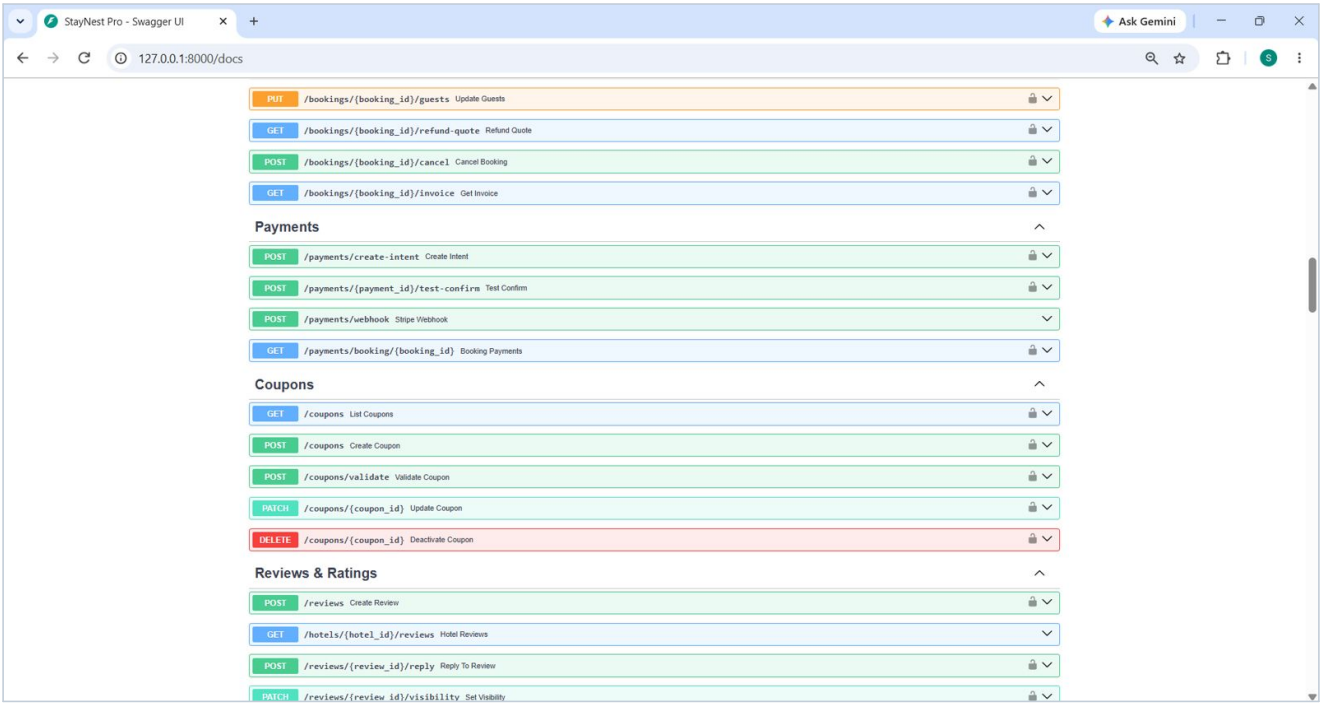
Owners register hotels, update or deactivate them, add branches and set amenities; admins verify hotels and list them by status.
Amenities can be listed, created and deleted, and room types are created and listed per hotel.

Figure 10. Rooms, Search, Gallery and Bookings



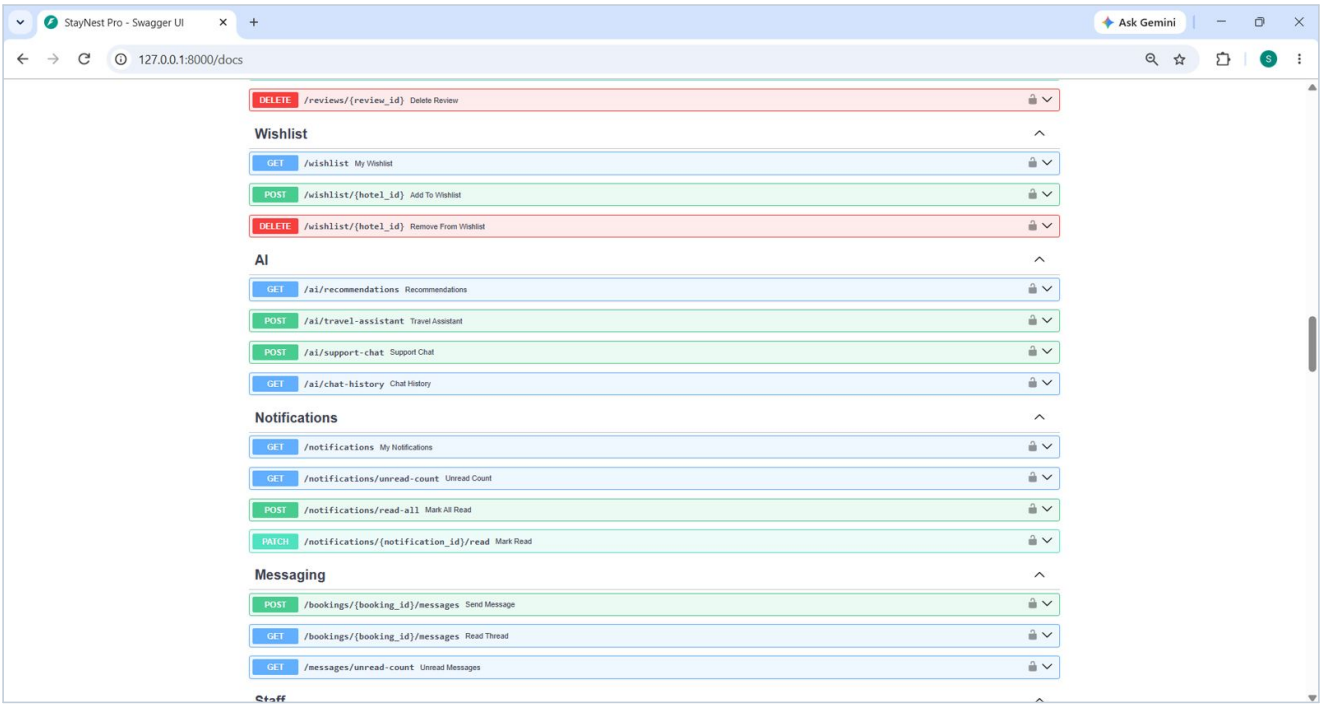
Rooms support create, list, update, delete and status change; customers search hotels and check availability.
Hotel details and gallery images are public, and the Bookings group starts with create booking and my bookings.

Figure 11. Bookings, Payments, Coupons and Reviews



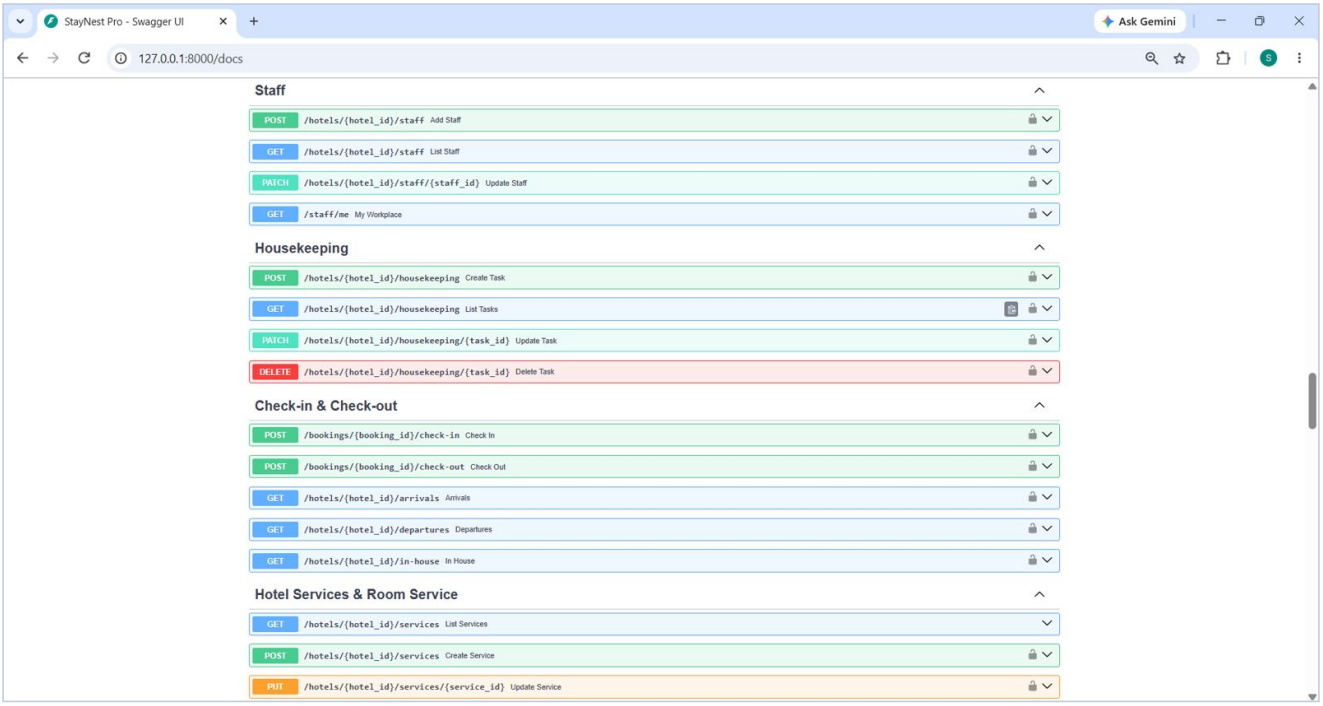
Bookings support guest updates, refund quote, cancel and invoice; Payments has create-intent, test-confirm and the Stripe webhook.
Coupons can be created, validated, updated and deactivated, and customers post reviews that owners can reply to.

Figure 12. Wishlist, AI, Notifications and Messaging



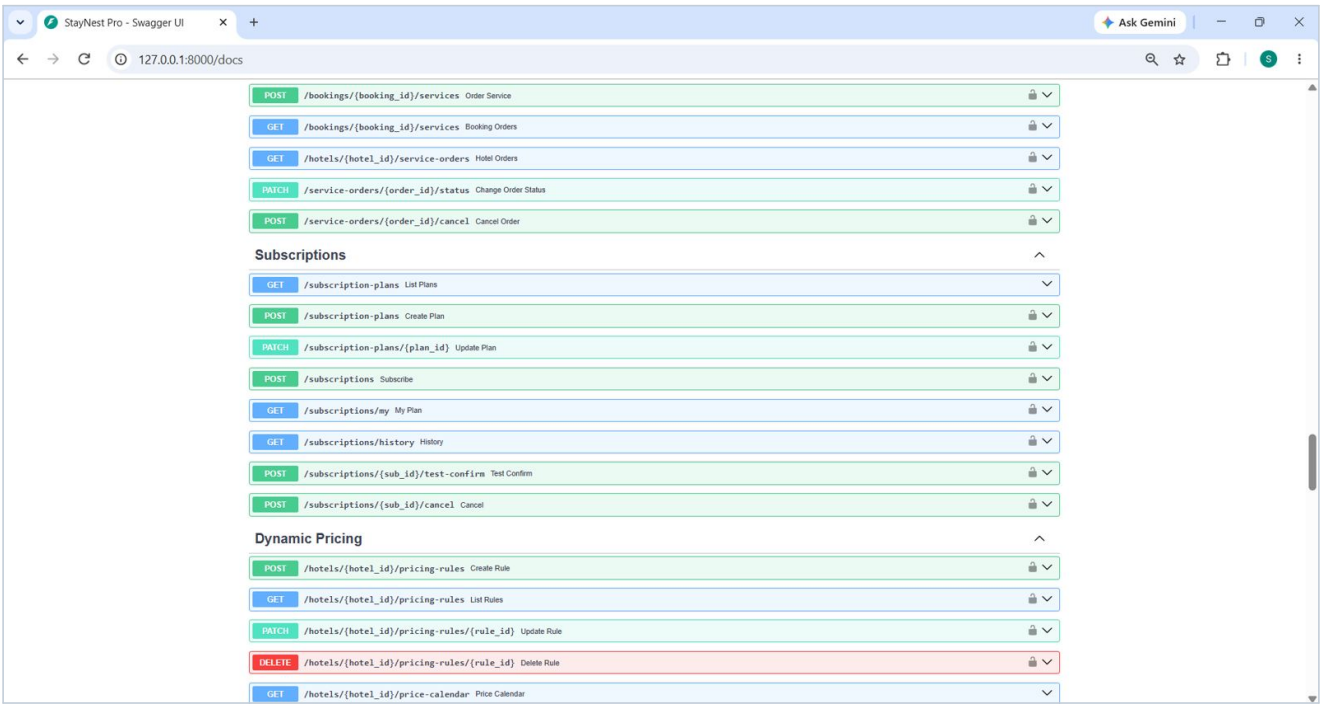
Customers save hotels to a wishlist and use AI recommendations, a travel assistant and a support chat with history.
In-app notifications show unread counts and read status, and guests message the hotel inside each booking.

Figure 13. Staff, Housekeeping, Check-in and Hotel Services



Owners add and manage staff, and housekeeping tasks are created, listed, updated and deleted per hotel.
Staff check guests in and out and see arrivals, departures and in-house guests; hotels manage extra services.

Figure 14. Room Service, Subscriptions and Dynamic Pricing



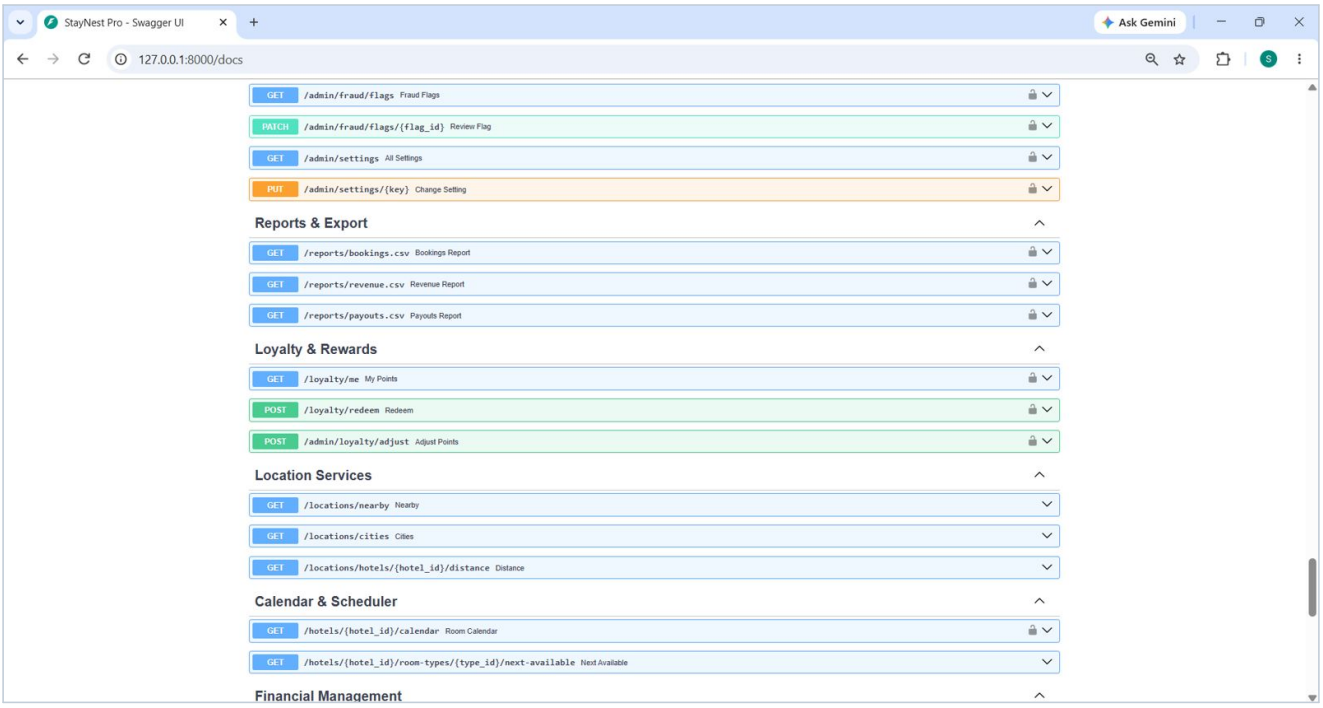
Guests order services on a booking and staff change order status; owners subscribe to plans with test-confirm and cancel.
Dynamic pricing rules are created, updated and deleted per hotel, with a price calendar to preview rates.

Figure 15. Analytics, Dashboards and Admin



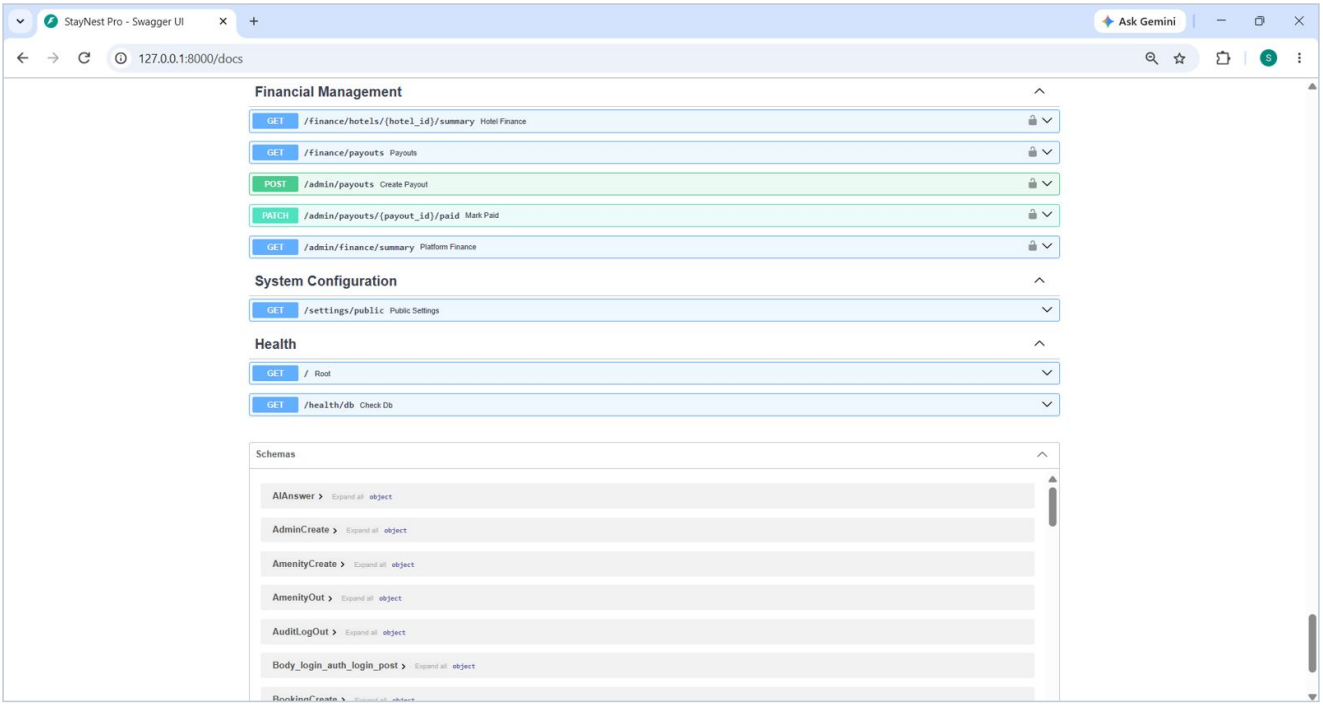
Revenue, occupancy and customer analytics, a BI overview and separate dashboards for owner, customer and admin.
Admins list users, block or unblock them, force logout, create admins, suspend or reinstate hotels and view audit logs.

Figure 16. Fraud, Settings, Reports, Loyalty, Location and Calendar



Admins review fraud flags and change platform settings; bookings, revenue and payout reports export as CSV.
Customers earn and redeem loyalty points, find nearby hotels and cities, and see room calendars and next available dates.

Figure 17. Financial Management, System Configuration and Health



Hotel finance summaries, payouts and platform finance are available, with admin create-payout and mark-paid. Public settings, a root health check and a database health check are listed above the Pydantic schemas.

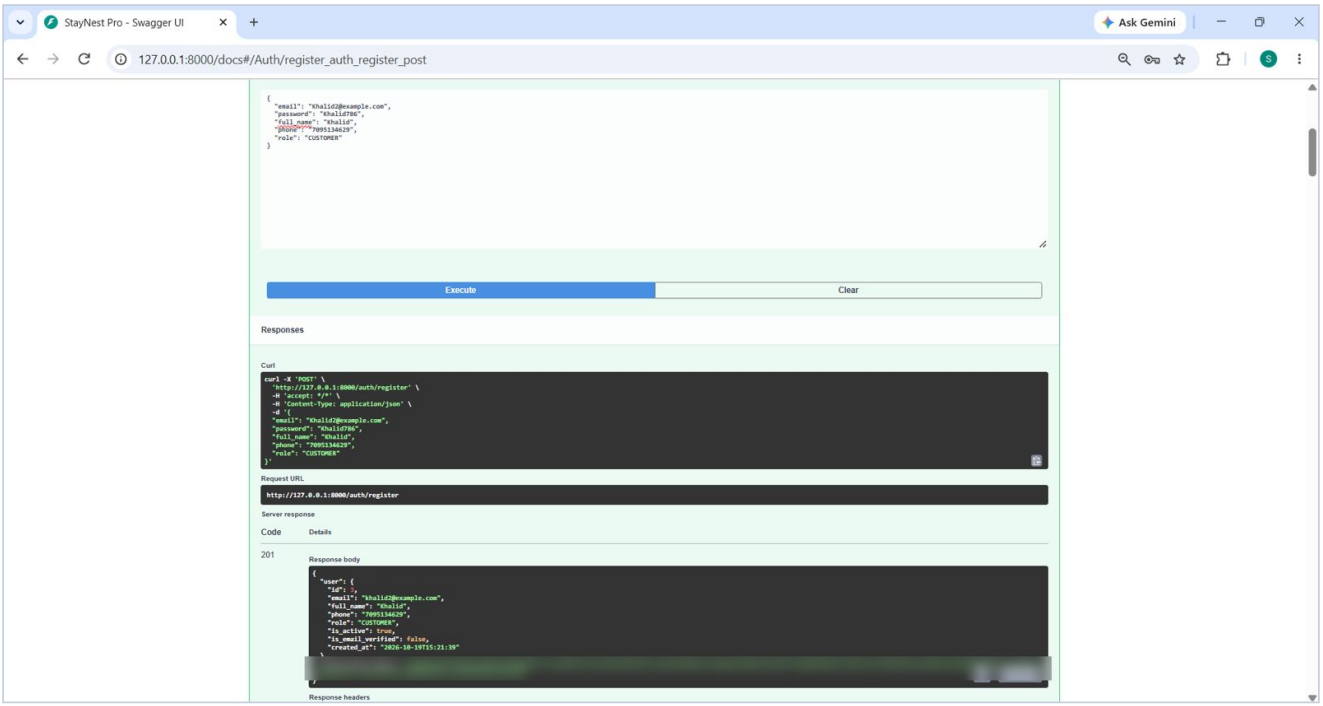
Part 3 - Module-wise Testing (50 Modules)

Each module is tested in Swagger, Postman or MySQL in the order of the task's module list and five-day plan.

Module 1. Authentication and RBAC

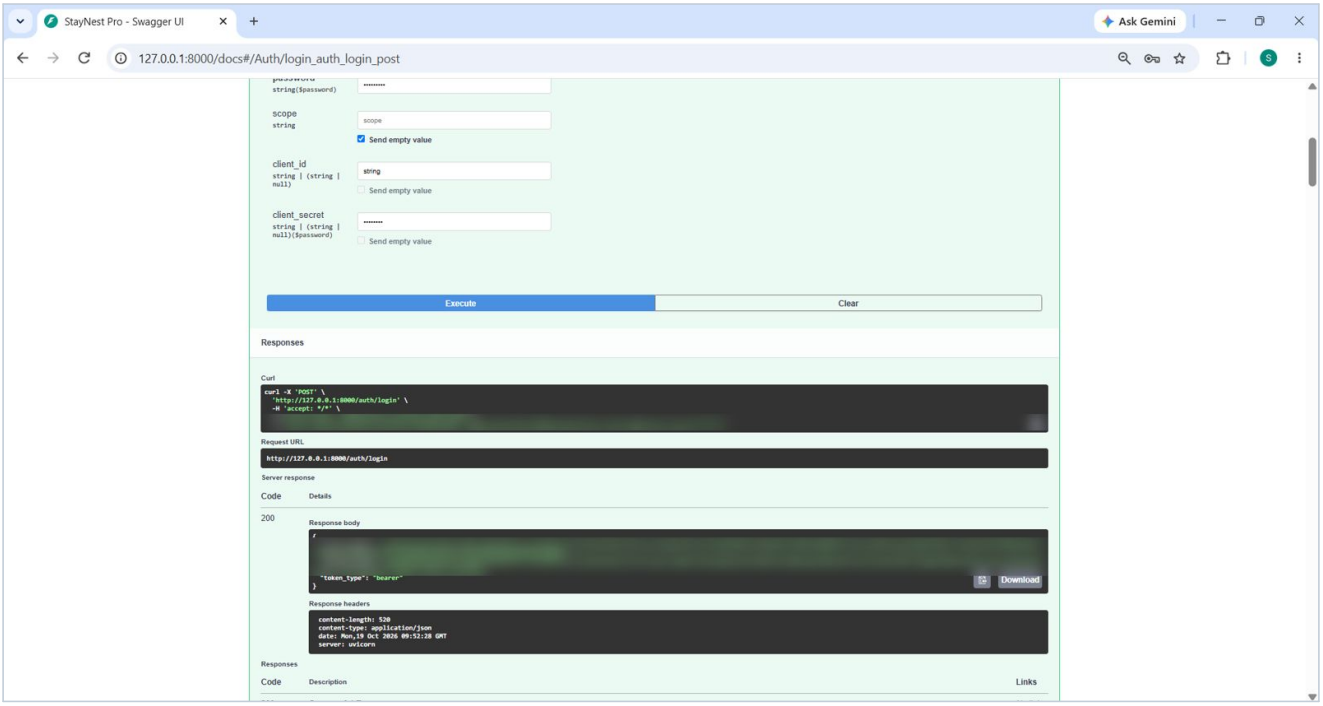
Day 1

Figure 18. Register a customer - POST /auth/register



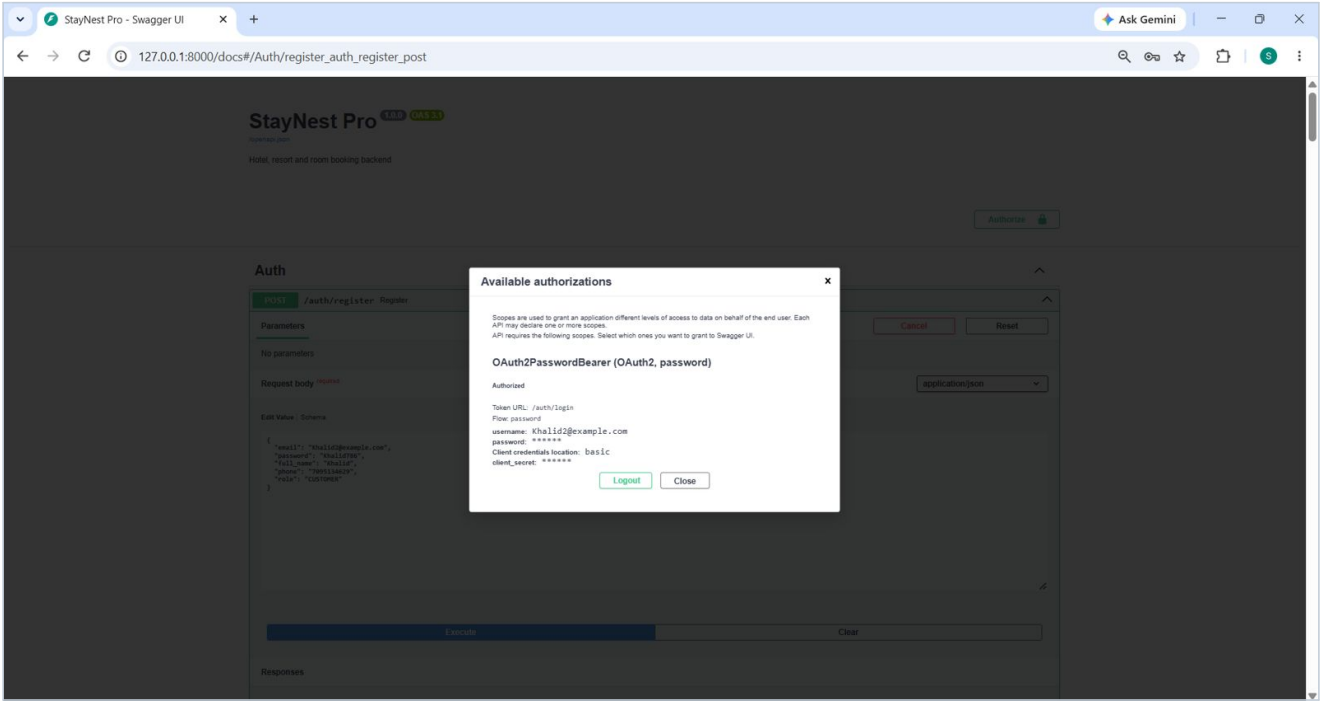
A new customer account is created with email, password, name, phone and role CUSTOMER, returning 201 Created.
The response shows user id 3 with is_email_verified false and a verification_token (blurred here) for the email check.

Figure 19. Login - POST /auth/login



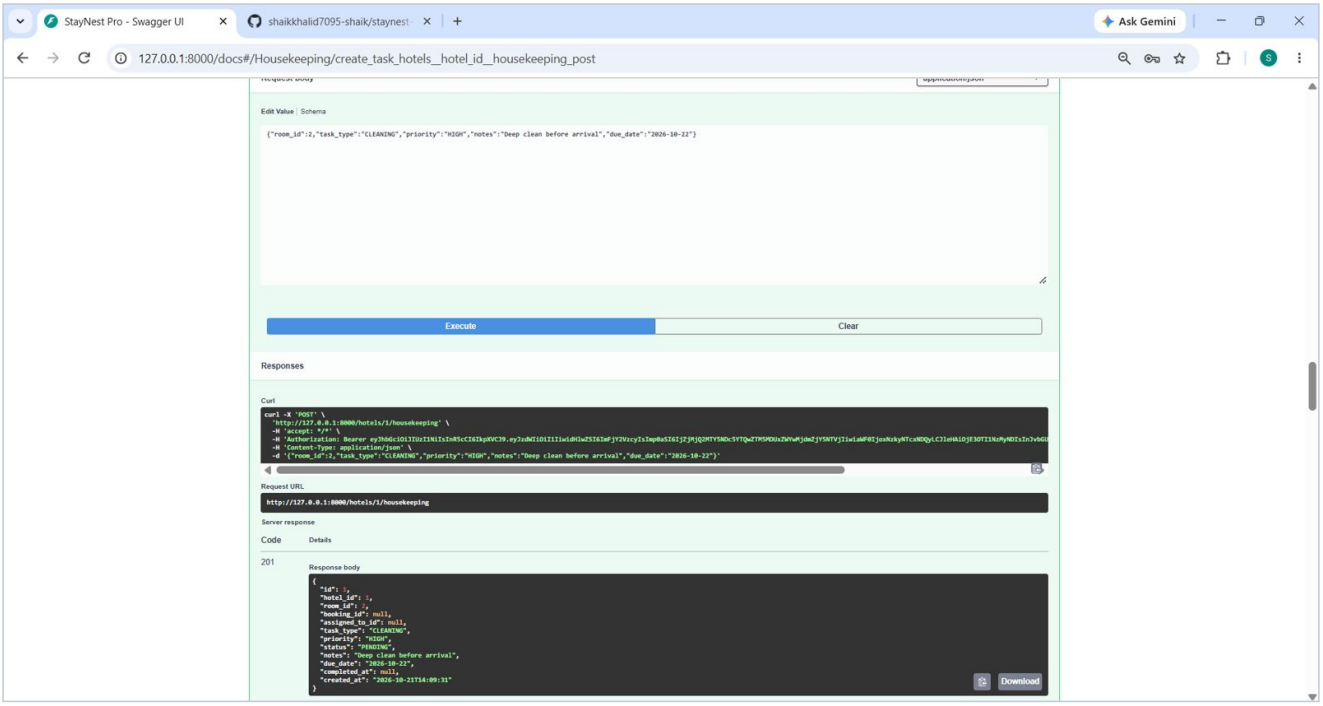
Login uses the OAuth2 password form (x-www-form-urlencoded), and the API returns 200 OK.
The response carries a 30 minute access_token, a 7 day refresh_token and token_type bearer (tokens blurred).

Figure 20. Authorize in Swagger UI



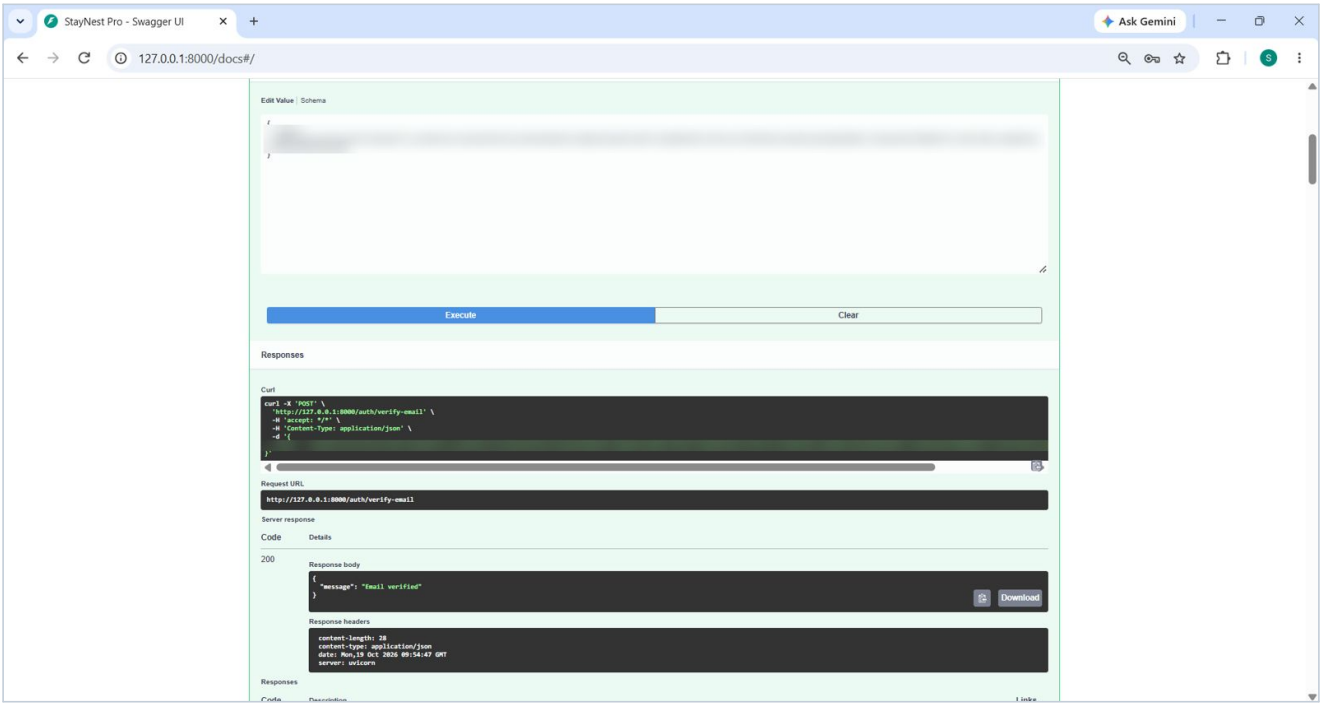
The Authorize dialog logs in through /auth/login with the OAuth2 password flow.
After this, Swagger sends the Bearer token with every request to a locked endpoint.

Figure 21. Authorize dialog in Swagger UI



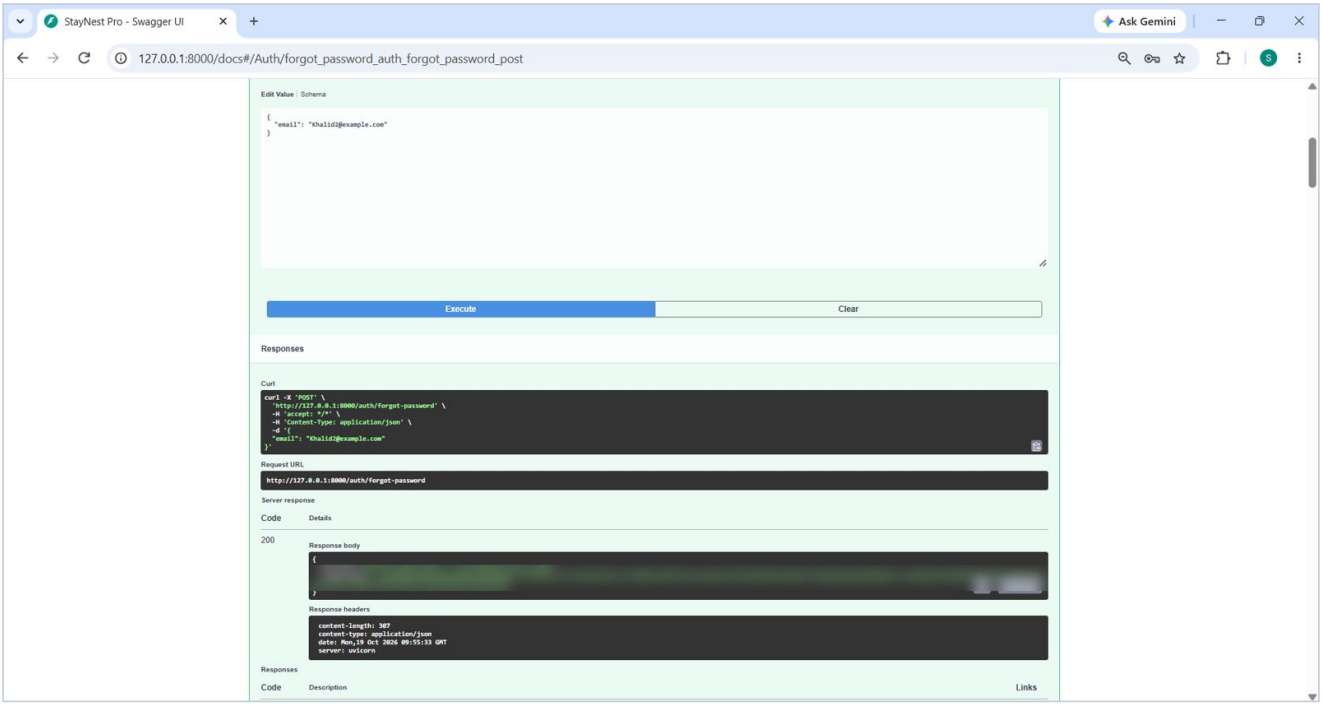
Swagger is logged in through the OAuth2 password flow as khalid.owner1065@test.com, and the password is hidden.
The Logout button switches users, which is how each role (owner, customer, admin) was tested.

Figure 22. Verify email - POST /auth/verify-email



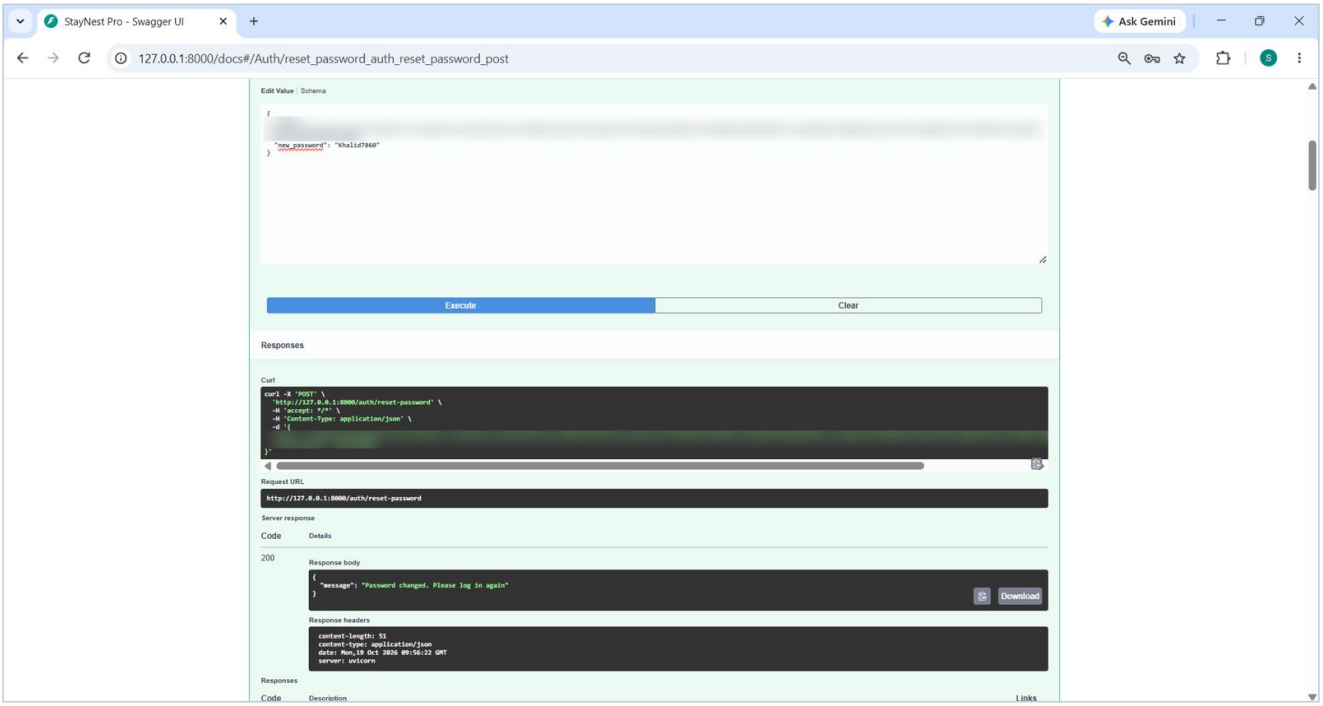
The verification token from registration is sent in the request body.
The API answers 200 OK with the message Email verified, so the account is now verified.

Figure 23. Forgot password - POST /auth/forgot-password



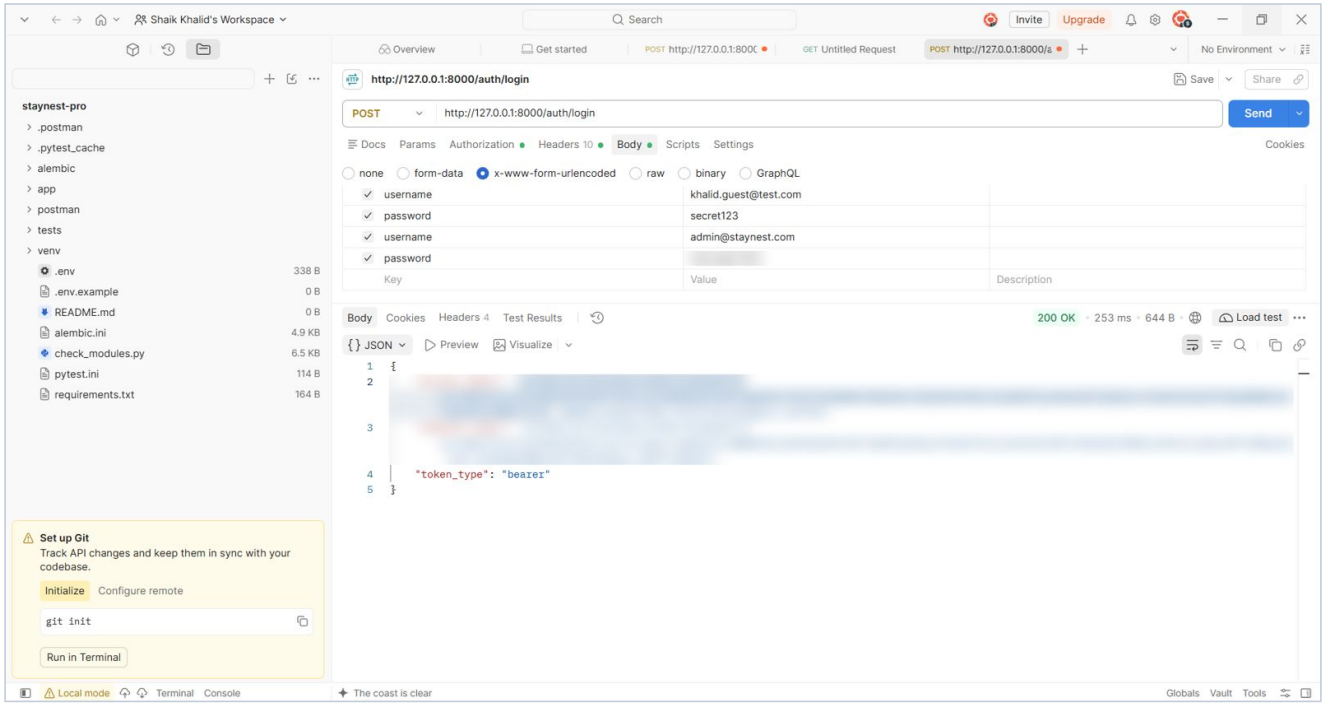
The user sends only their email, and the API always gives the same message so registered emails cannot be guessed.
A short-lived reset_token is returned for testing, because no email server is set up yet (token blurred).

Figure 24. Reset password - POST /auth/reset-password



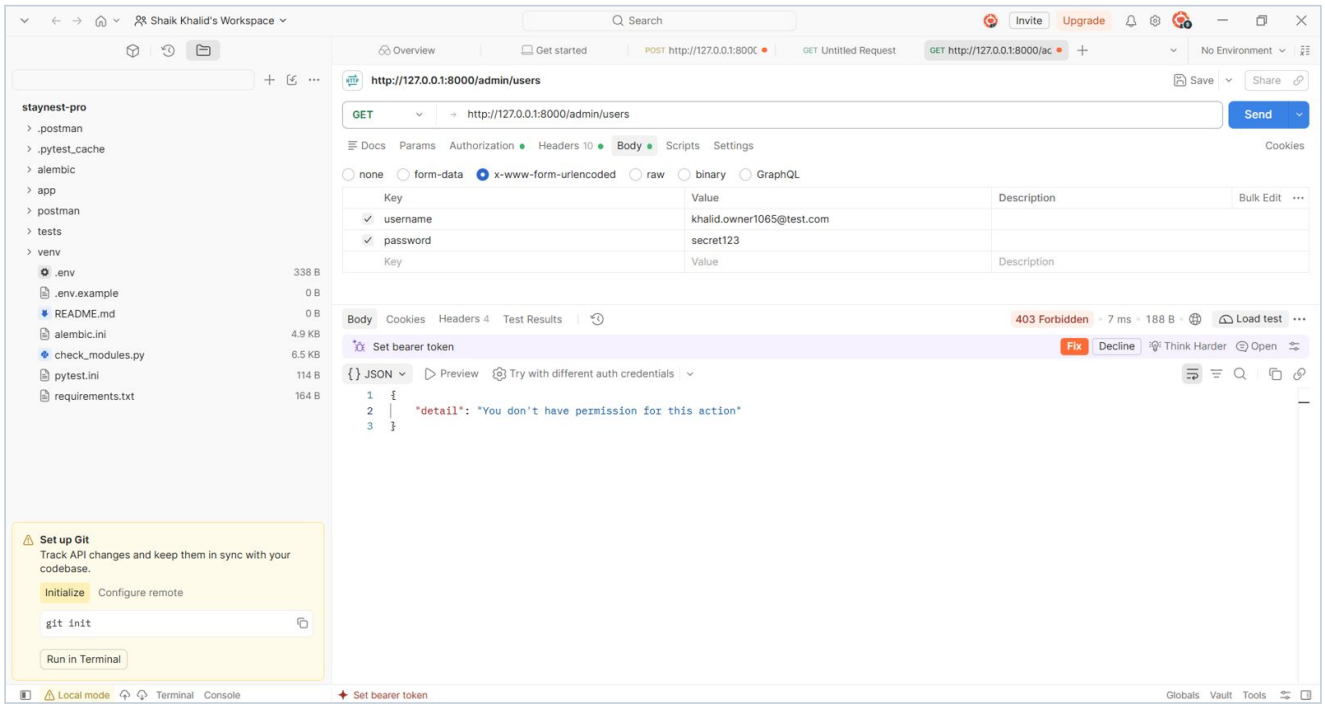
The reset token and a new password are sent together in the request body.
The API answers 200 OK, Password changed. Please log in again, and logs the user out of all devices.

Figure 25. Postman - admin login



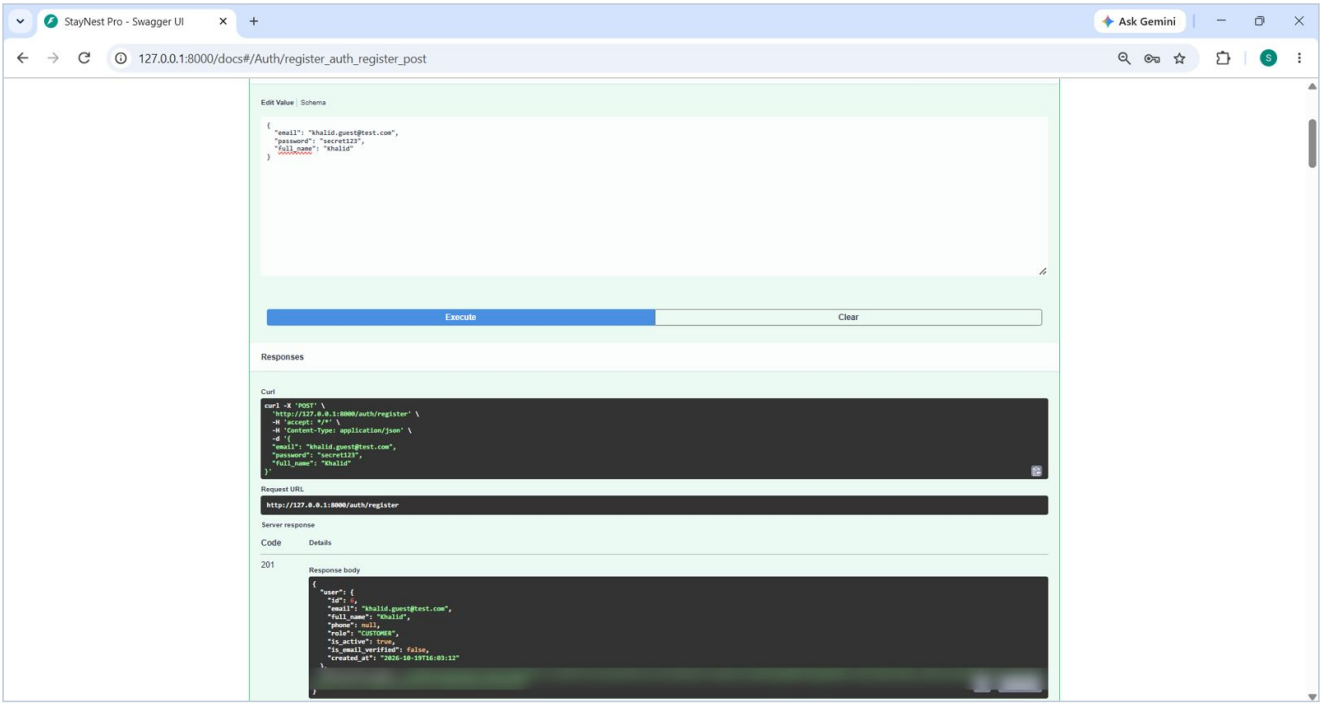
POST /auth/login with x-www-form-urlencoded username and password.
200 OK returns the access and refresh tokens (blurred here).

Figure 26. Postman - RBAC check (403 Forbidden)



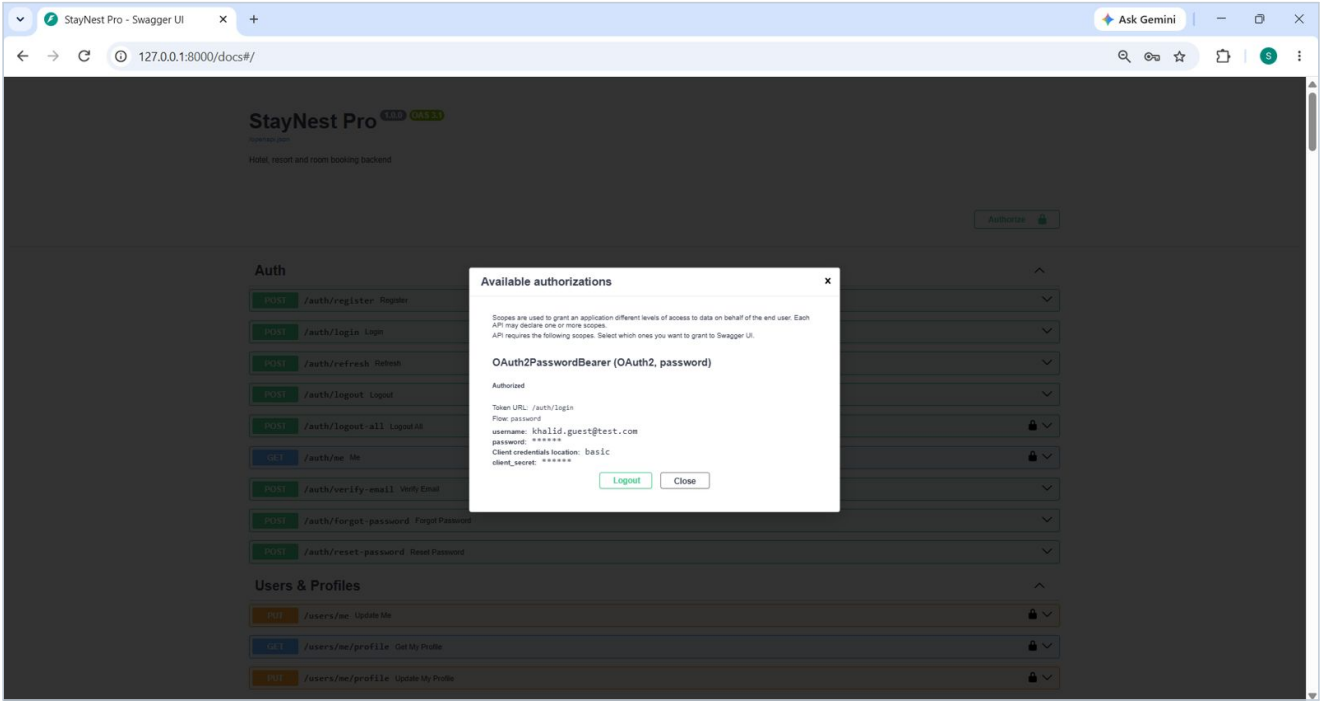
The owner's token is used on the admin-only GET /admin/users.
The API refuses with 403: You don't have permission for this action.

Figure 27. Register the test customer - khalid.guest@test.com



A second customer account is registered with only email, password and full name, since phone is optional. The 201 response returns user id 6 with role CUSTOMER, and the verification token is blurred.

Figure 28. Authorize as the customer

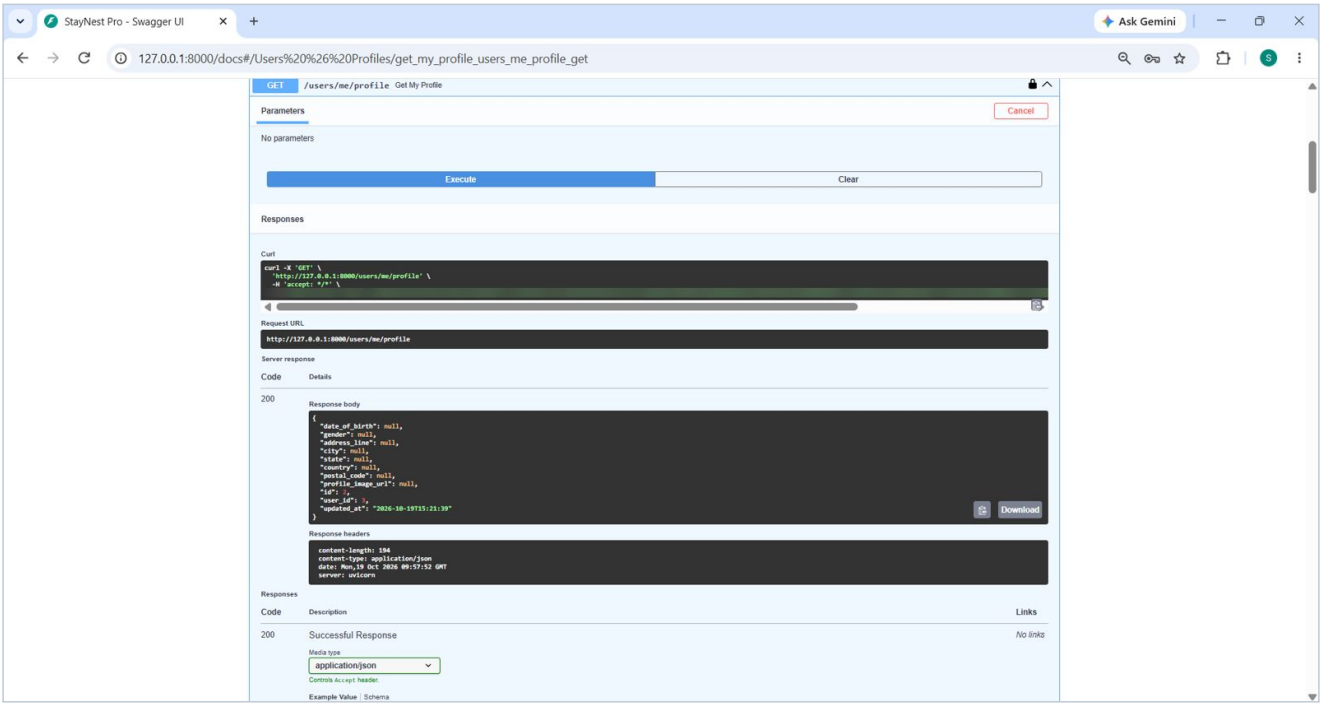


Swagger is authorized as khalid.guest@test.com through the OAuth2 password flow. All booking calls after this run with the customer's Bearer token, so RBAC applies customer rules.

Module 2. Customer Profile Management

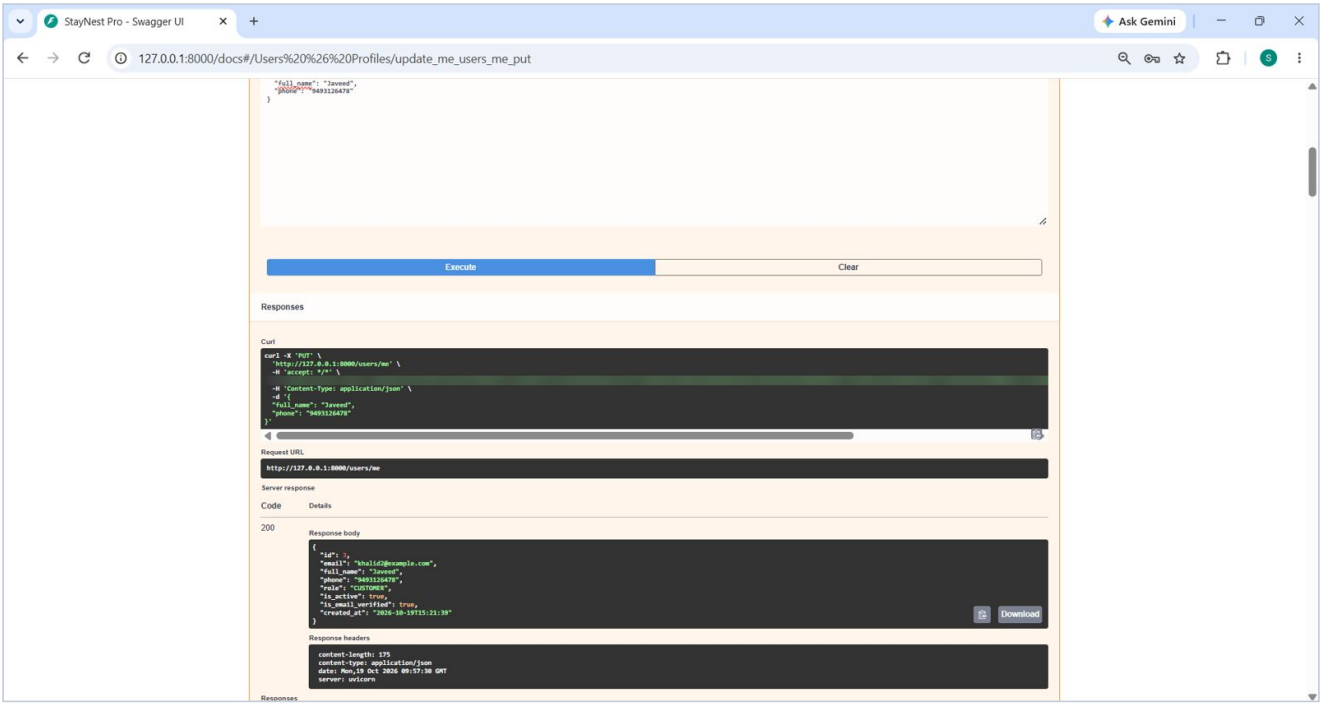
Day 1

Figure 29. Customer profile - GET /users/me/profile



A logged-in customer reads their own profile using the Bearer token.
The 200 response shows the profile fields (date of birth, gender, address, city) linked to user_id 3.

Figure 30. Update account - PUT /users/me

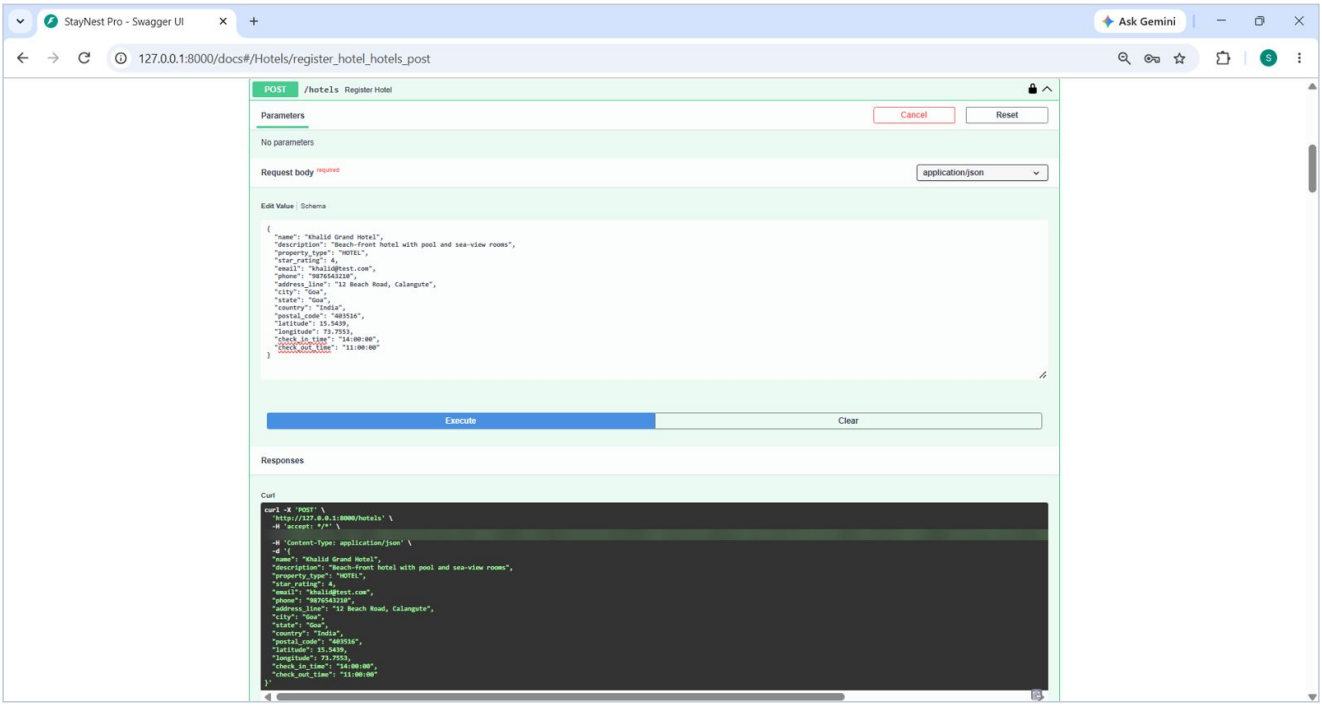


The customer changes their full name and phone number.
The 200 response returns the updated user with role CUSTOMER and is_email_verified true.

Module 3. Hotel Registration and Verification

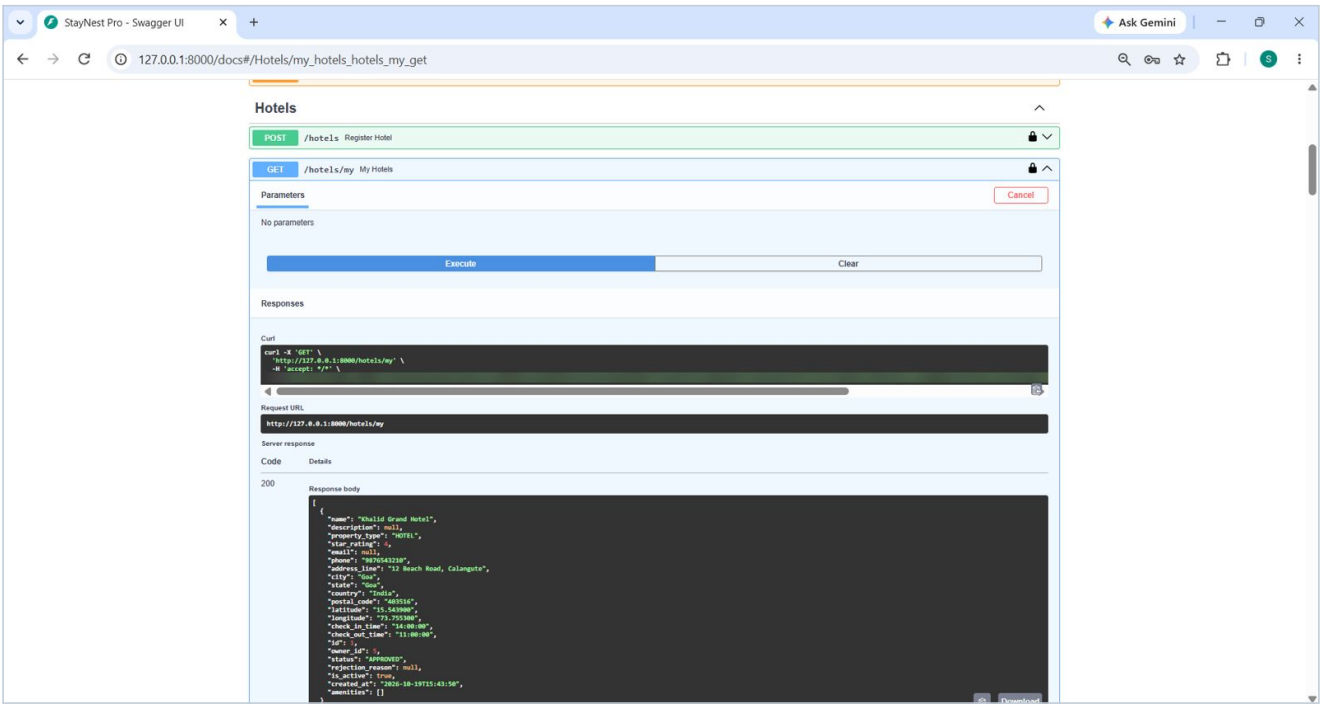
Day 1

Figure 31. Register a hotel - POST /hotels



A hotel owner registers Khalid Grand Hotel in Goa with type, star rating, contact, address, map location and check-in / check-out times.
Only HOTEL_OWNER users can call this, and the new hotel waits as PENDING until an admin approves it.

Figure 32. My hotels - GET /hotels/my



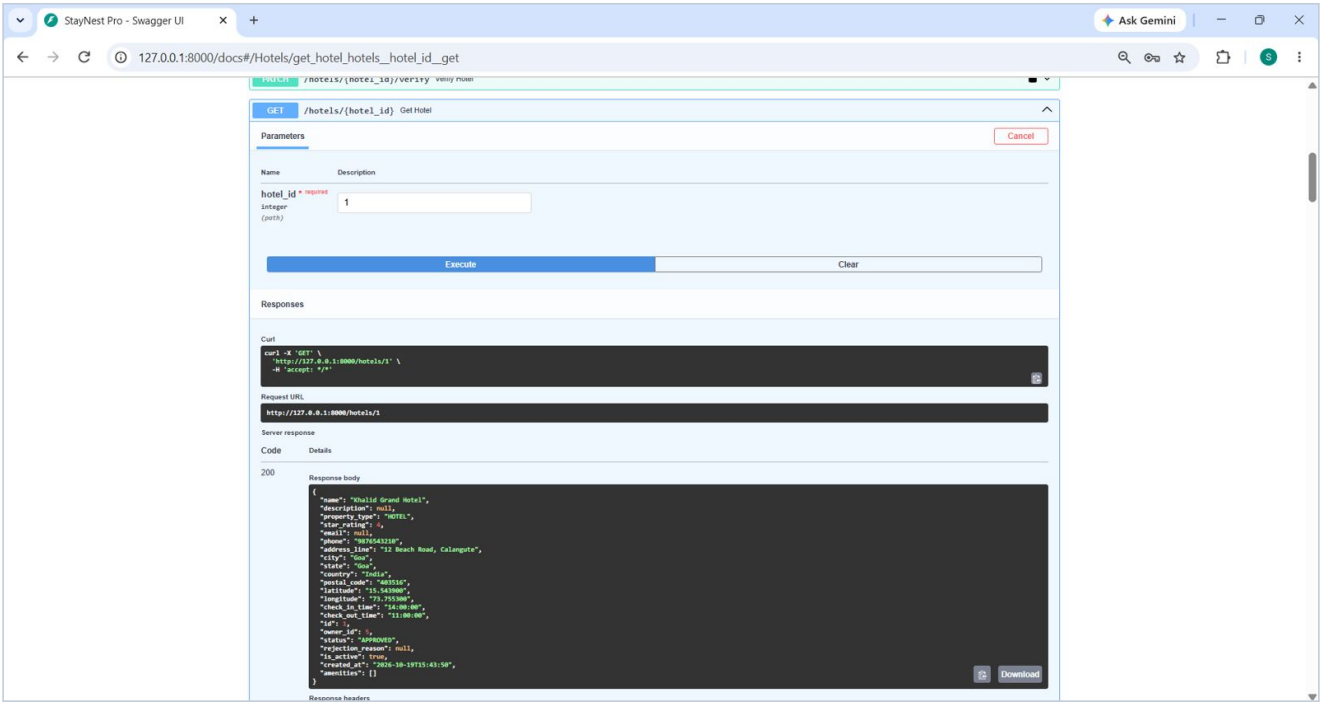
The owner lists only the hotels they own, using their Bearer token.
Khalid Grand Hotel (id 1, owner_id 5) is shown with status APPROVED and is_active true.

Module 4. Hotel Profile Management

Day 1

Also shown in Figure 32.

Figure 33. Hotel details - GET /hotels/1

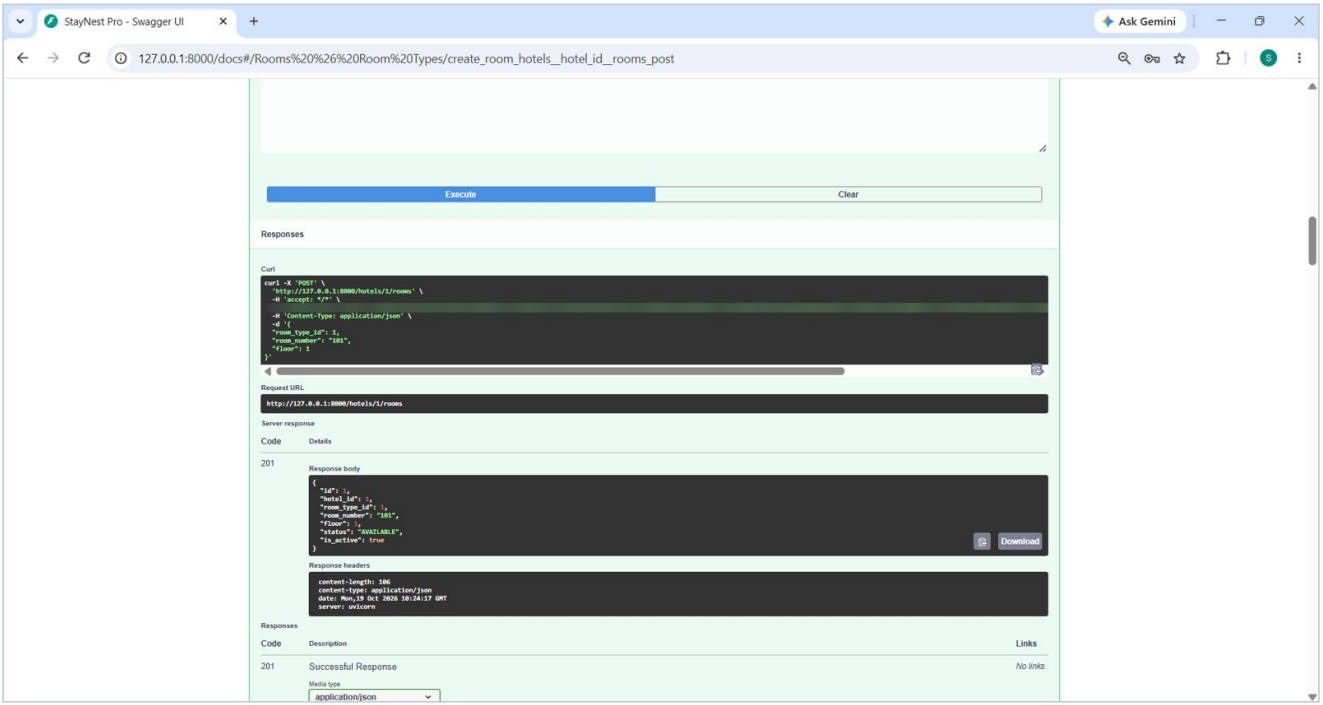


Anyone can read an approved hotel without logging in.
The 200 response returns the full hotel record, including city Goa, latitude, longitude and check-in time 14:00.

Module 5. Room Management

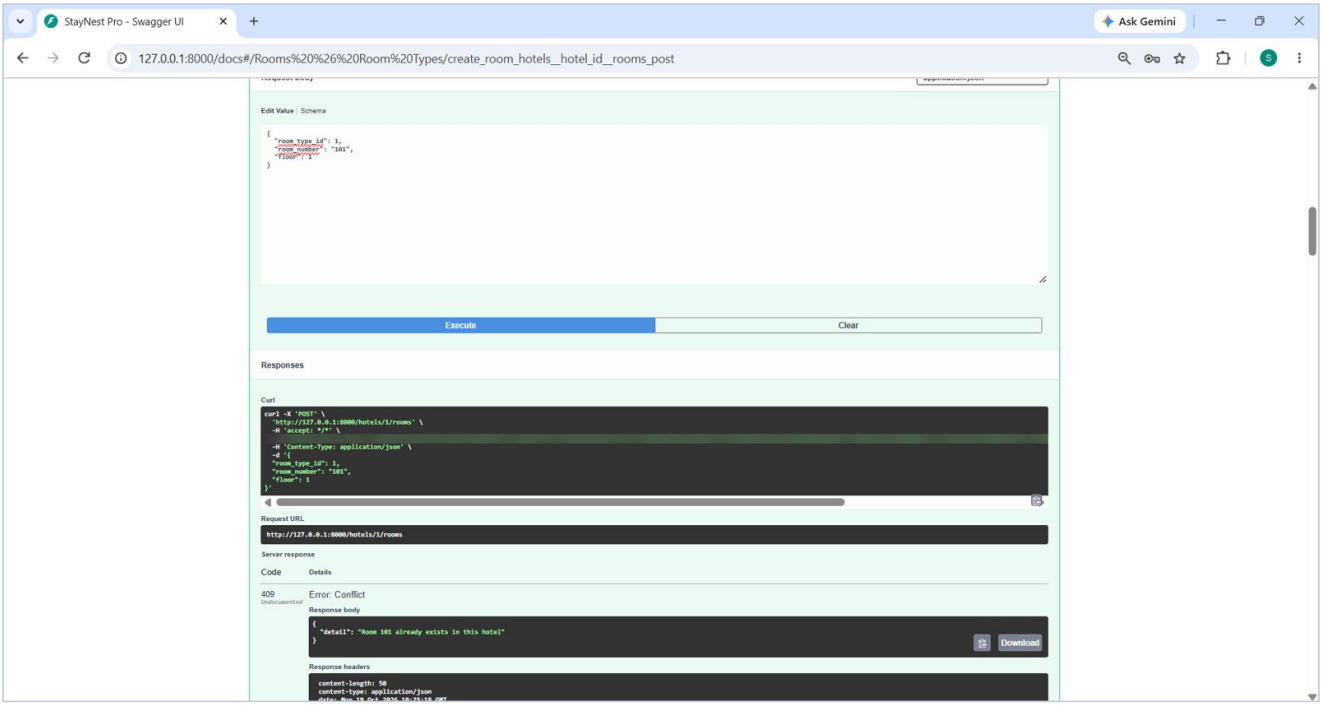
Day 1

Figure 34. Create a room - POST /hotels/1/rooms



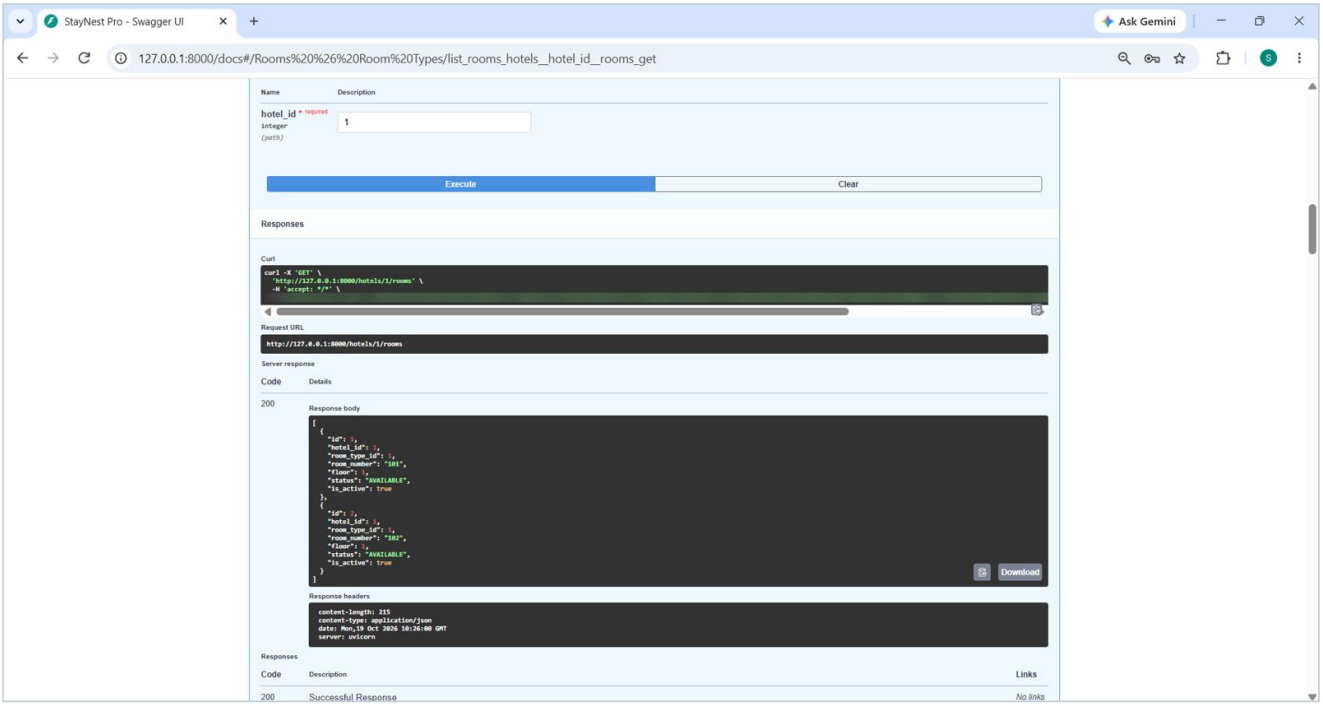
The owner adds room 101 on floor 1 under room type 1 (Deluxe Sea View).
The API returns 201 Created with room id 1 and status AVAILABLE.

Figure 35. Duplicate room check - 409 Conflict



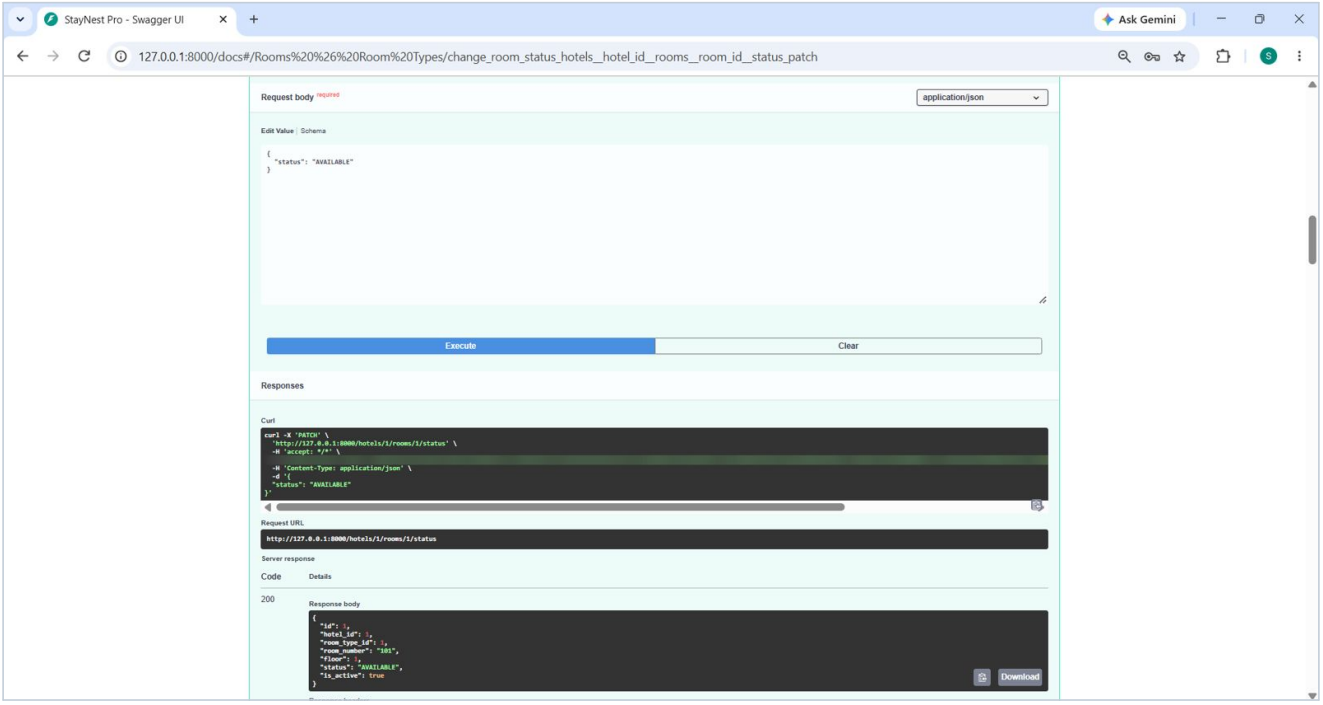
Sending room 101 a second time tests the unique room number rule.
The API refuses with 409 Conflict: Room 101 already exists in this hotel.

Figure 36. List rooms - GET /hotels/1/rooms



The owner lists all rooms of hotel 1.
Rooms 101 and 102 are both on floor 1, type 1, with status AVAILABLE.

Figure 37. Change room status - PATCH /hotels/1/rooms/1/status

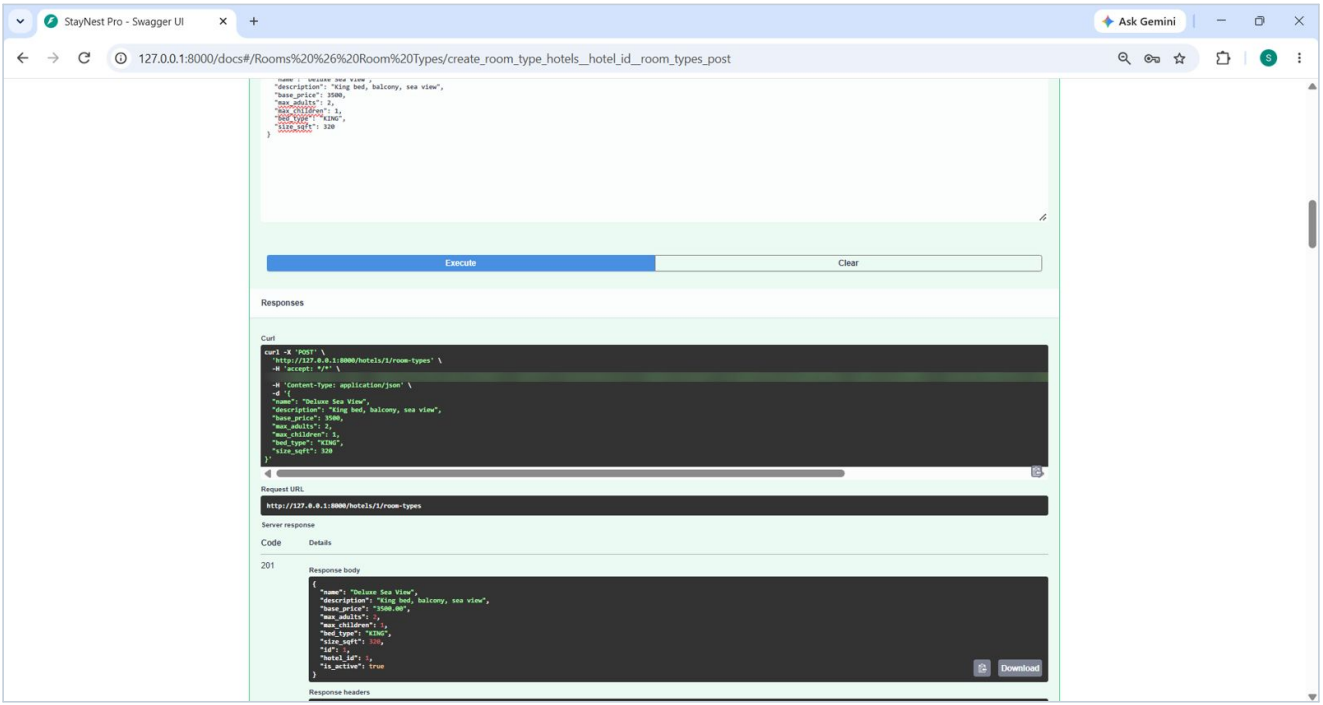


Staff or owner set a room's status, for example AVAILABLE, CLEANING or MAINTENANCE.
Room 101 is returned with status AVAILABLE and is_active true.

Module 6. Room Type Management

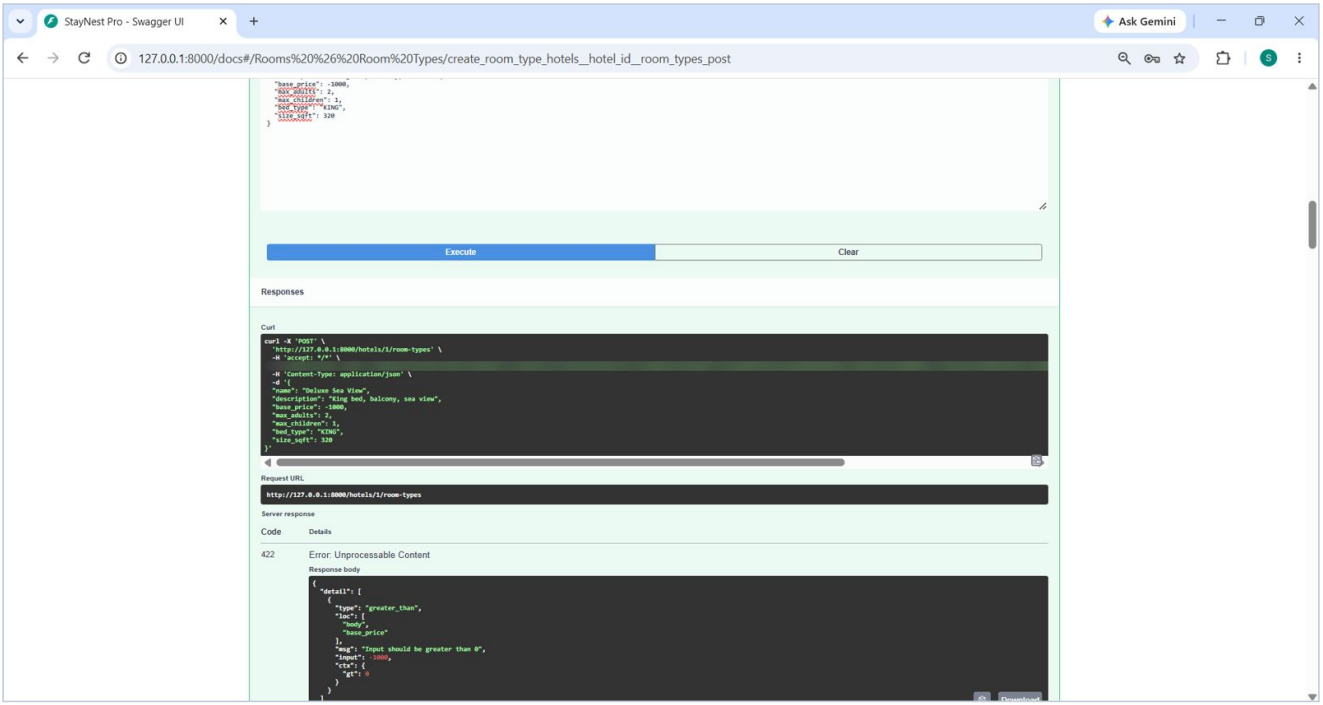
Day 1

Figure 38. Create a room type - POST /hotels/1/room-types



The owner adds Deluxe Sea View with a base price of Rs 3500, 2 adults, 1 child, a KING bed and 320 sq ft.
The API returns 201 Created with room type id 1 linked to hotel_id 1.

Figure 39. Validation check - negative price rejected

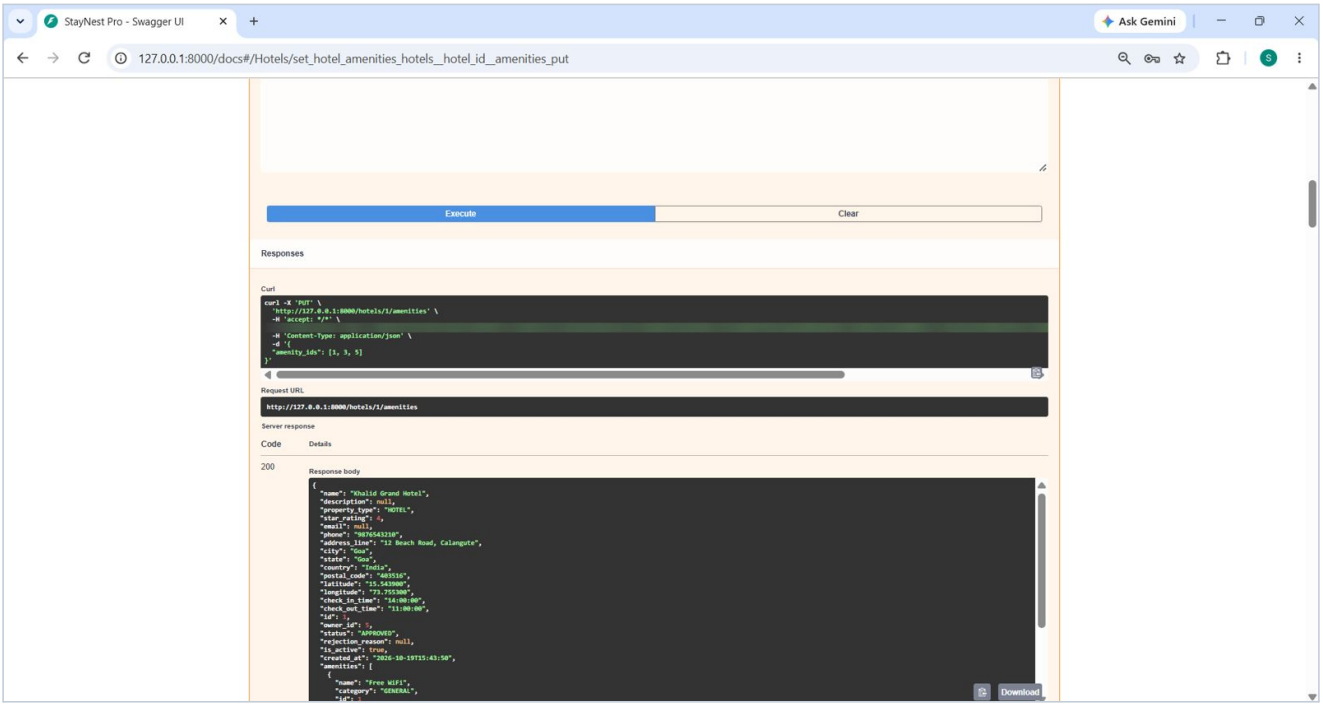


The same request with base_price -1000 is sent to test input validation.
Pydantic rejects it with 422 Unprocessable Content: Input should be greater than 0.

Module 7. Hotel Amenities Management

Day 1

Figure 40. Set hotel amenities - PUT /hotels/1/amenities

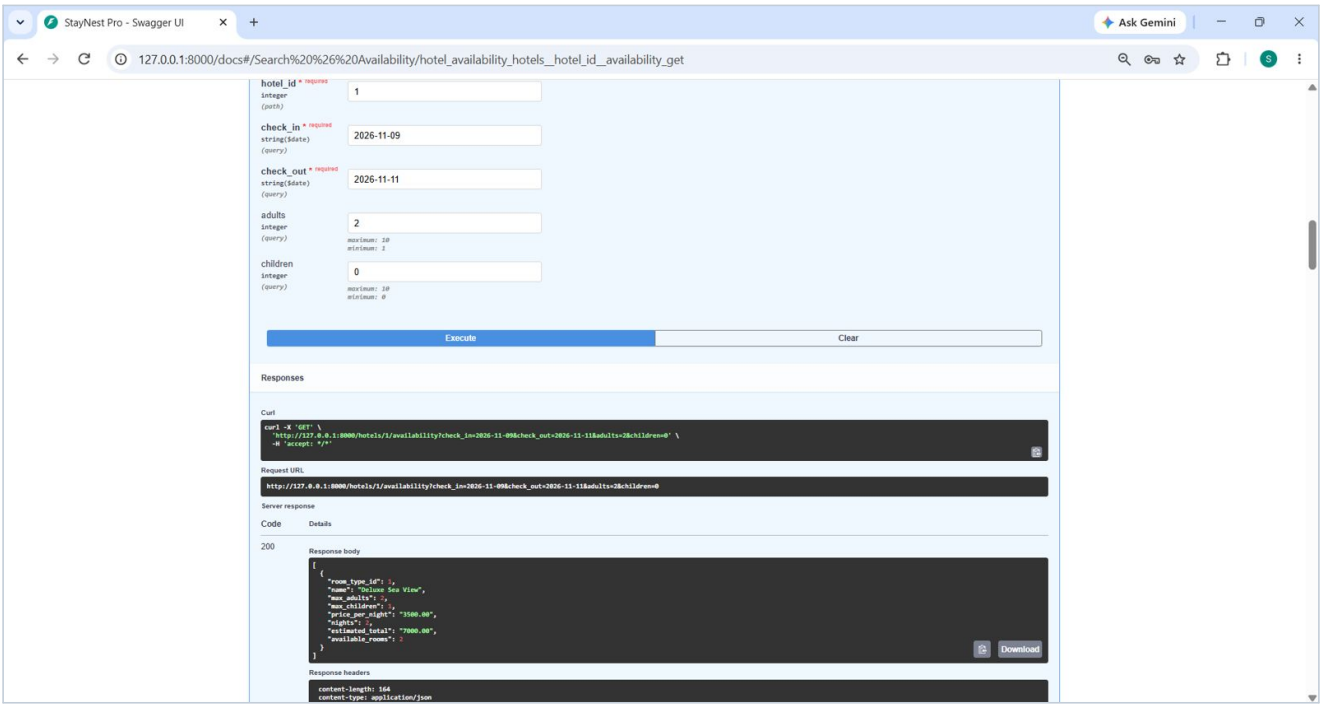


The owner links amenity ids 1, 3 and 5 to Khalid Grand Hotel in one request.
The 200 response returns the full hotel, now listing amenities such as Free WiFi under GENERAL.

Module 8. Room Availability Management

Day 2

Figure 41. Room availability - GET /hotels/1/availability



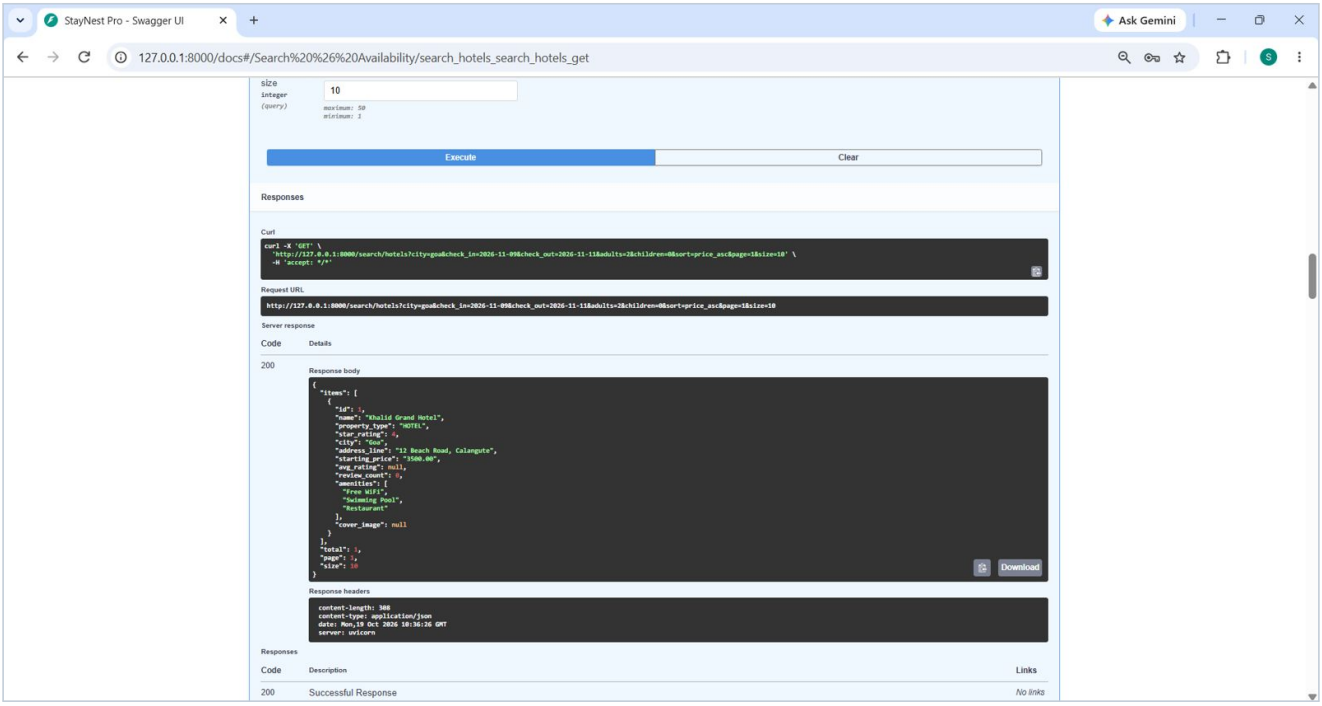
The guest checks free rooms for 9 to 11 November 2026 for 2 adults.
Deluxe Sea View shows 2 available rooms at Rs 3500 per night, Rs 7000 for 2 nights.

Module 9. Hotel Search System

Day 2

Also shown in .

Figure 42. Search hotels - GET /search/hotels



A public search for Goa from 9 to 11 November 2026 for 2 adults, sorted by lowest price.
Khalid Grand Hotel is found with a starting price of Rs 3500 and amenities Free WiFi, Swimming Pool and Restaurant.

Module 10. Advanced Hotel Filters

Day 2

Also shown in Figure 42.

Module 11. Hotel Details and Gallery

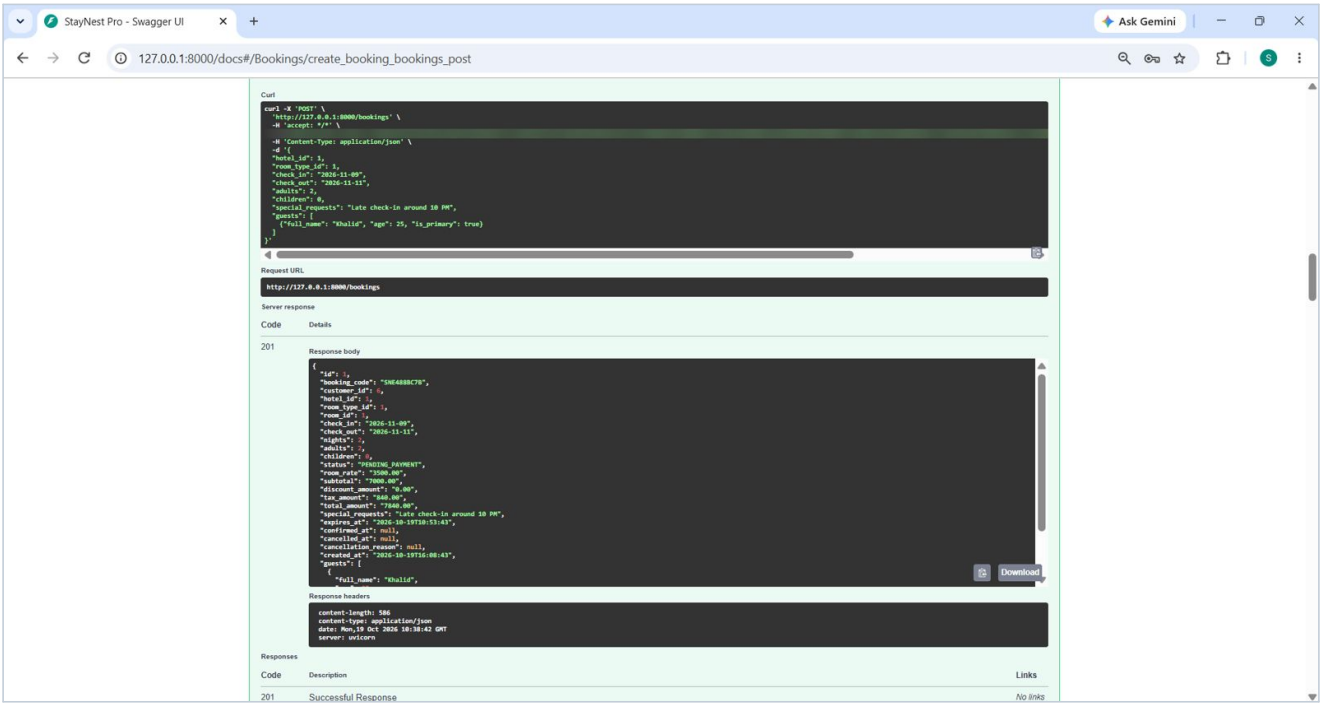
Day 2

Also shown in Figure 33.

Module 12. Room Booking System

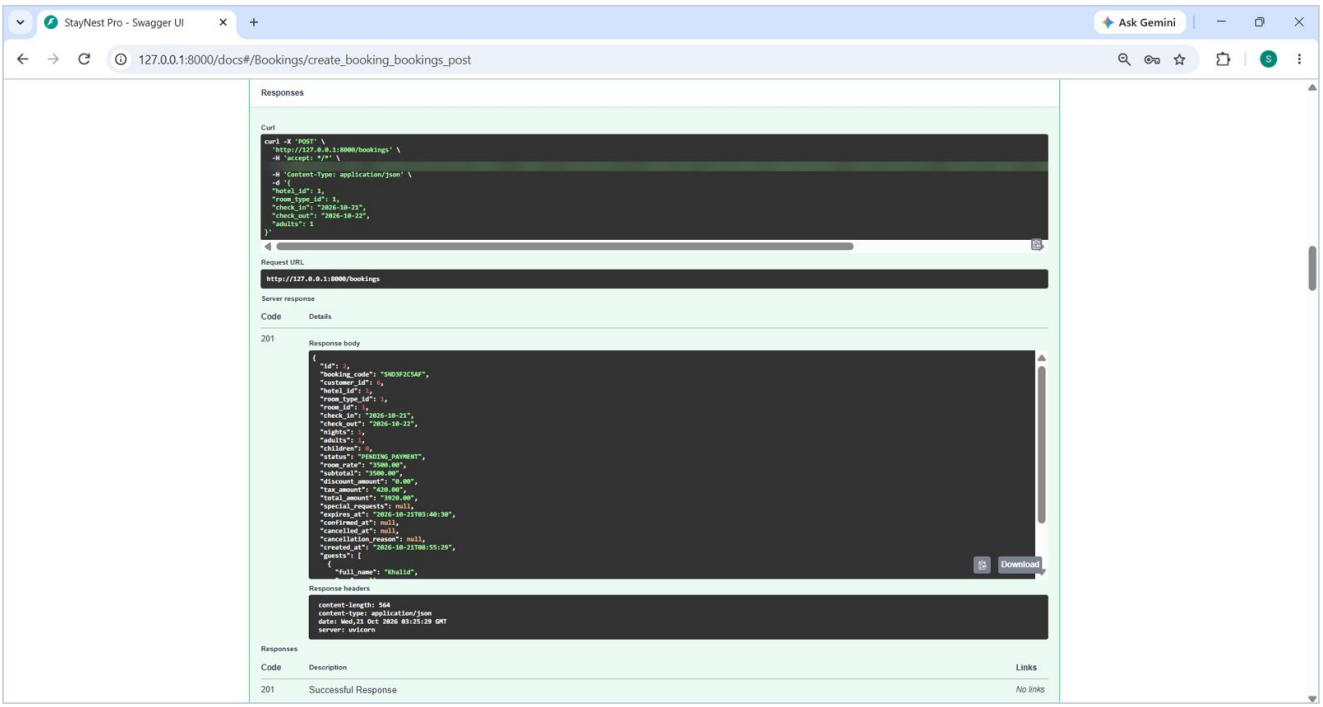
Day 2

Figure 43. Create a booking - POST /bookings (2 nights)



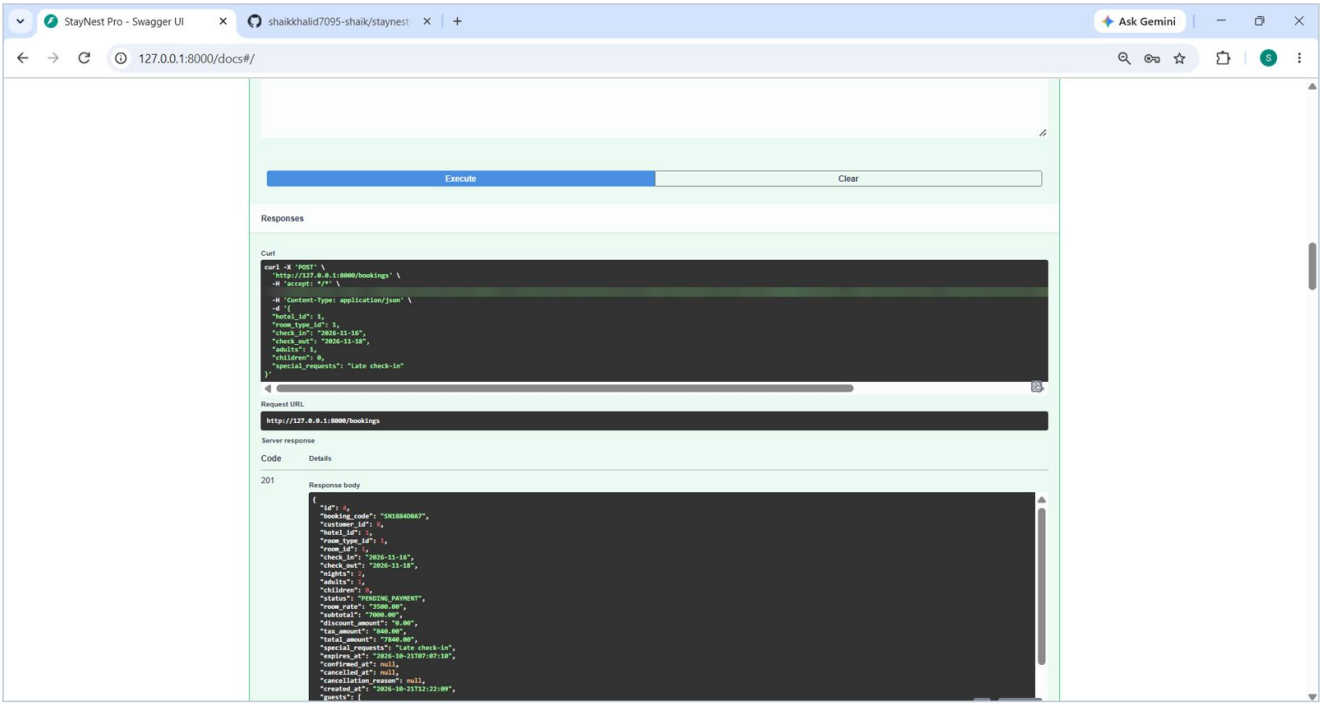
The customer books Deluxe Sea View from 9 to 11 November 2026 for 2 adults with a late check-in request. Booking SNE488BC7B is PENDING_PAYMENT: subtotal Rs 7000 + 12% GST Rs 840 = Rs 7840, held for 15 minutes.

Figure 44. Create a booking - POST /bookings (1 night)



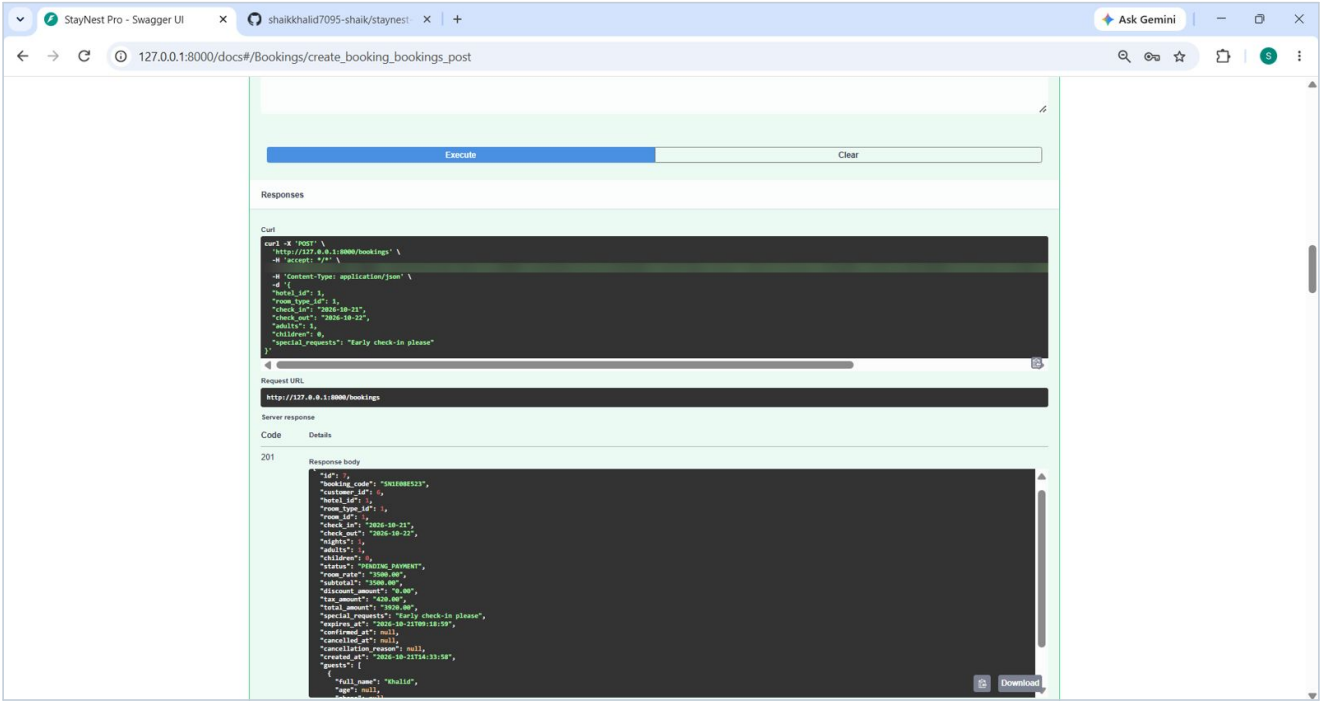
A second booking for one night, 21 to 22 October 2026, for 1 adult.
Booking SND3F2C5AF is PENDING_PAYMENT: Rs 3500 + GST Rs 420 = Rs 3920, with room 1 assigned automatically.

Figure 45. Step 1 - create a booking (201 Created)



Customer 8 books Deluxe Sea View from 16 to 18 November 2026 with a late check-in request.
Booking 4 (SN1884D0A7) is PENDING_PAYMENT at Rs 7000 + Rs 840 GST = Rs 7840, held for 15 minutes.

Figure 46. Booking for today - POST /bookings

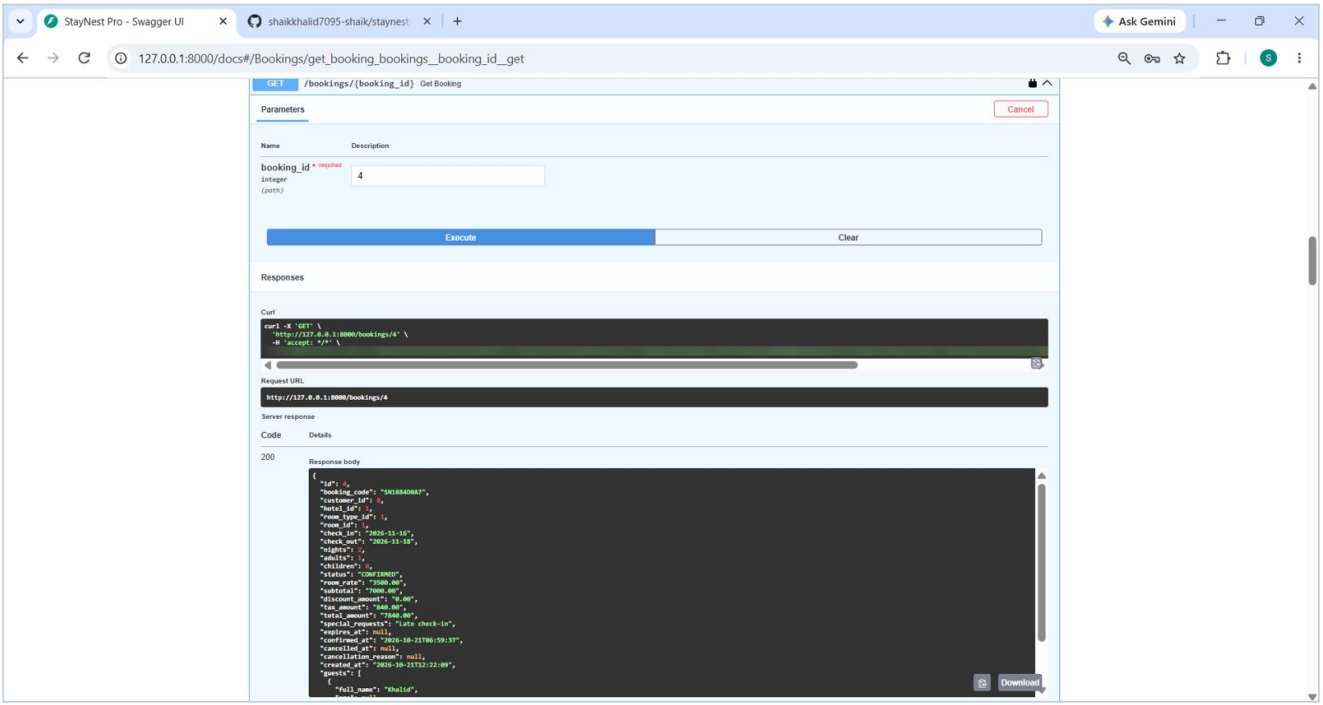


The customer books one night, 21 to 22 October 2026, with an early check-in request.
Booking 7 is PENDING_PAYMENT: Rs 3500 + Rs 420 GST = Rs 3920.

Module 13. Guest Management

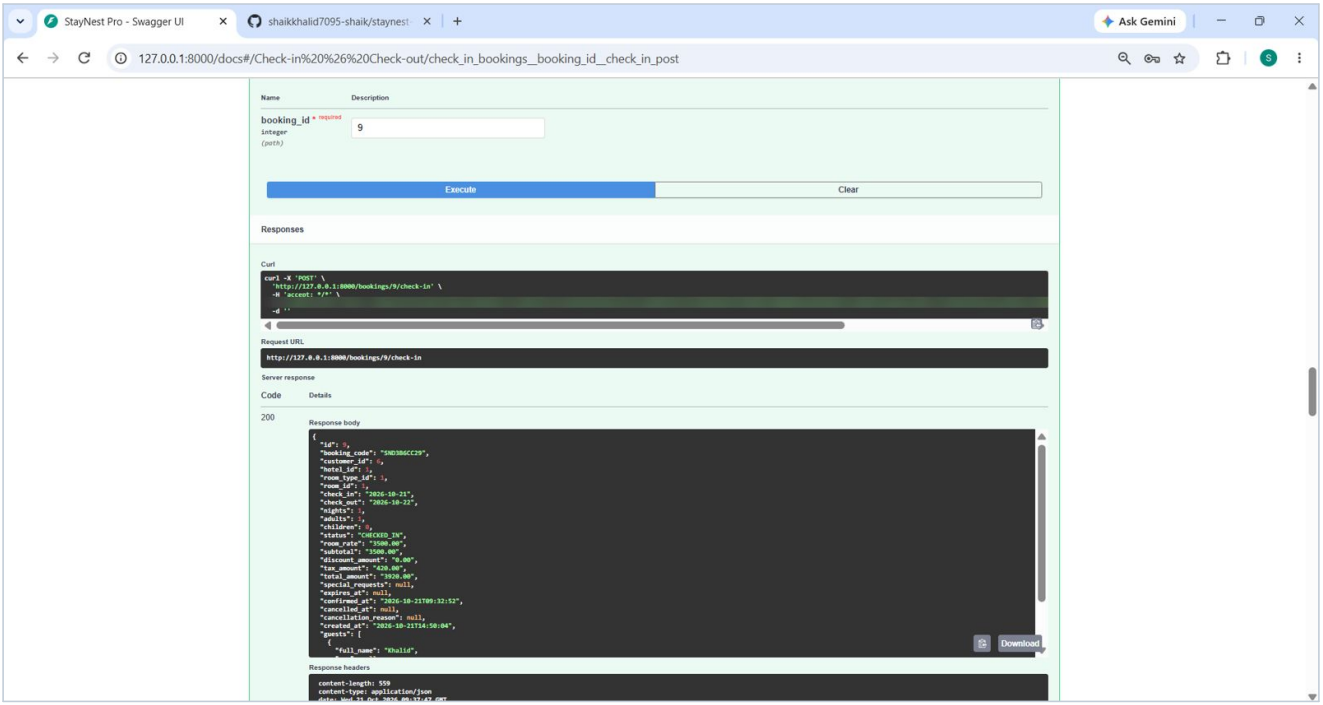
Day 2

Figure 47. Step 5 - booking is confirmed



GET /bookings/4 is called after the payment.
Status is CONFIRMED with confirmed_at filled, and expires_at is cleared.

Figure 48. Check-in - POST /bookings/9/check-in



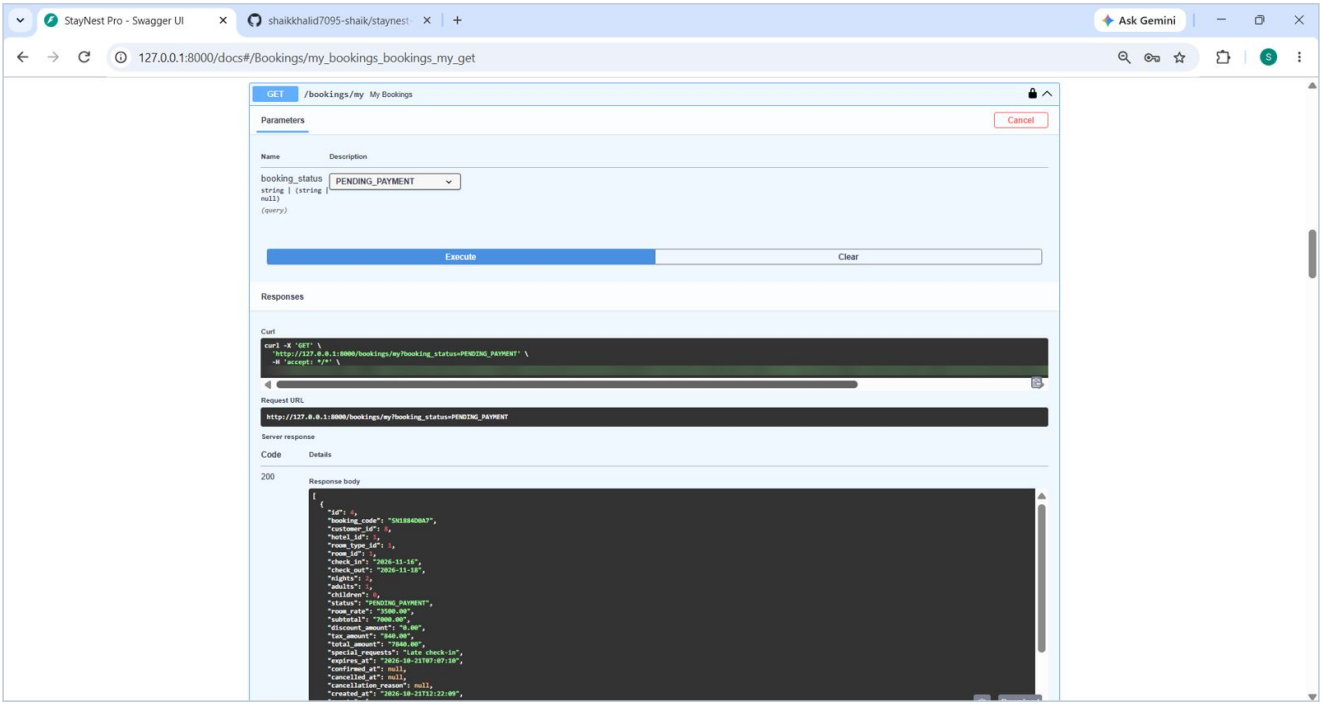
The hotel owner checks the guest in on the arrival date.
Booking 9 status changes to CHECKED_IN, with room 1 assigned.

Module 14. Booking Management

Day 2

Also shown in Figure 47.

Figure 49. Step 2 - find it in my bookings

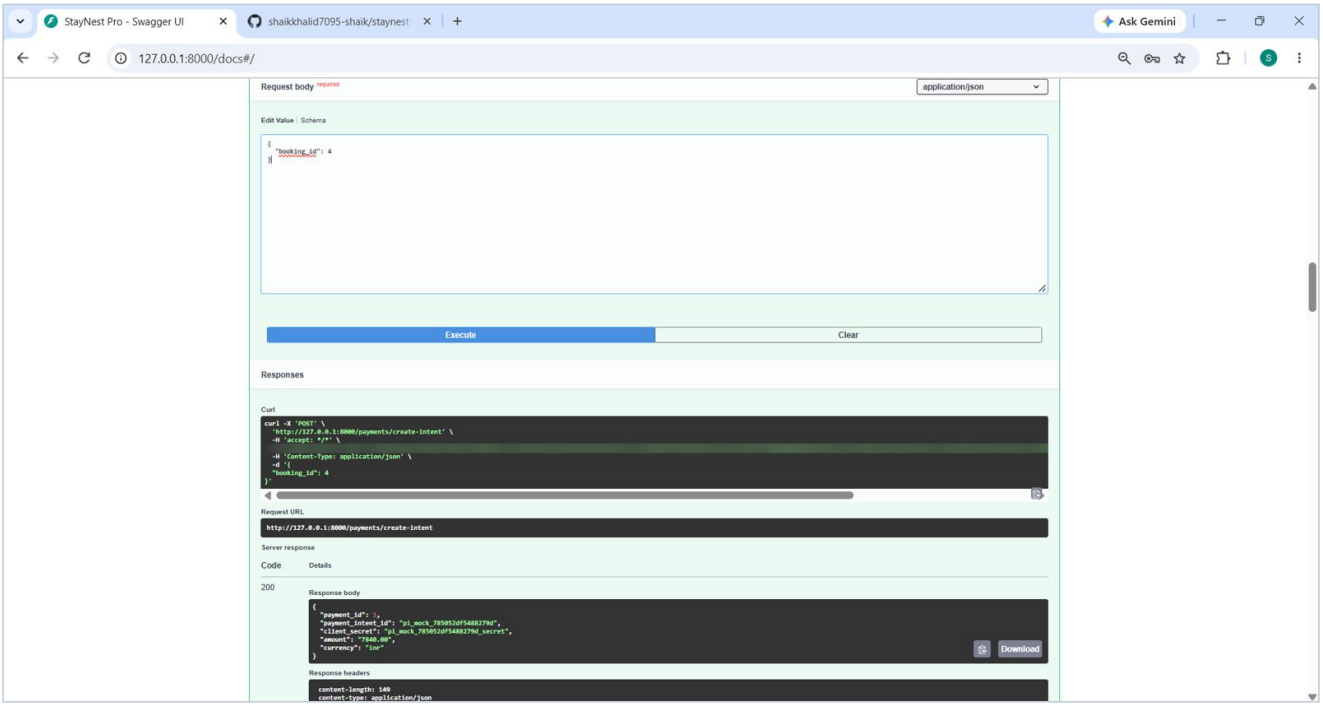


GET /bookings/my is filtered by status PENDING_PAYMENT.
Booking 4 is listed with its expiry time, so its id can be used to pay.

Module 15. Stripe Payment Integration

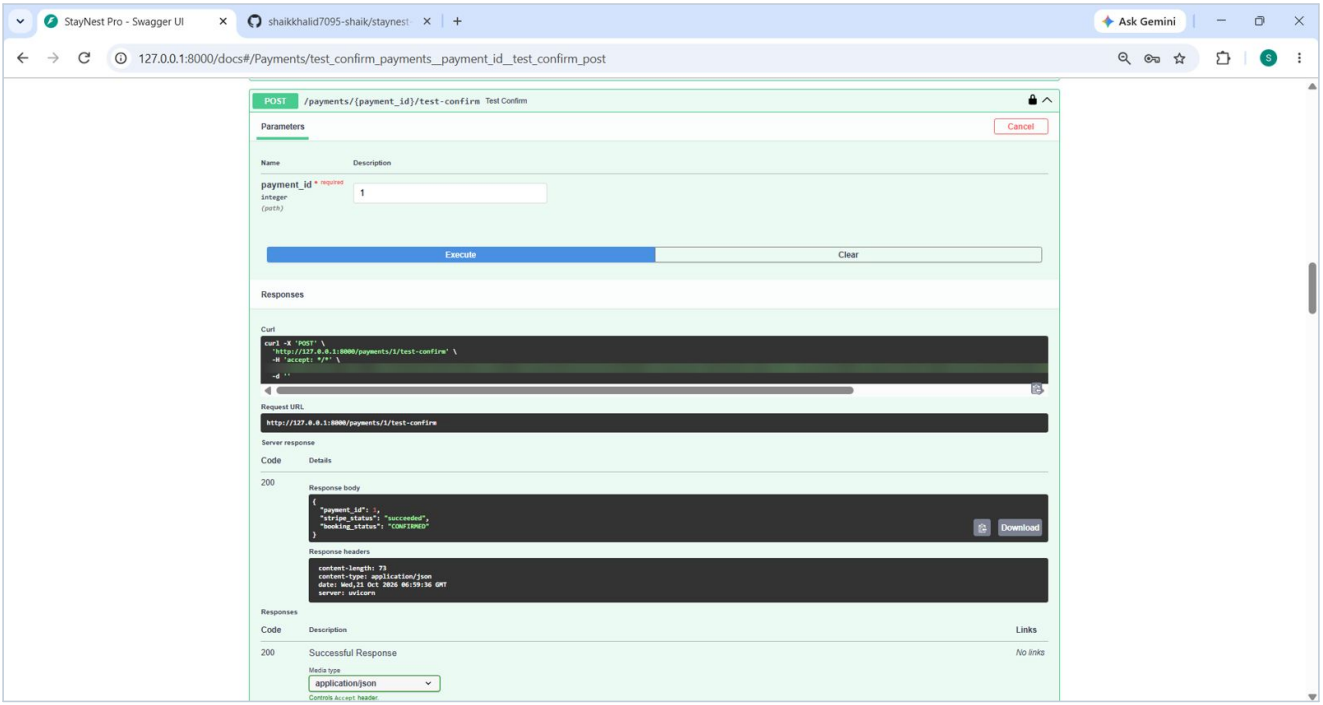
Day 2

Figure 50. Step 3 - start payment (POST /payments/create-intent)



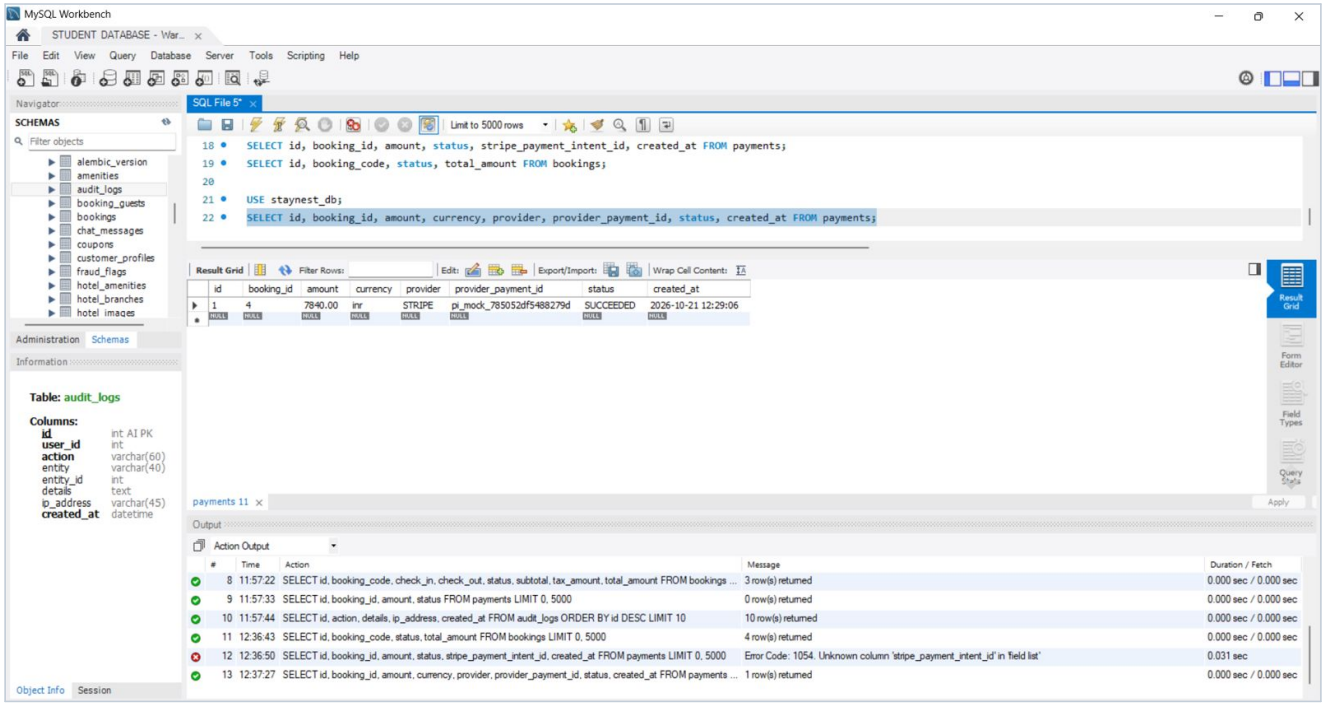
The customer sends booking_id 4 to start a Stripe payment.
Mock Stripe returns payment 1 with intent pi_mock_785052df5488279d for Rs 7840 in INR.

Figure 51. Step 4 - confirm payment (test-confirm)



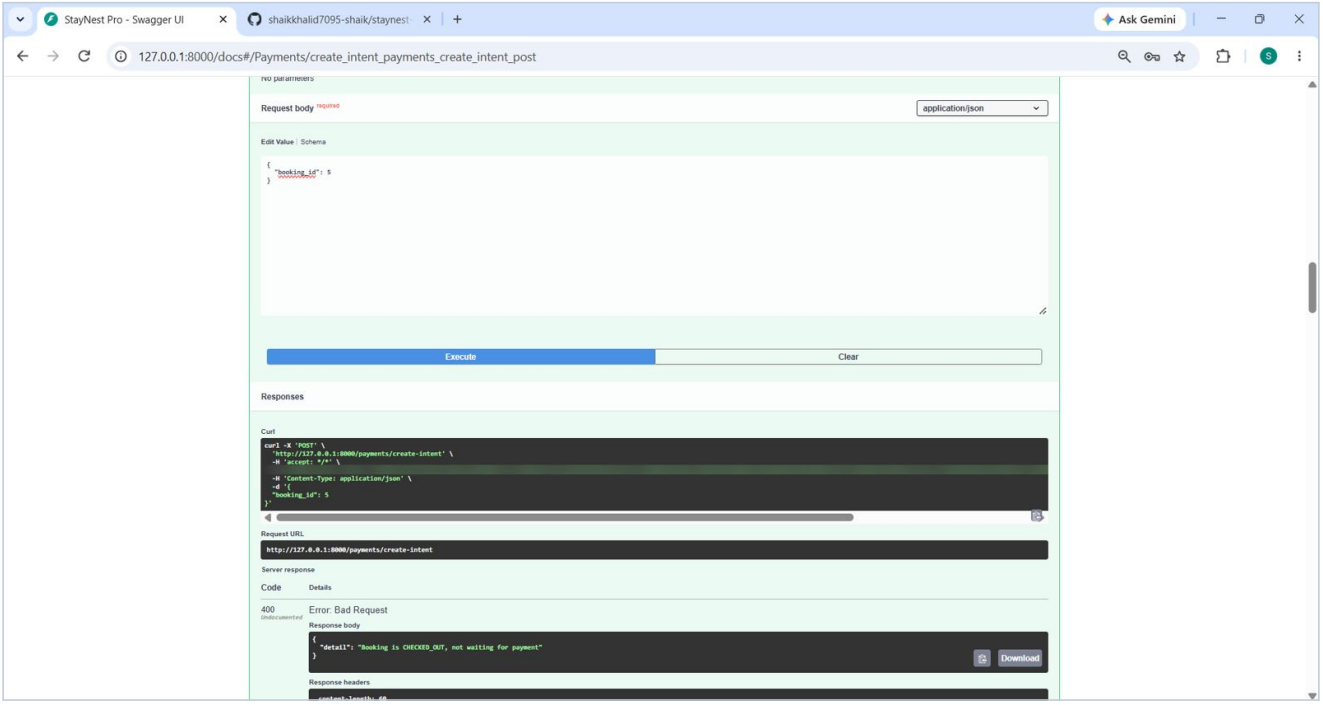
POST /payments/1/test-confirm marks the payment as paid, which works only in DEBUG mode.
Stripe status is succeeded and the booking status becomes CONFIRMED.

Figure 52. Step 7 - payment saved in MySQL



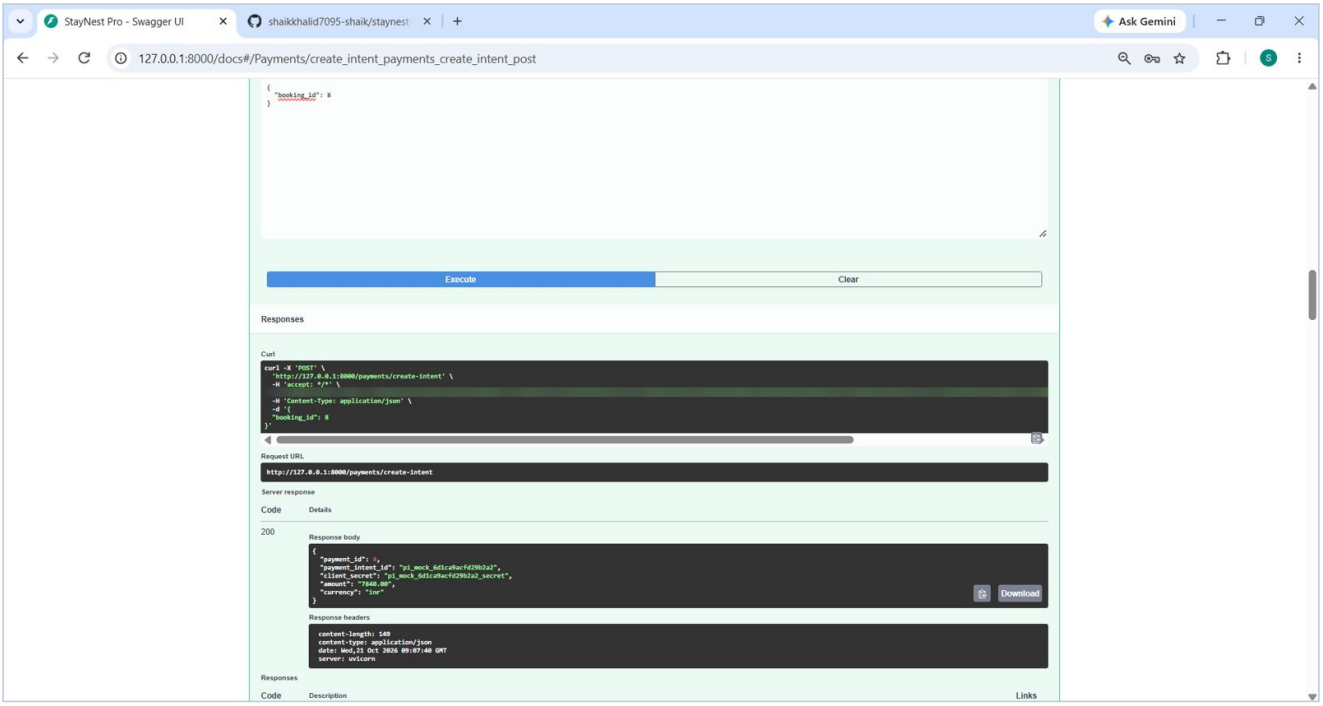
The payments table is queried in Workbench after paying.
Payment 1 for booking 4: Rs 7840.00 INR, provider STRIPE, pi_mock id, status SUCCEEDED.

Figure 53. Payment rule - only unpaid bookings



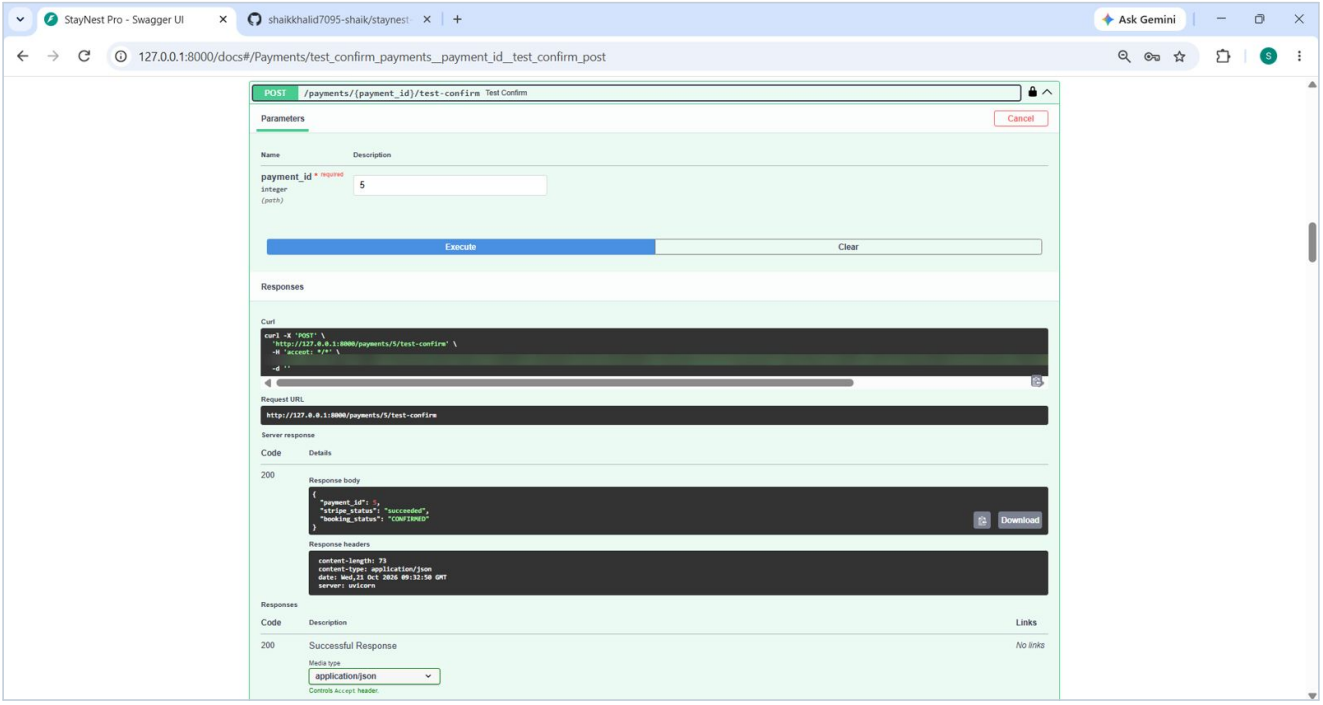
create-intent is tried on booking 5, which has already been checked out.
The API refuses with 400: Booking is CHECKED_OUT, not waiting for payment.

Figure 54. Refund flow 2 - start payment



POST /payments/create-intent for booking 8.
Mock Stripe returns payment 4 with a pi_mock intent for Rs 7840.

Figure 55. Pay today's booking - test-confirm

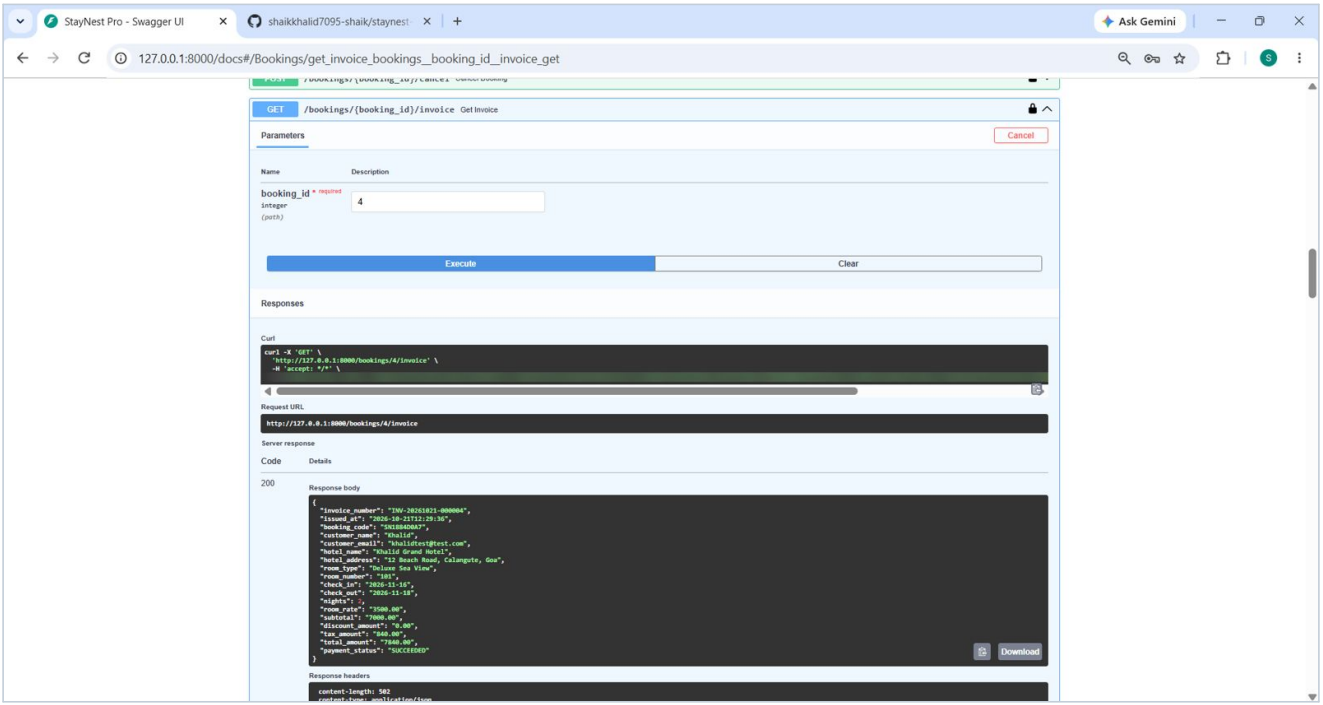


POST /payments/5/test-confirm pays booking 9 (one night, 21 to 22 October 2026).
Stripe status succeeded, and booking 9 becomes CONFIRMED, ready for check-in.

Module 16. Invoice and Receipt Management

Day 2

Figure 56. Step 6 - invoice (GET /bookings/4/invoice)

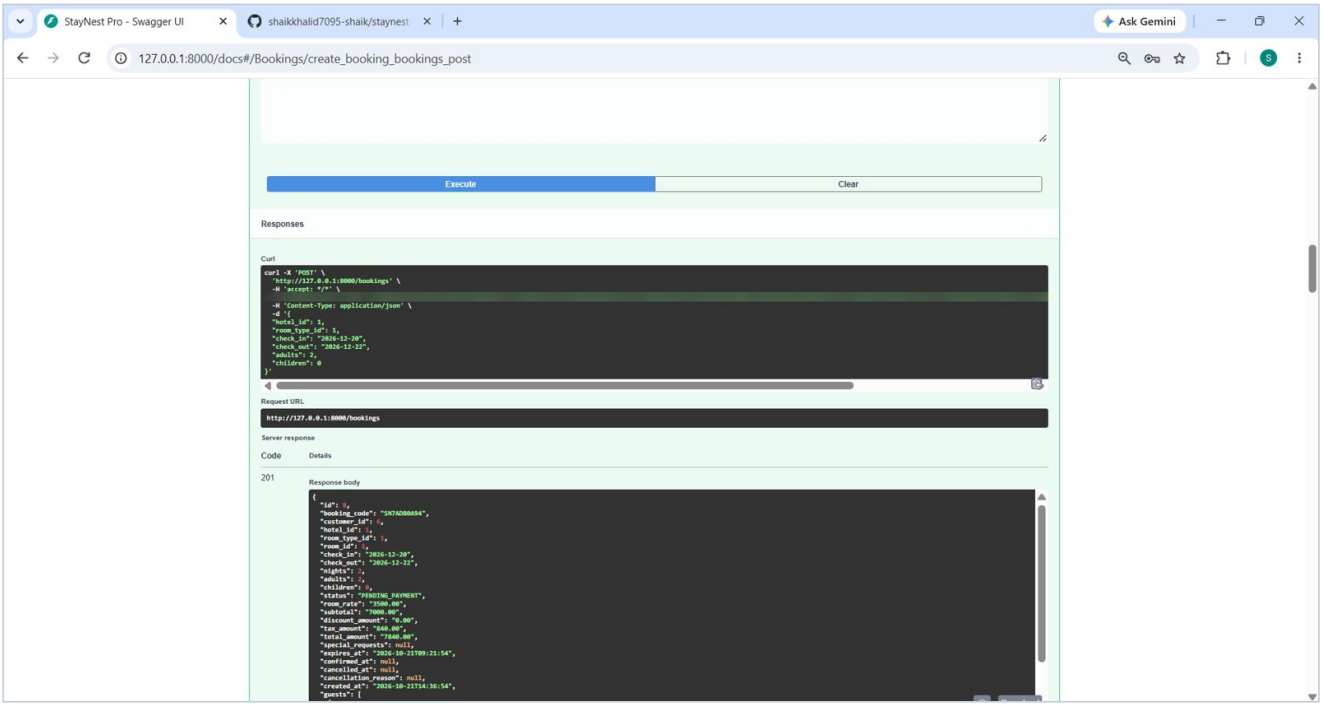


An invoice is generated for the paid booking.
INV-20261021-000004 lists the guest, hotel, room 101, 2 nights, Rs 7000 + Rs 840 GST = Rs 7840, payment SUCCEEDED.

Module 17. Cancellation and Refund Management

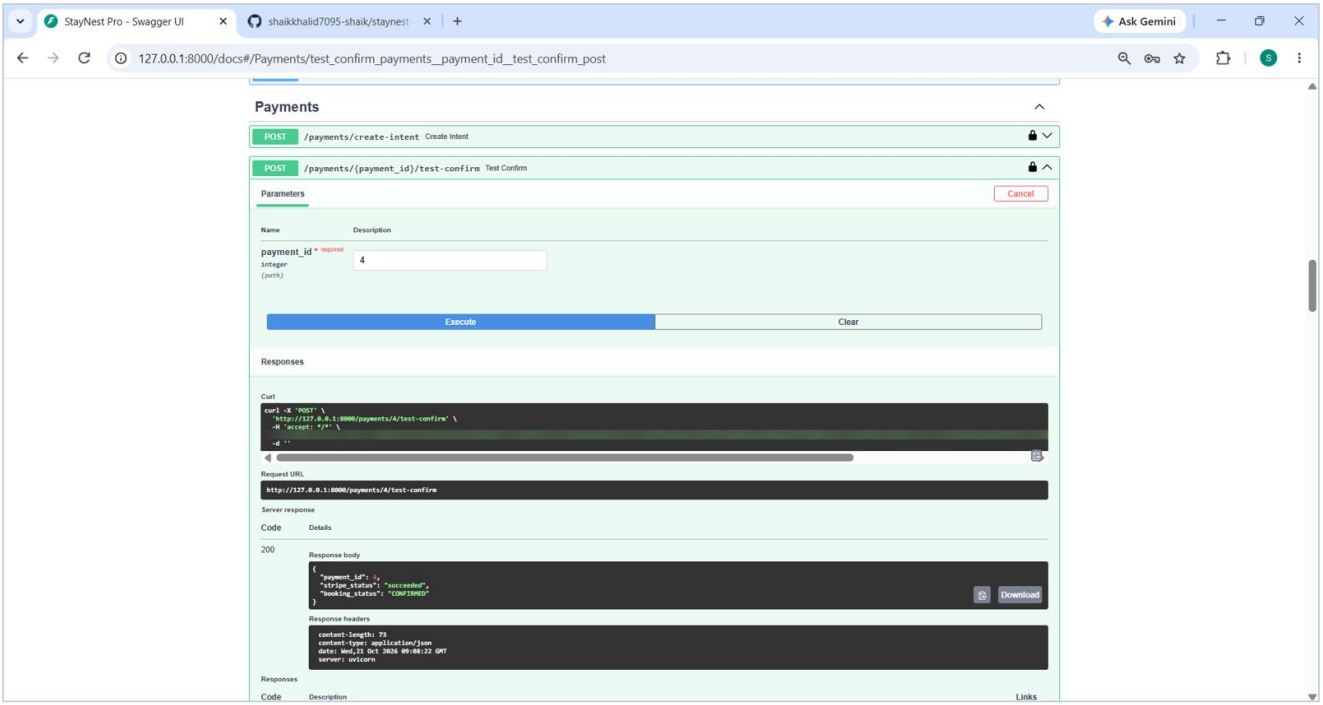
Day 2

Figure 57. Refund flow 1 - book a December stay



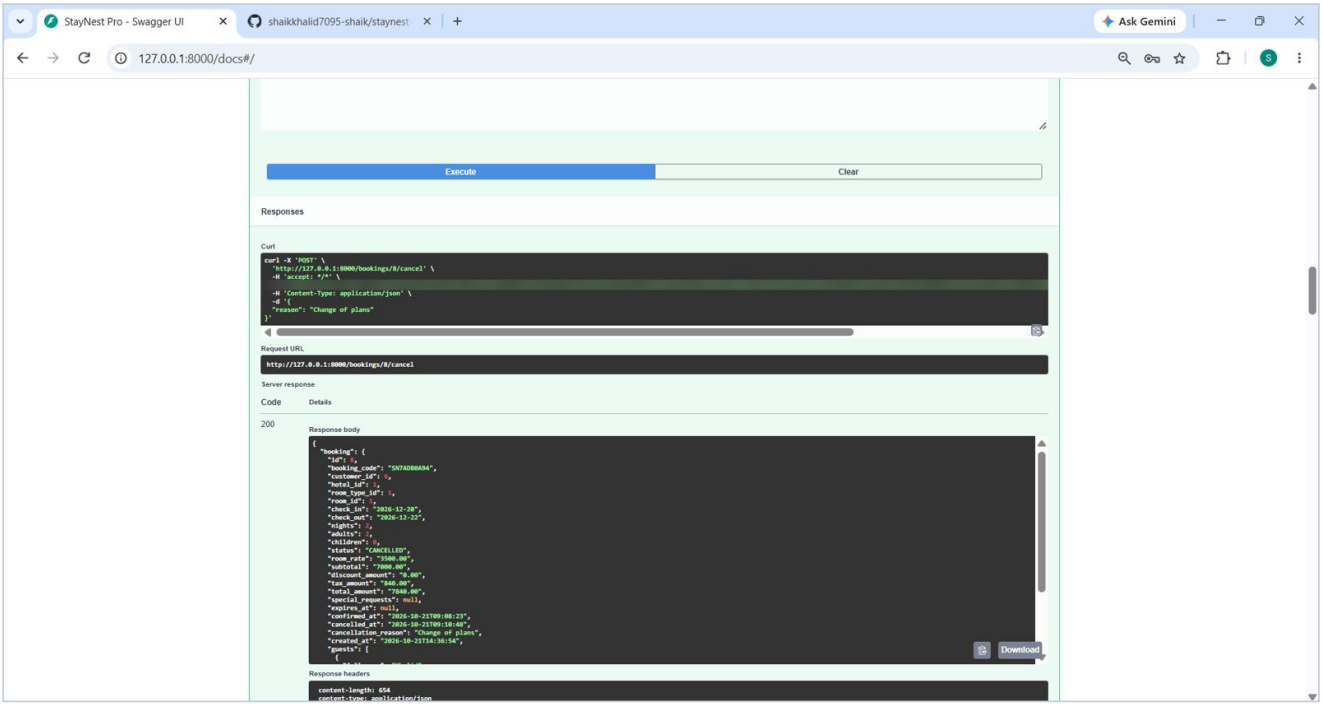
The customer books 20 to 22 December 2026 for 2 adults.
Booking 8 (SN7ADB0A94) is PENDING_PAYMENT at Rs 7840.

Figure 58. Refund flow 3 - confirm payment



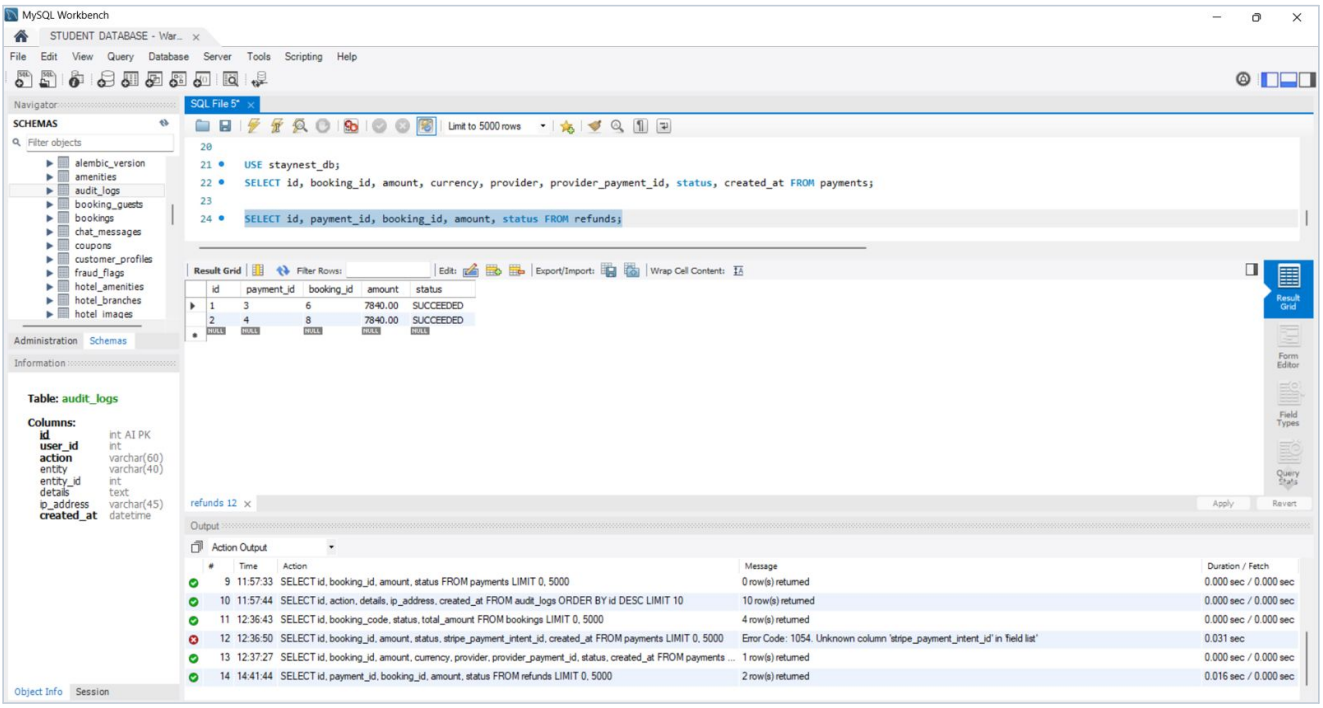
POST /payments/4/test-confirm marks the payment as paid.
Stripe status succeeded, and booking 8 becomes CONFIRMED.

Figure 59. Refund flow 4 - cancel booking



POST /bookings/8/cancel with the reason Change of plans.
Booking 8 is CANCELLED, with cancelled_at and the reason saved, and the refund is issued.

Figure 60. Refunds saved in MySQL

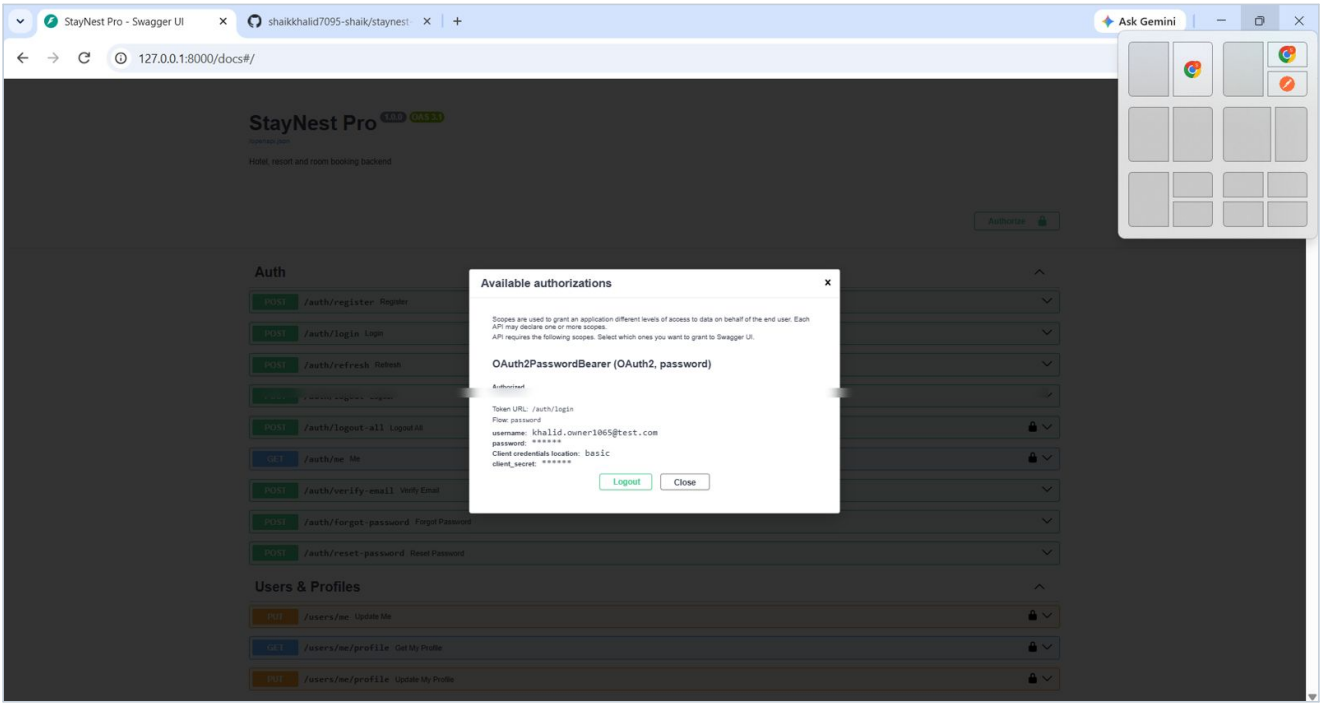


The refunds table is queried in Workbench.
2 refunds of Rs 7840 each for bookings 6 and 8, both SUCCEEDED.

Module 18. Coupon and Discount Management

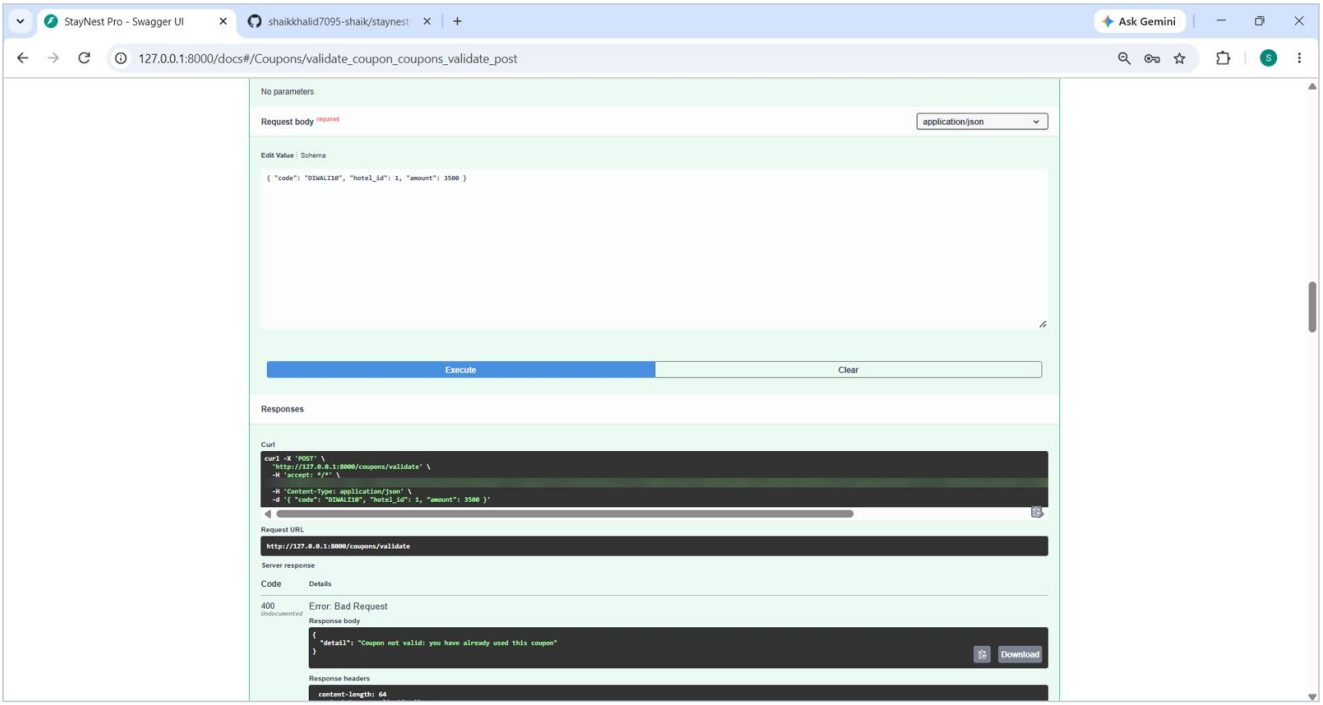
Day 2

Figure 61. Create a coupon - POST /coupons



The hotel owner creates DIWALI10, a 10% coupon capped at Rs 1000, valid for October to December 2026.
201 Created with usage_limit 100, per_user_limit 1 and hotel_id 1.

Figure 62. Coupon rule - one use per customer

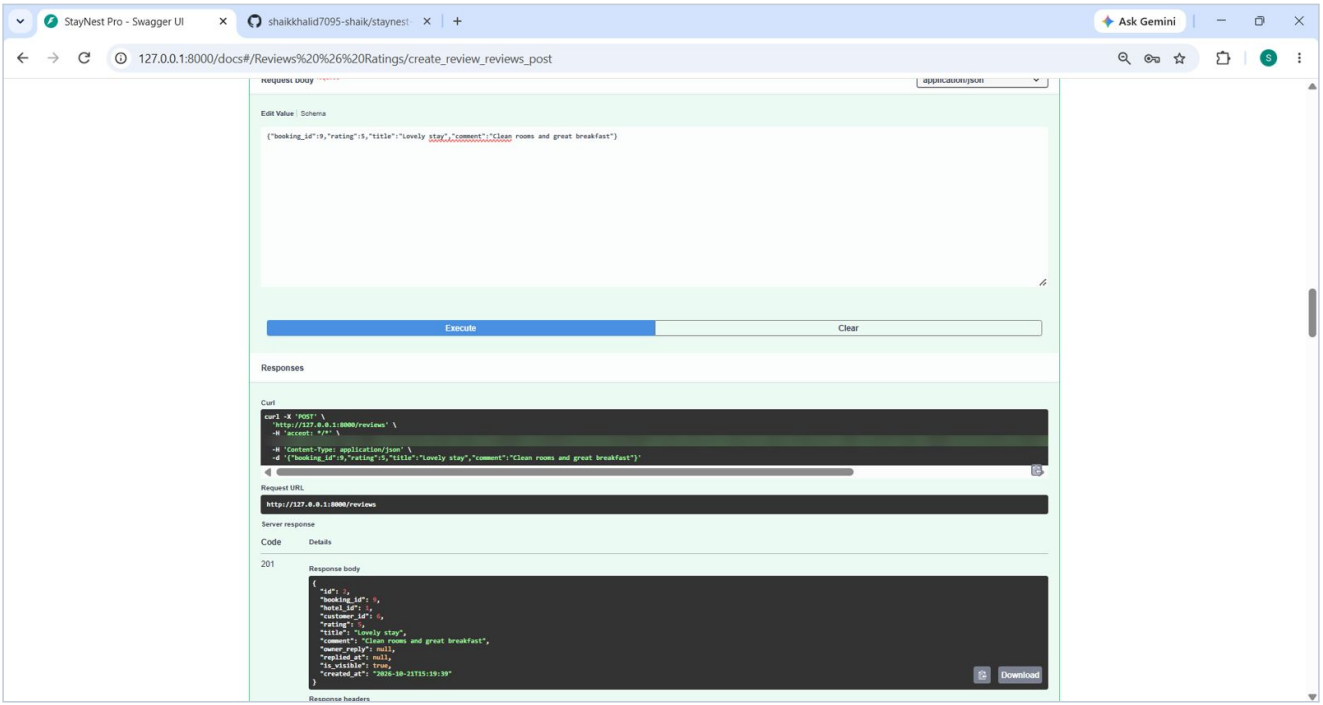


The customer validates DIWALI10 after already using it once.
The API refuses with 400: Coupon not valid: you have already used this coupon.

Module 19. Reviews and Ratings

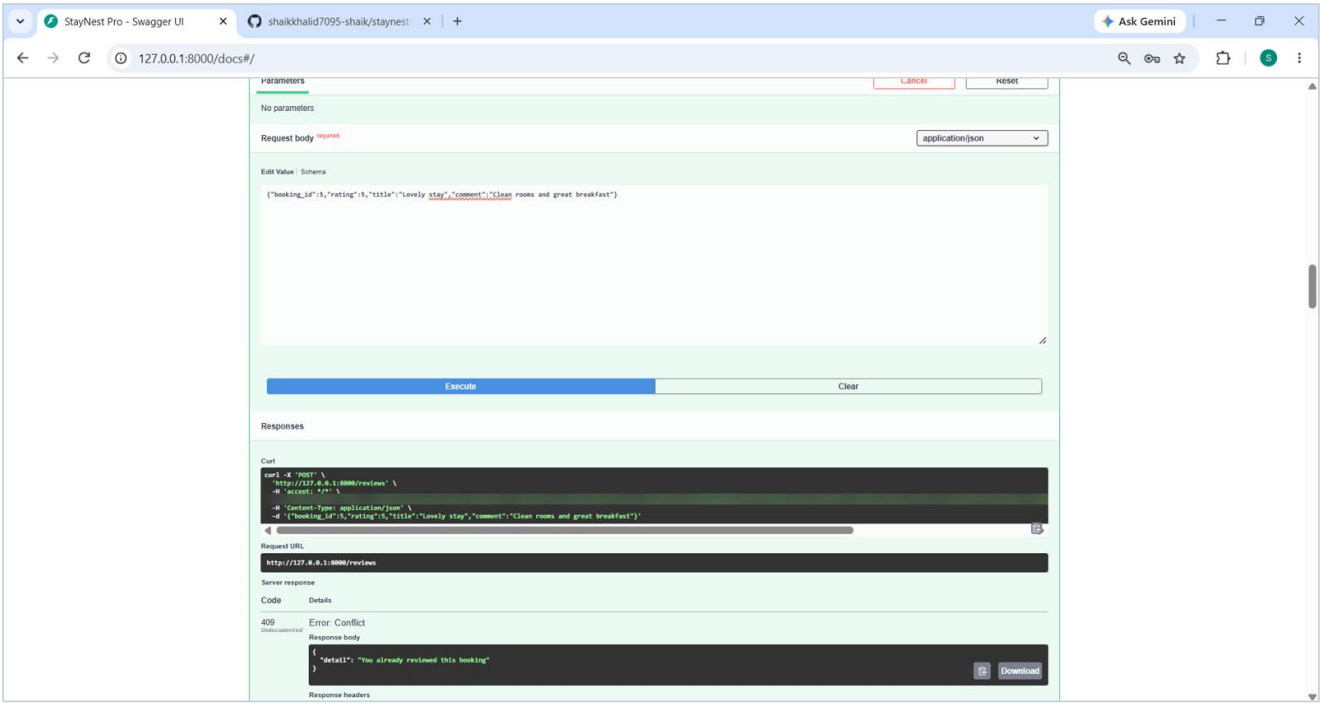
Day 2

Figure 63. Write a review - POST /reviews



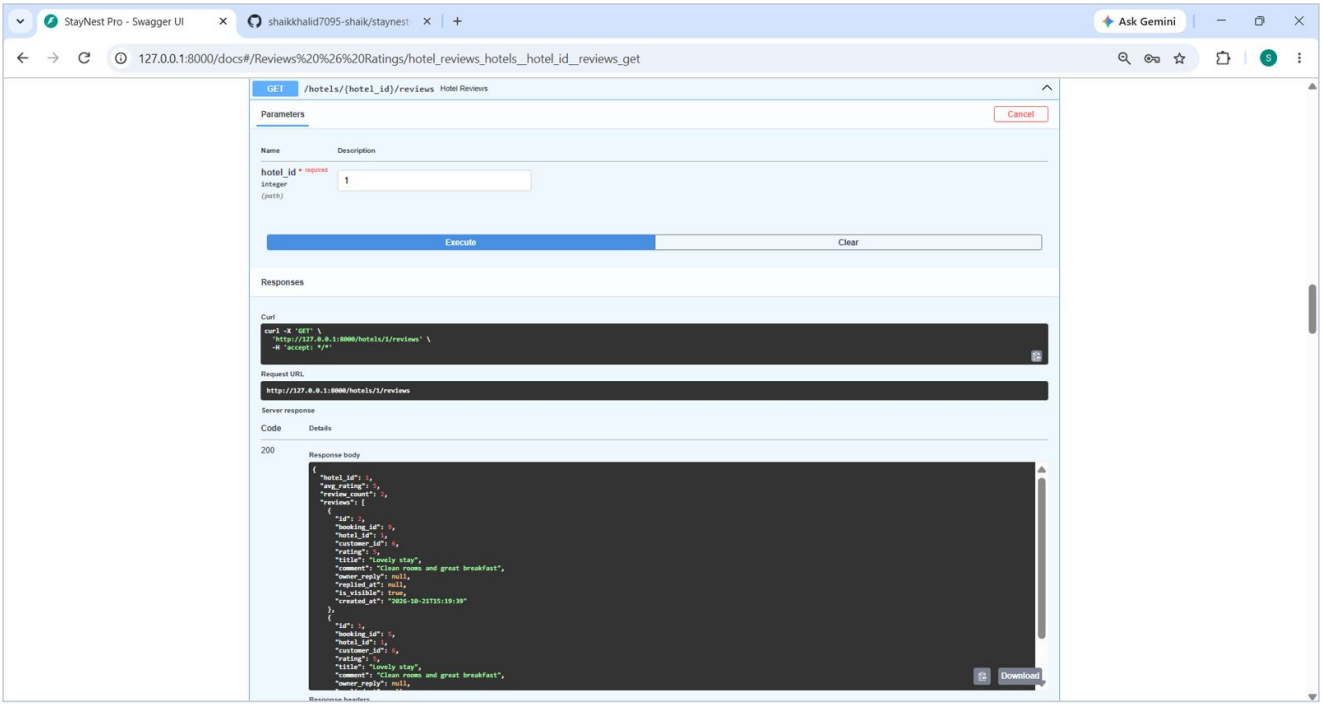
After checking out, the guest rates booking 9 with 5 stars.
201 Created: review 2, Lovely stay, visible on the hotel page.

Figure 64. Review rule - one review per booking



The guest tries to review booking 5 a second time.
The API refuses with 409: You already reviewed this booking.

Figure 65. Hotel reviews - GET /hotels/1/reviews

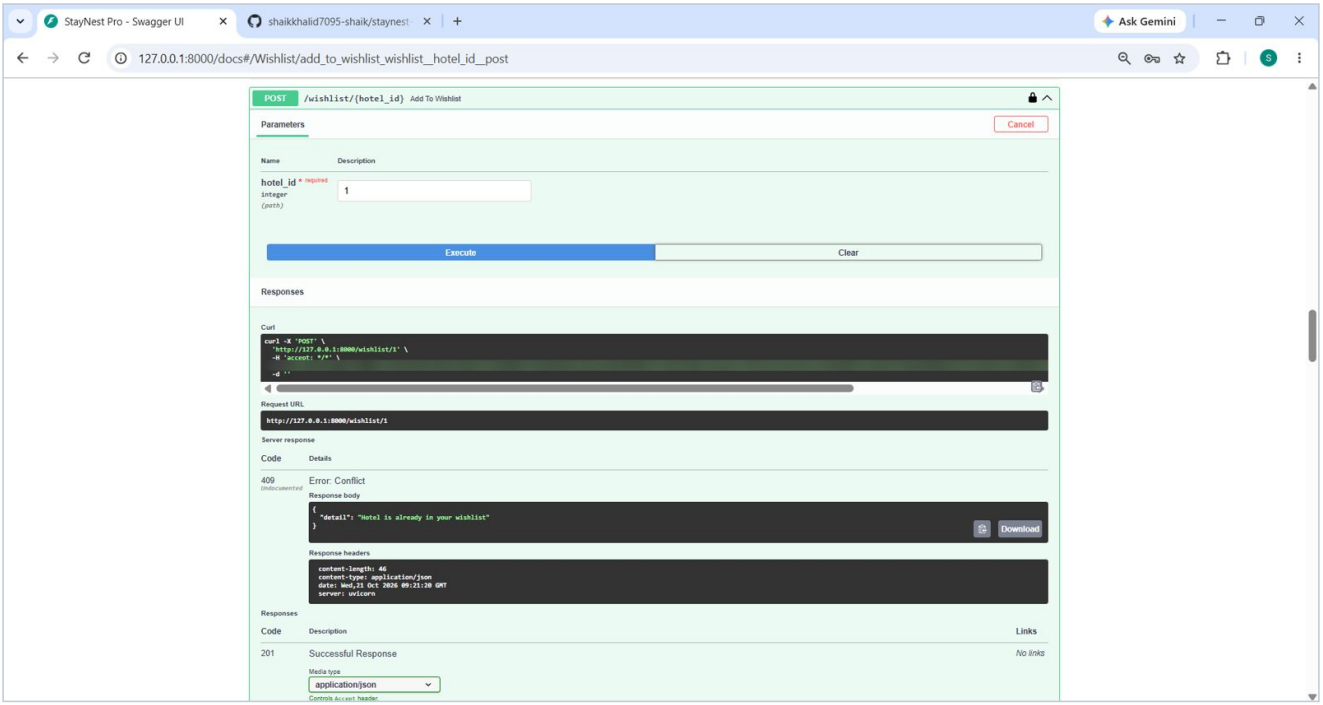


Anyone can read a hotel's reviews without logging in.
Khalid Grand Hotel has avg_rating 5 from 2 reviews.

Module 20. Wishlist and Favorite Hotels

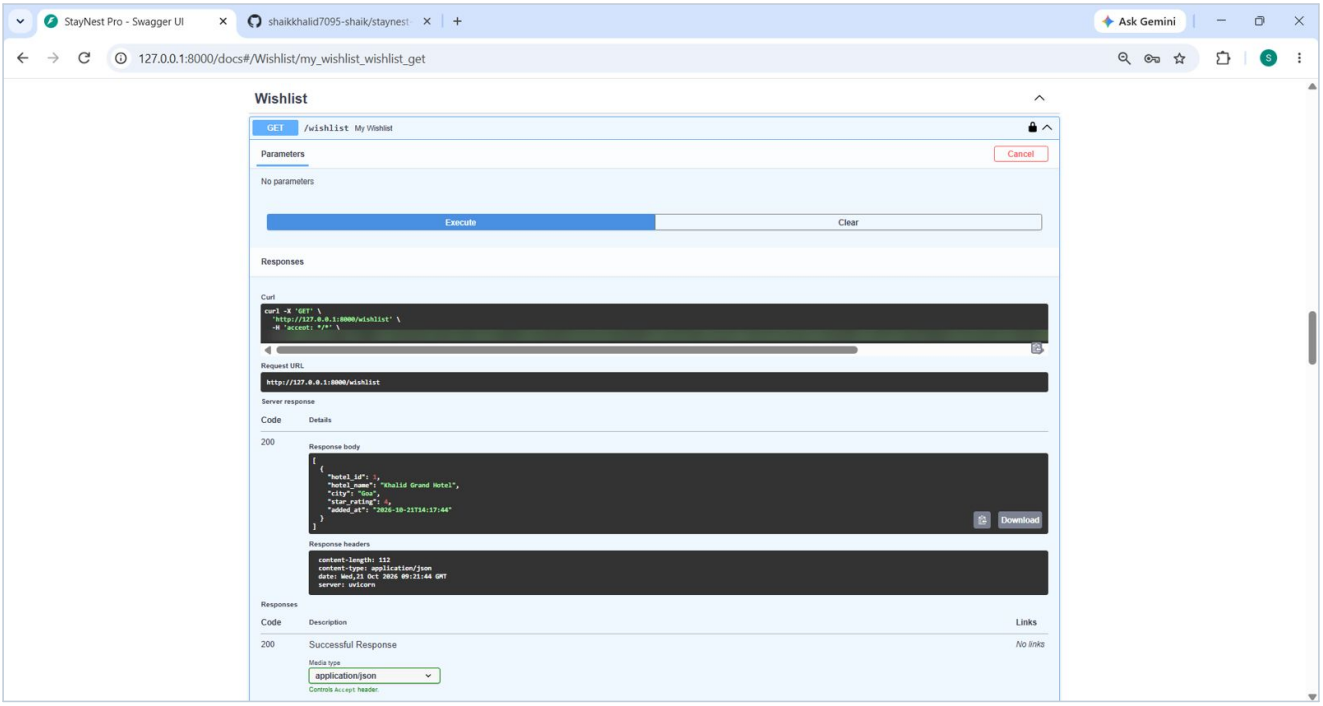
Day 2

Figure 66. Add to wishlist - POST /wishlist/1



The customer saves Khalid Grand Hotel to their wishlist.
The hotel was already saved, so the API answers 409: Hotel is already in your wishlist.

Figure 67. My wishlist - GET /wishlist

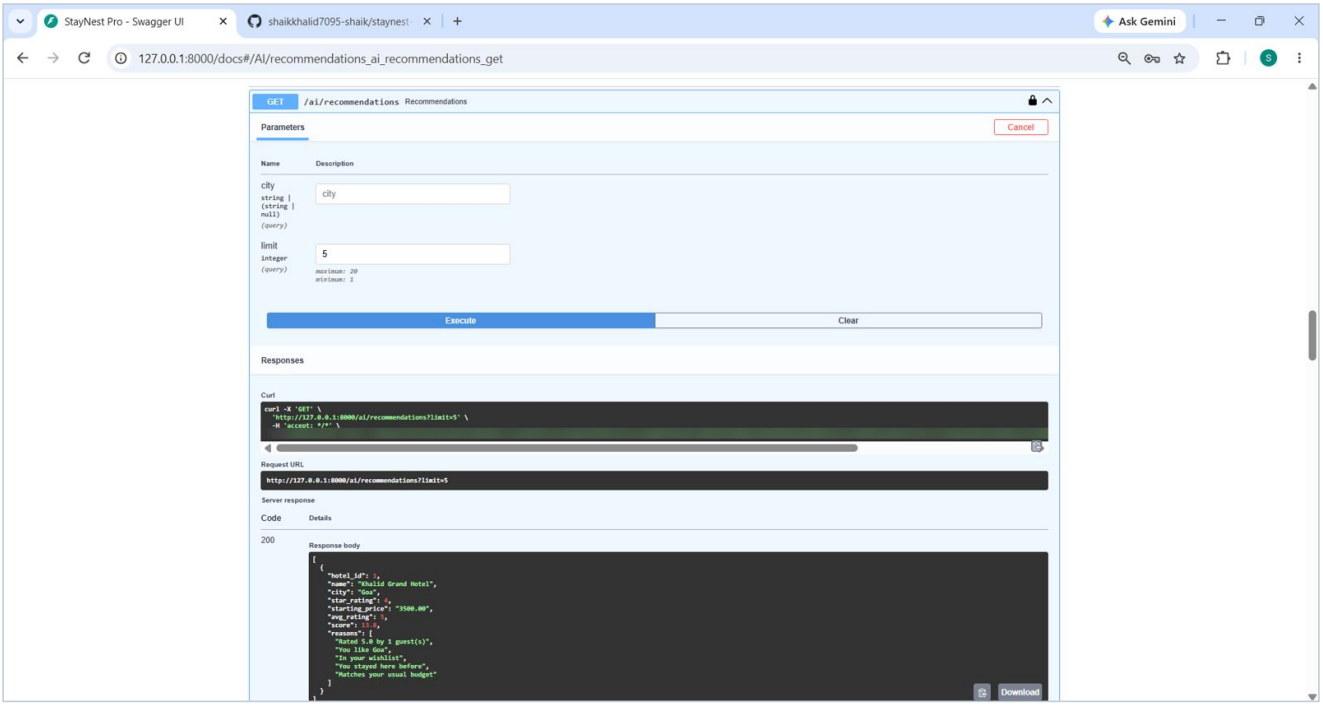


The customer opens their saved hotels.
Khalid Grand Hotel (Goa, 4 stars) is listed with the date it was added.

Module 21. AI Hotel Recommendation Engine

Day 3

Figure 68. AI hotel recommendations - GET /ai/recommendations

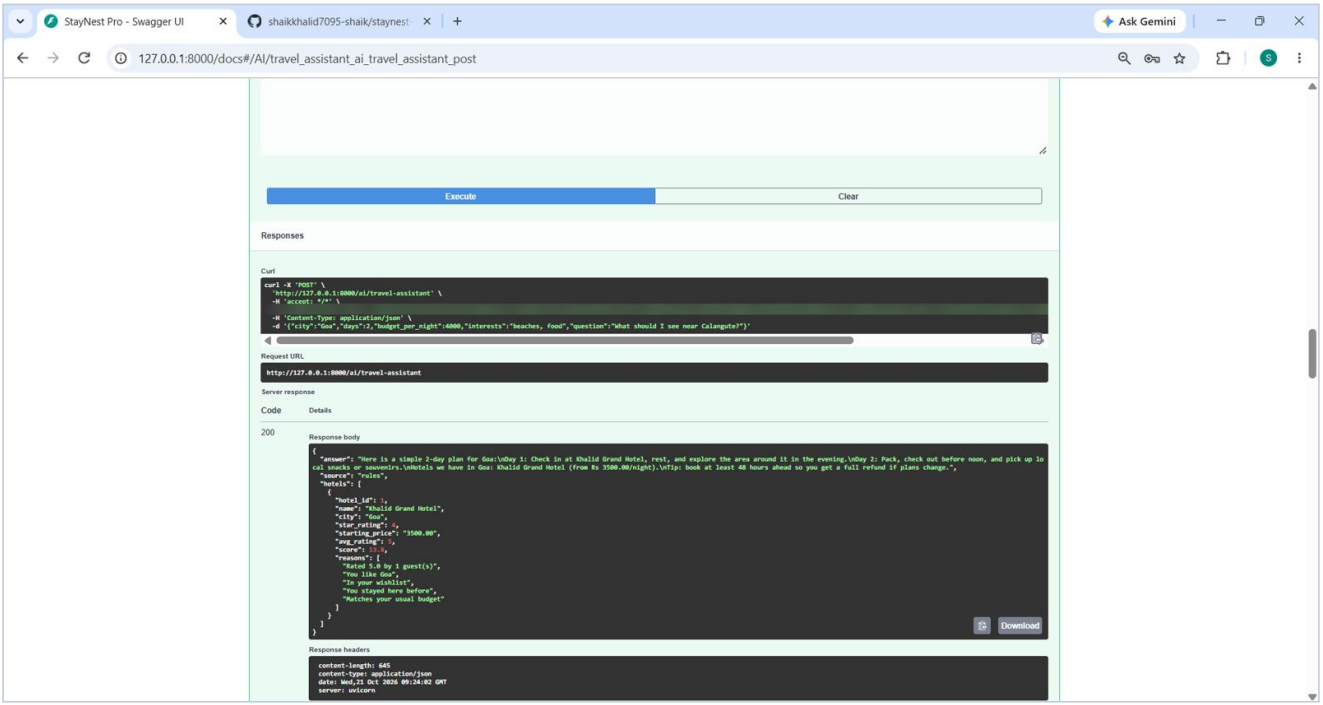


Hotels are scored using the customer's ratings, wishlist, past stays and budget.
Khalid Grand Hotel scores 13.8, with reasons such as In your wishlist and You stayed here before.

Module 22. AI Travel Assistant

Day 3

Figure 69. AI travel assistant - POST /ai/travel-assistant

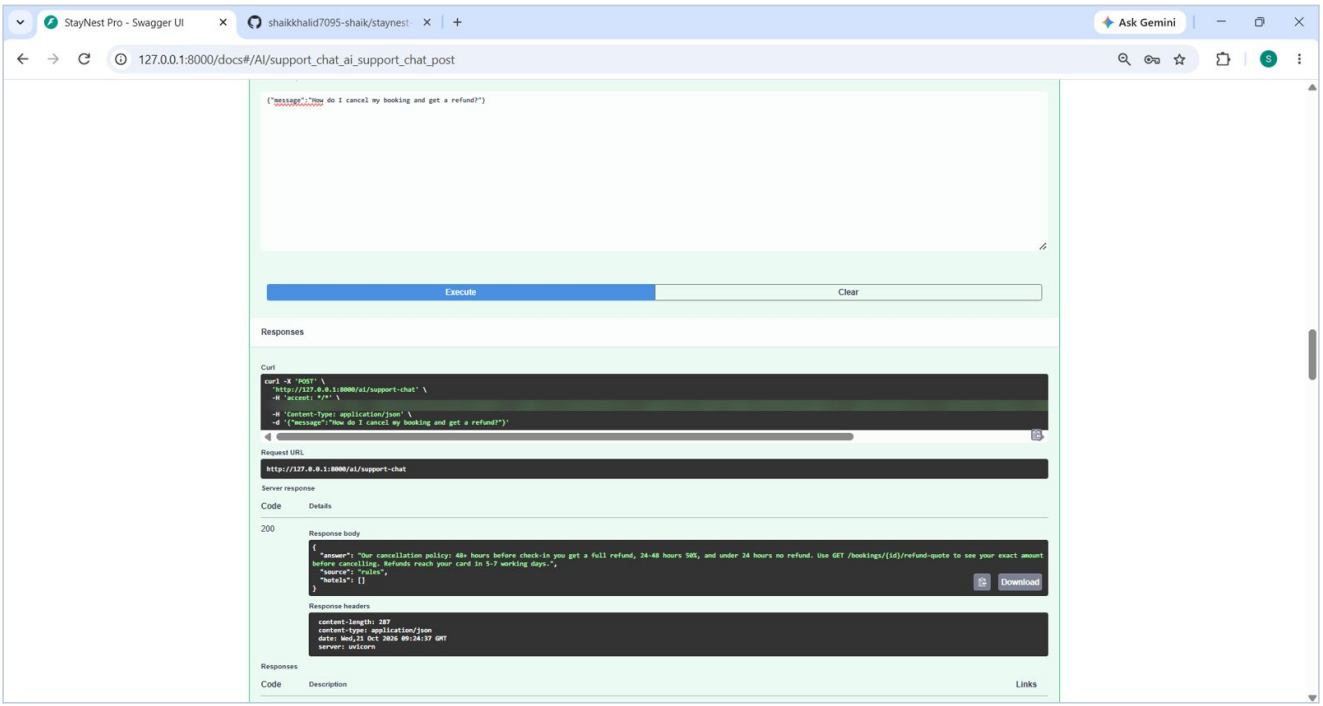


The customer asks for a 2-day Goa plan with a Rs 4000 budget and an interest in beaches and food.
A day-by-day plan comes back with matching hotels; source is rules because no AI key is set.

Module 23. AI Customer Support Chatbot

Day 3

Figure 70. AI support chatbot - POST /ai/support-chat

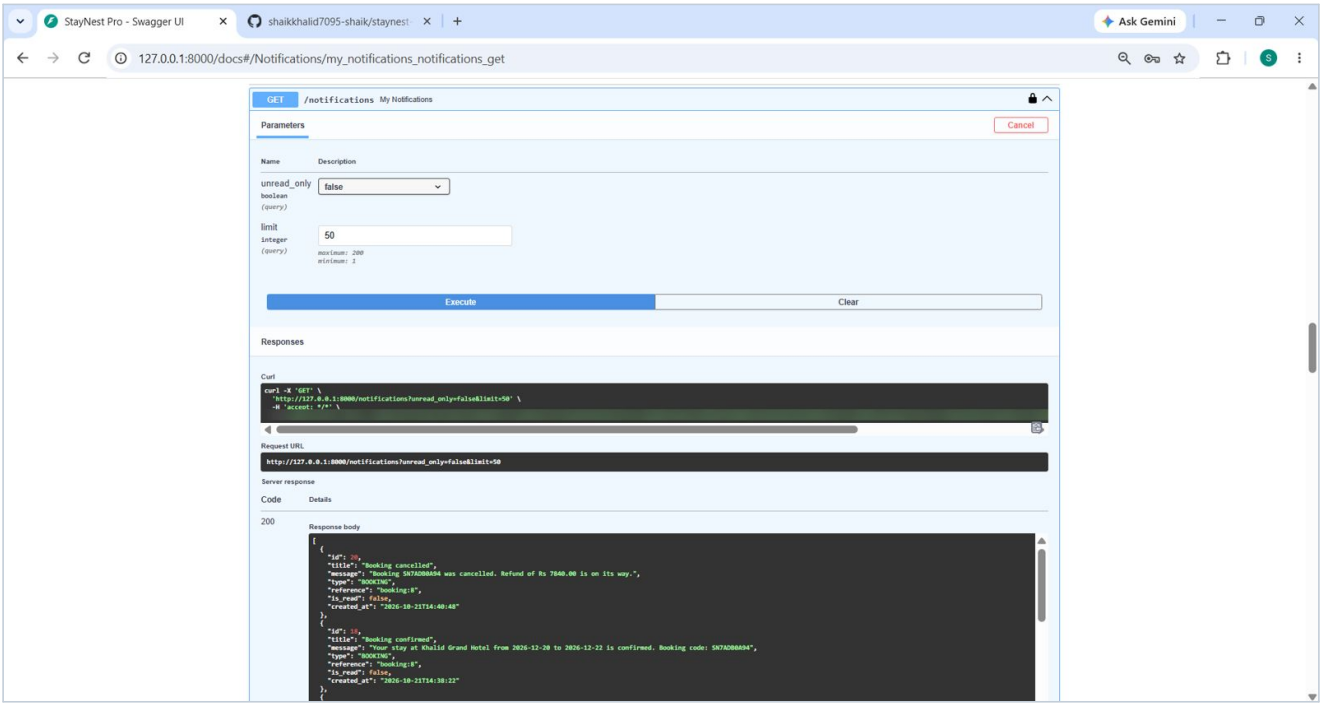


The customer asks how to cancel a booking and get a refund.
The bot explains the policy: full refund 48+ hours before, 50% at 24-48 hours, none under 24 hours.

Module 24. In-App Notifications

Day 3

Figure 71. Notifications - GET /notifications

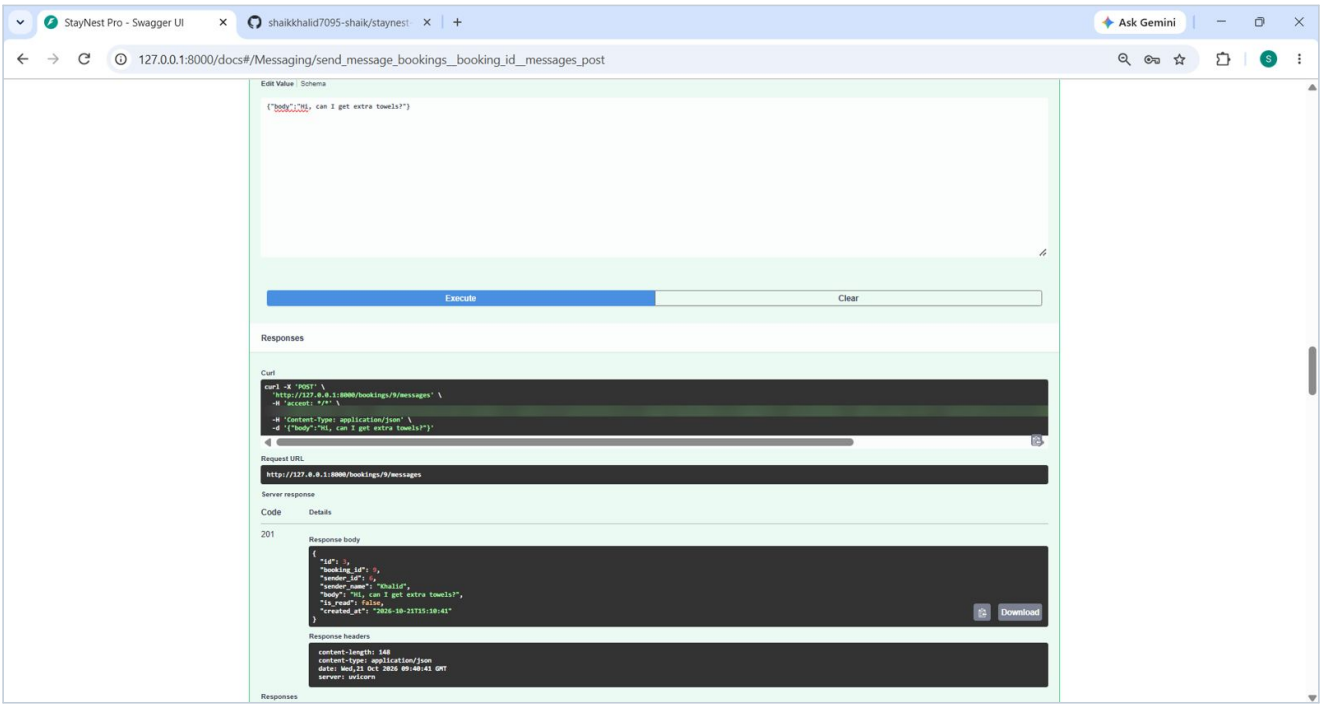


The customer's in-app notifications, newest first.
Booking confirmed and Booking cancelled (with the Rs 7840 refund) appear automatically.

Module 25. In-App Messaging

Day 3

Figure 72. In-app messaging - POST /bookings/9/messages

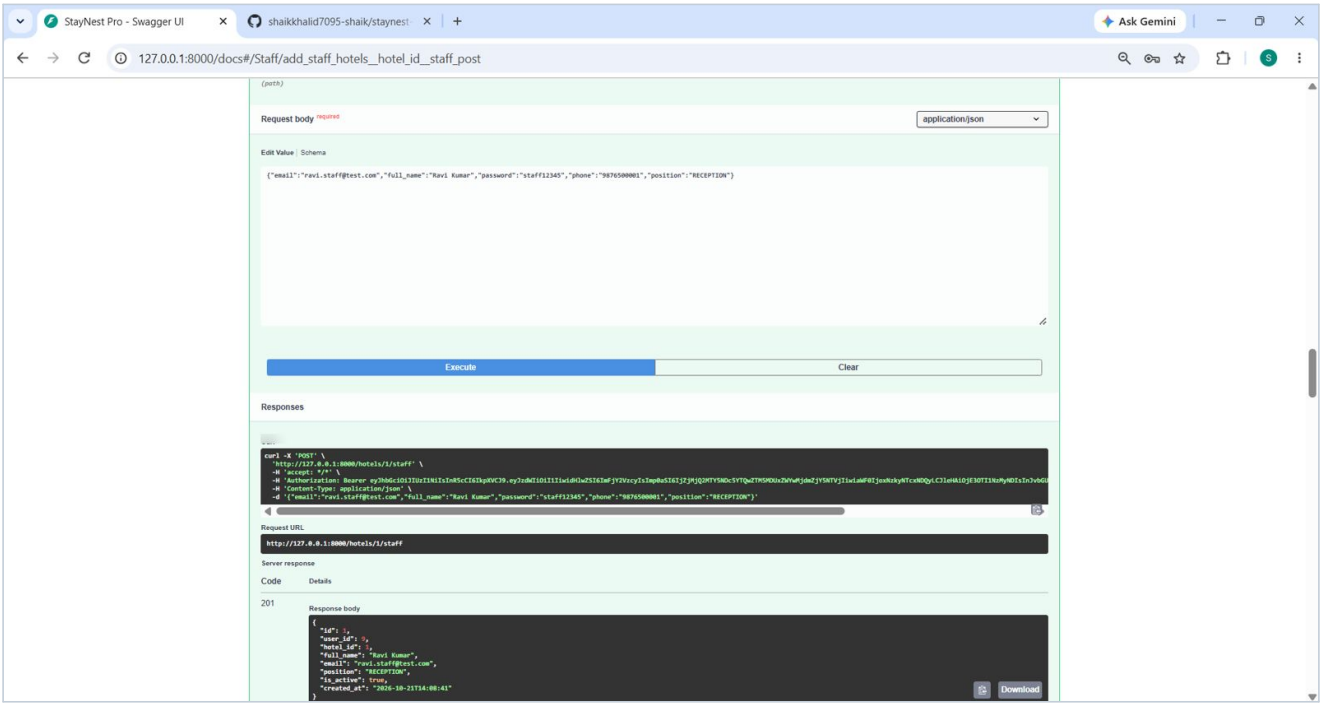


The guest sends the hotel a message on booking 9: Hi, can I get extra towels?
201 Created: message 3 from Khalid, unread until the hotel opens it.

Module 26. Housekeeping Management

Day 3

Figure 73. Housekeeping task - POST /hotels/1/housekeeping

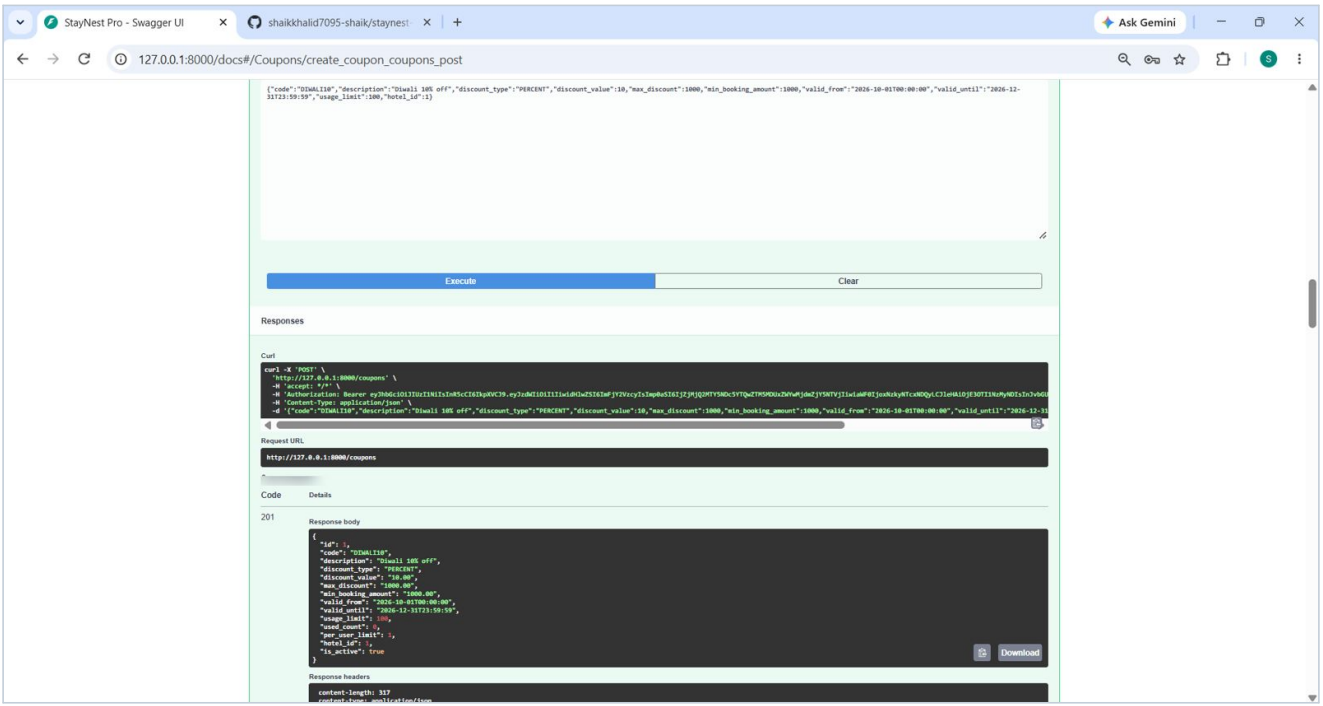


A HIGH priority CLEANING task is created for room 2 with a due date of 22 October 2026.
201 Created with status PENDING, ready to be assigned to staff.

Module 27. Hotel Staff Management

Day 3

Figure 74. Add staff - POST /hotels/1/staff



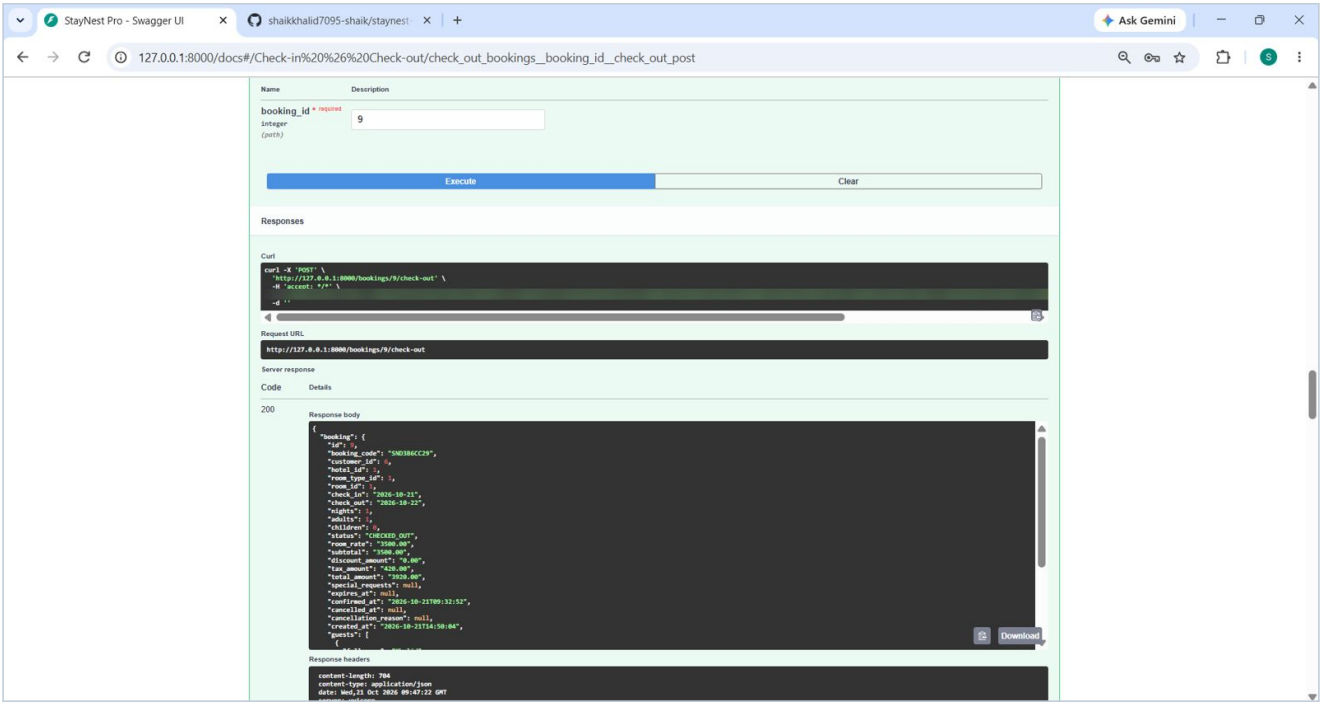
The owner creates a login for Ravi Kumar as RECEPTION staff at hotel 1.
201 Created: staff id 1 linked to new user id 9 with the HOTEL_STAFF role.

Module 28. Check-In and Check-Out Management

Day 3

Also shown in Figure 48.

Figure 75. Check-out - POST /bookings/9/check-out

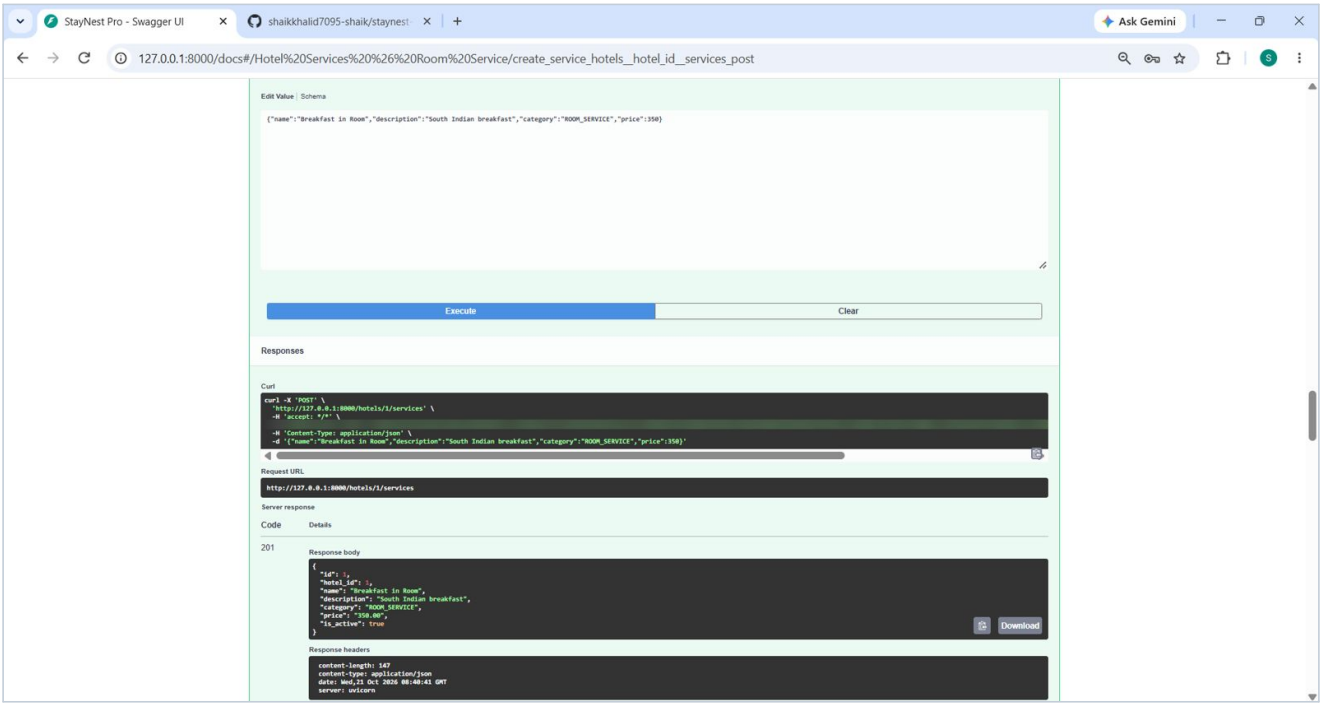


The owner checks the guest out after all service orders are closed.
Booking 9 becomes CHECKED_OUT, and the response includes the final bill summary.

Module 29. Additional Services Management

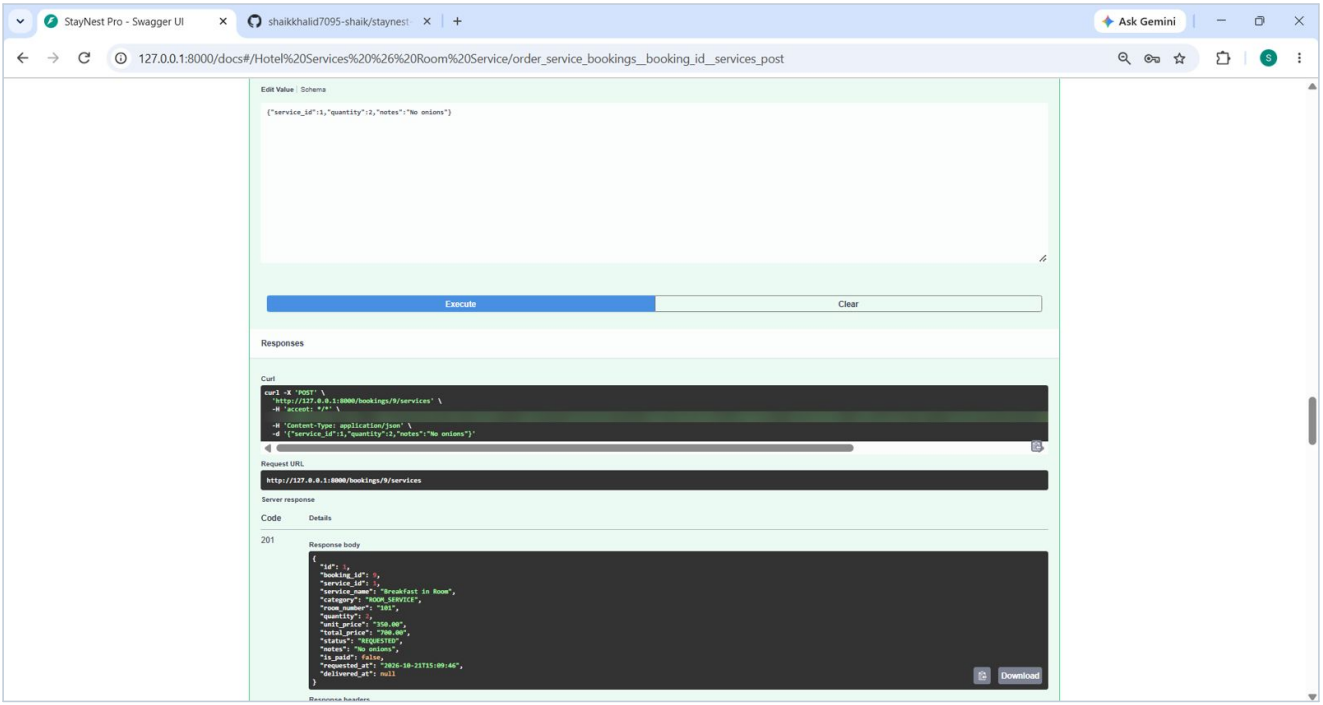
Day 3

Figure 76. Additional service - POST /hotels/1/services



The owner adds Breakfast in Room in the ROOM_SERVICE category for Rs 350.
201 Created: service id 1 is active and guests can order it during their stay.

Figure 77. Order room service - POST /bookings/9/services



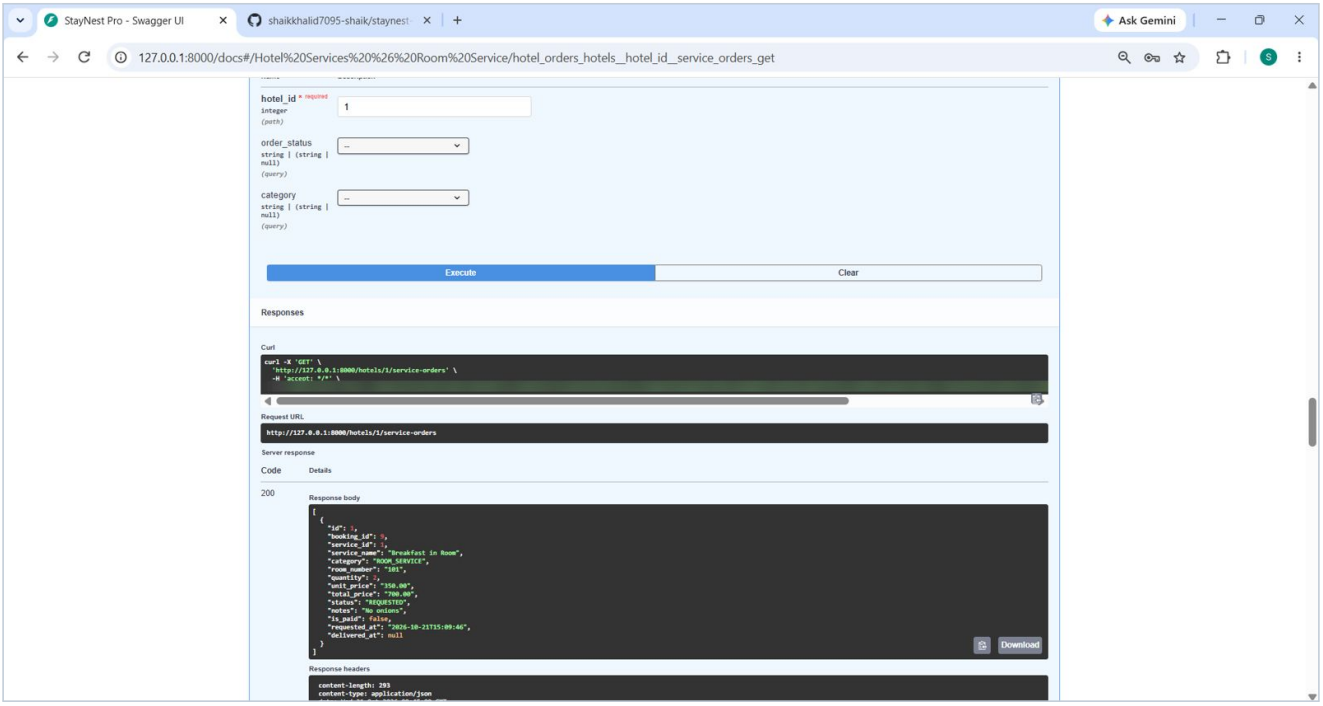
The checked-in guest orders 2 x Breakfast in Room with the note No onions.
201 Created: order 1 for room 101, total Rs 700, status REQUESTED.

Module 30. Room Service Management

Day 3

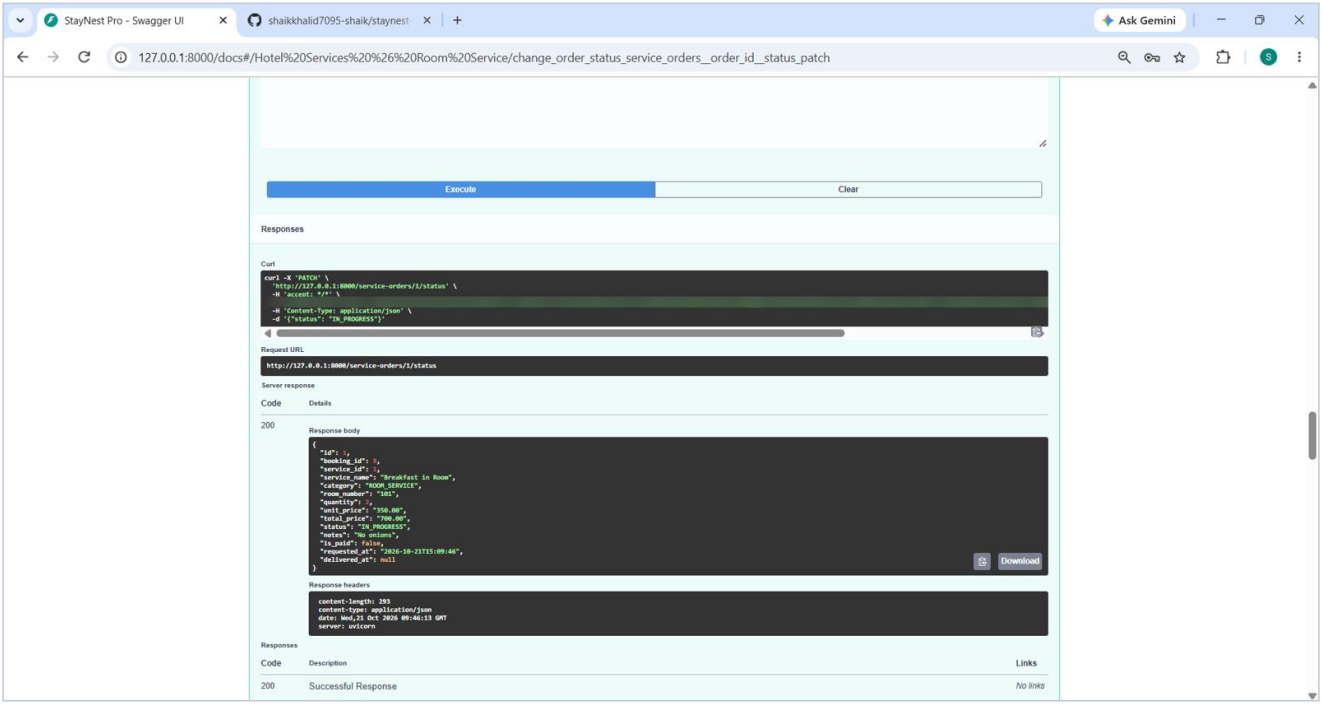
Also shown in Figure 77.

Figure 78. Hotel service orders - GET /hotels/1/service-orders



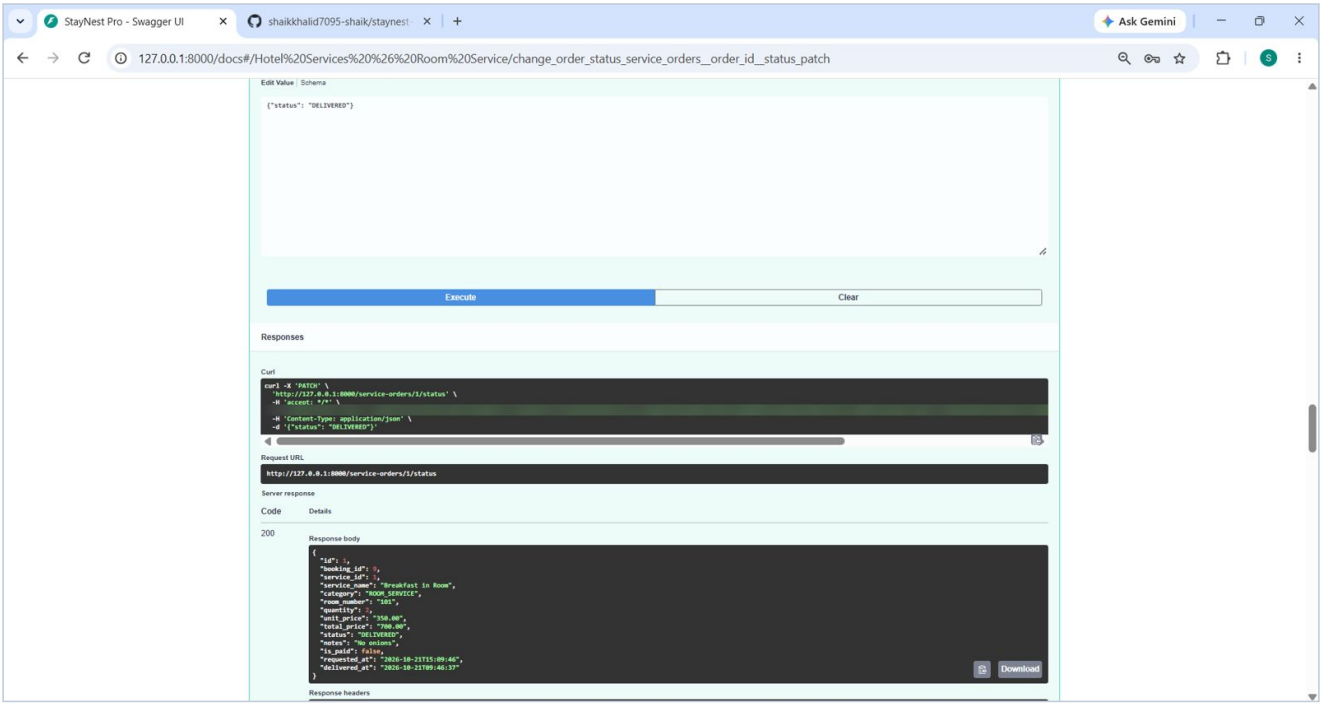
The owner sees every room service order placed at the hotel.
Order 1 for booking 9 is listed as REQUESTED, waiting for the kitchen.

Figure 79. Order in progress - PATCH /service-orders/1/status



The owner moves the order to IN_PROGRESS while it is being prepared.
200 OK; the guest gets a notification about the new status.

Figure 80. Order delivered - PATCH /service-orders/1/status

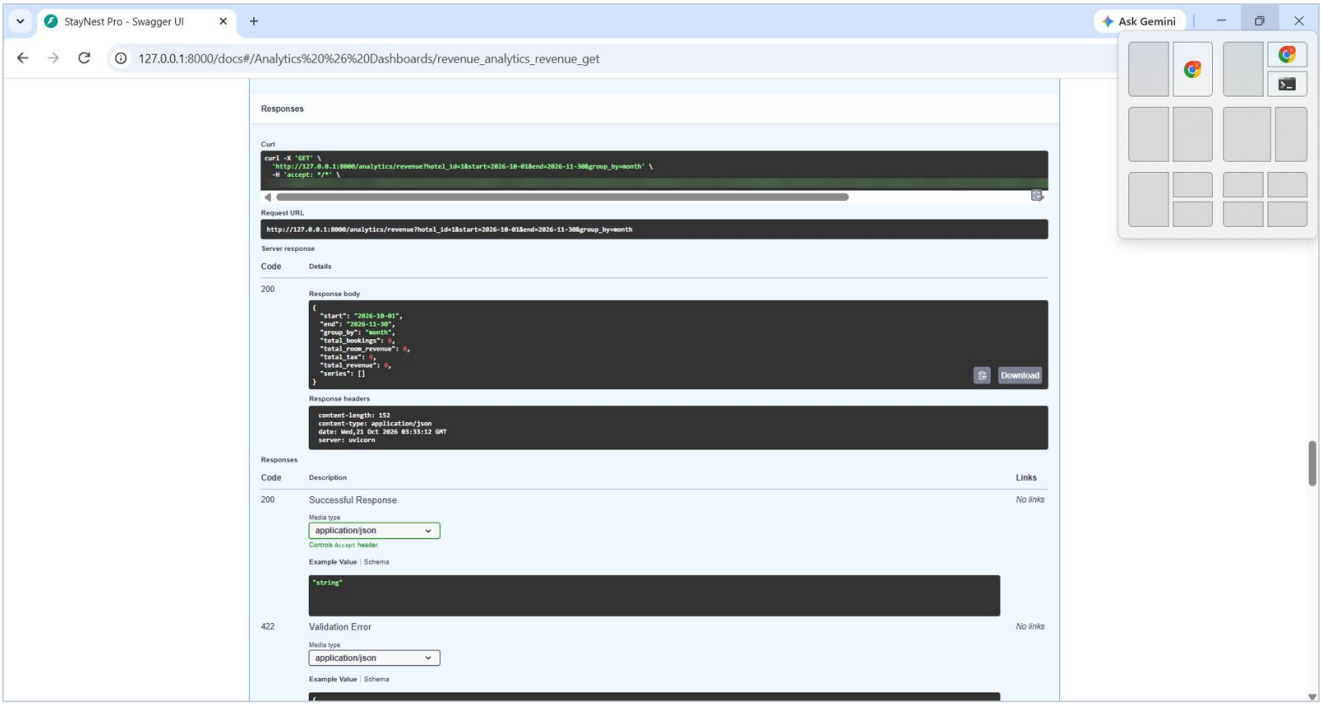


The order is marked DELIVERED once the breakfast reaches room 101.
delivered_at is filled in, so the order no longer blocks check-out.

Module 31. Hotel Revenue Analytics

Day 4

Figure 81. Revenue analytics - GET /analytics/revenue

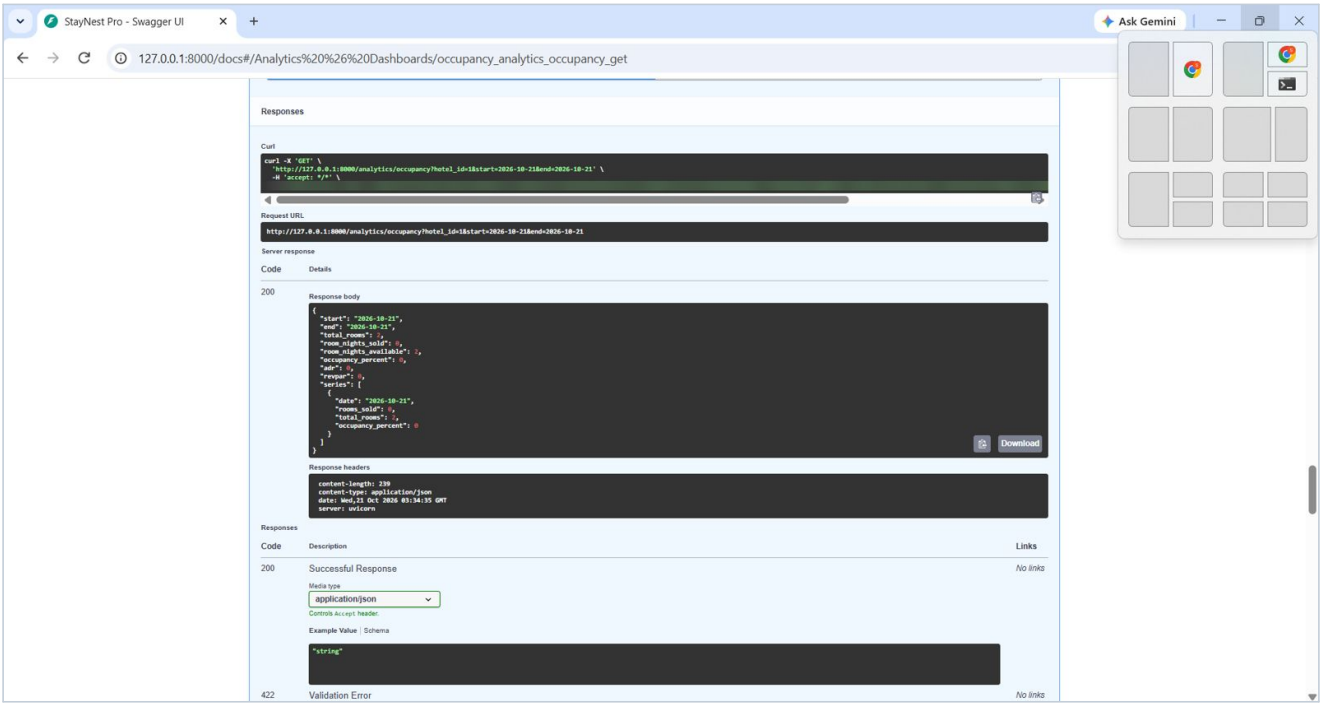


Revenue for hotel 1 from 1 October to 30 November 2026, grouped by month.
Totals for bookings, room revenue, tax and revenue are 0 because no booking has been paid yet.

Module 32. Hotel Occupancy Analytics

Day 4

Figure 82. Occupancy analytics - GET /analytics/occupancy

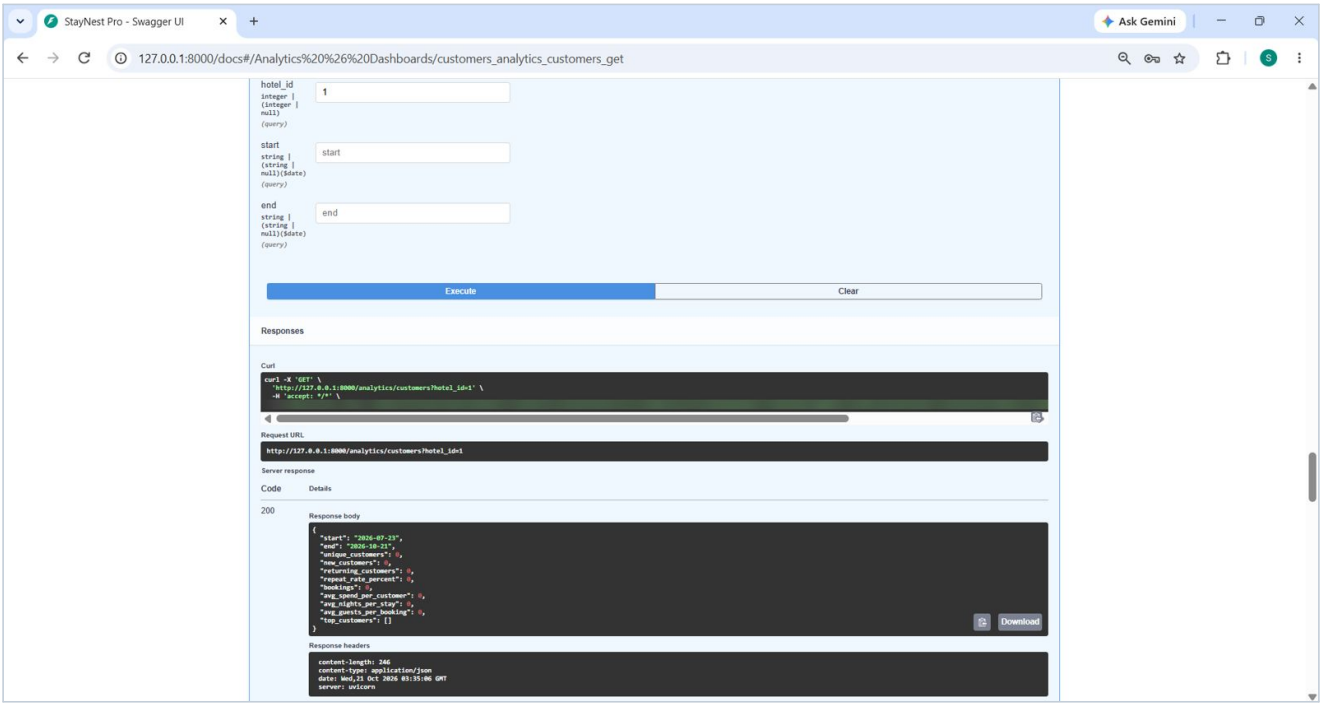


Occupancy for hotel 1 on 21 October 2026 across its 2 rooms.
The response gives room nights sold and available, occupancy percent, ADR, RevPAR and a daily series.

Module 33. Customer Analytics

Day 4

Figure 83. Customer analytics - GET /analytics/customers

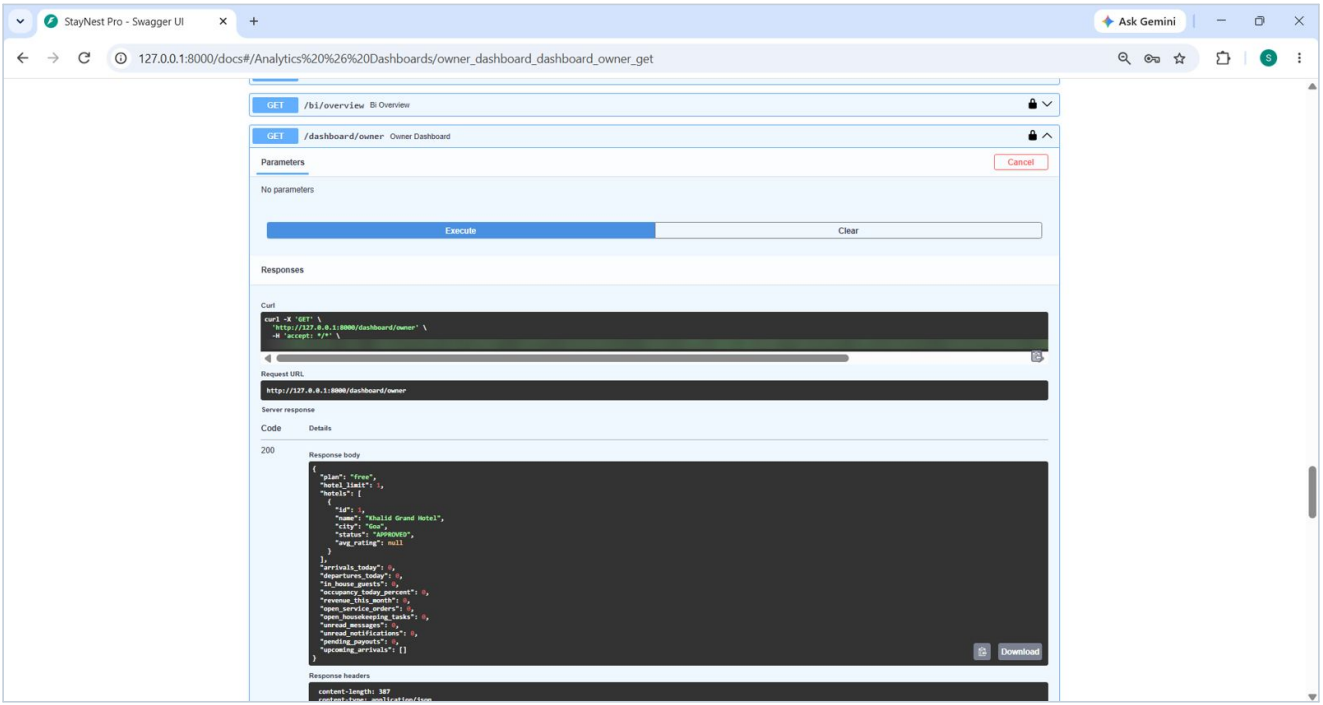


Customer statistics for hotel 1 over the last 90 days.
It reports unique, new and returning customers, repeat rate, average spend, nights and guests per booking, and top customers.

Module 34. Hotel Owner Dashboard APIs

Day 4

Figure 84. Owner dashboard - GET /dashboard/owner

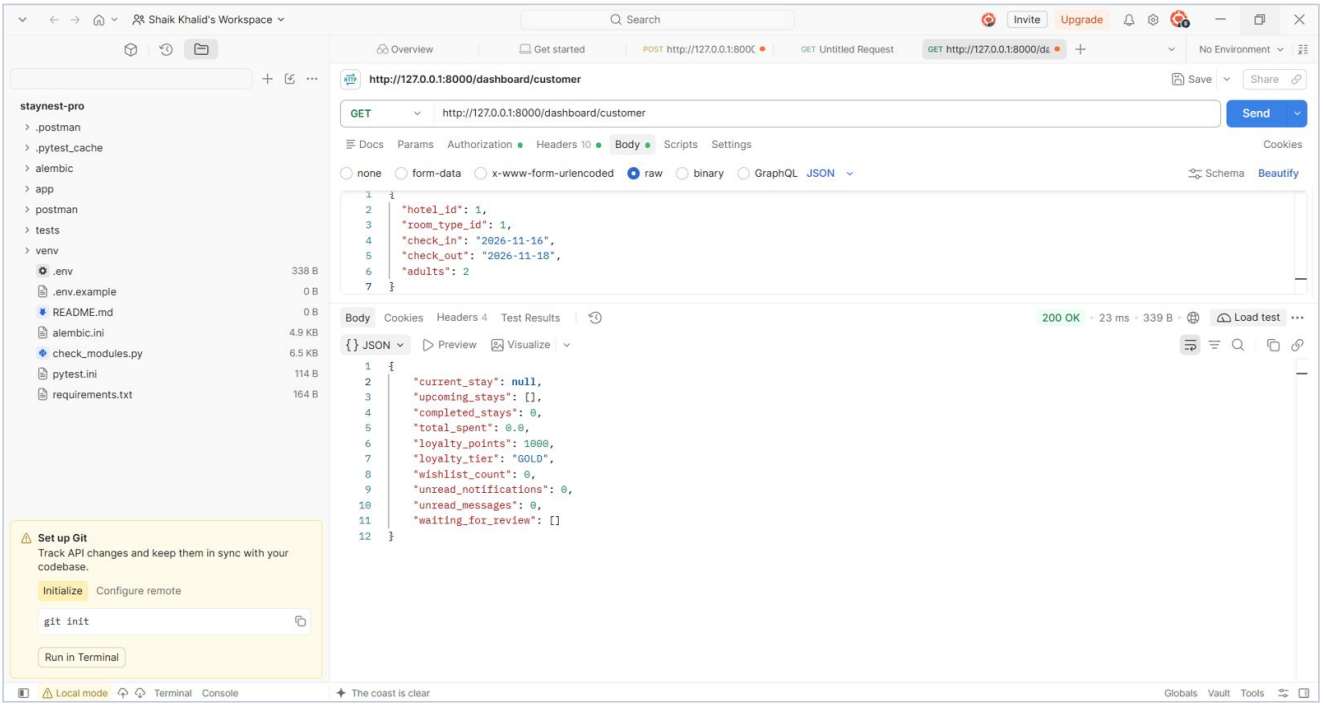


The owner sees plan free with a 1 hotel limit and Khalid Grand Hotel (Goa, APPROVED).
Today's arrivals, departures, in-house guests, occupancy, revenue, open tasks and unread messages are shown in one call.

Module 35. Customer Dashboard APIs

Day 4

Figure 85. Postman - customer dashboard

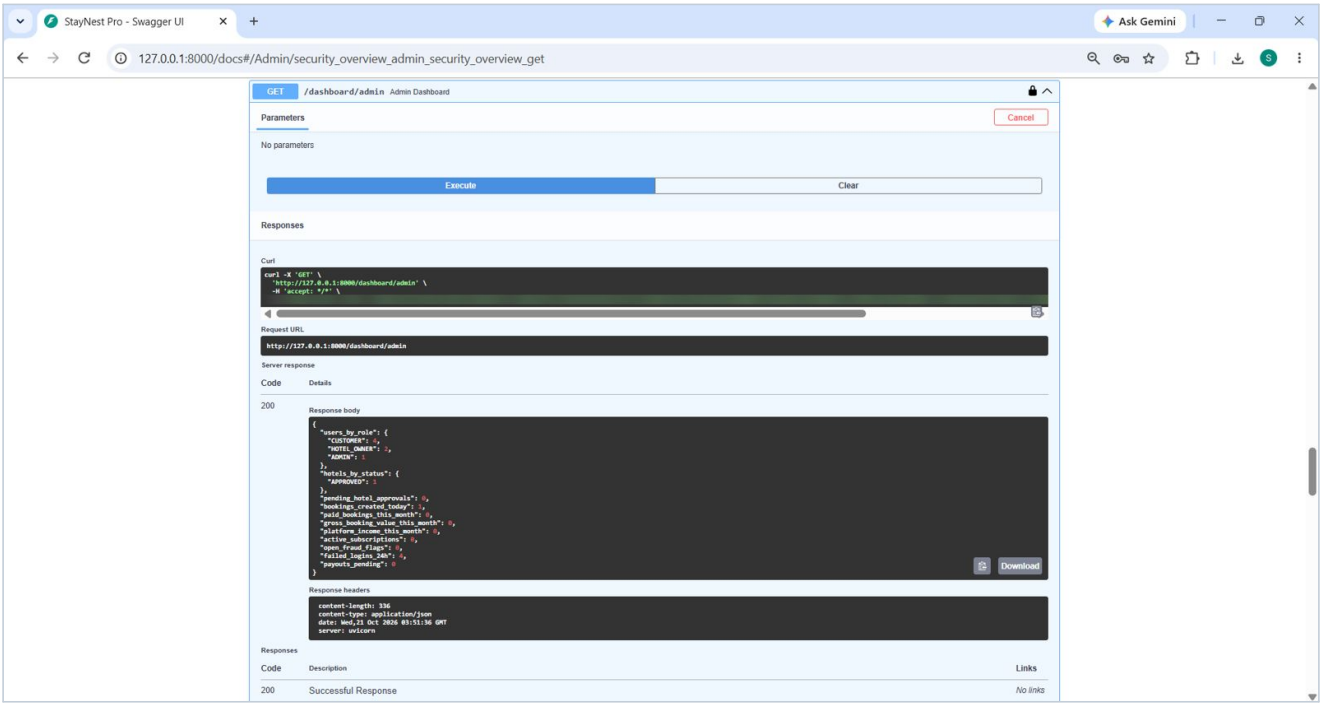


GET /dashboard/customer shows the customer's summary.
Loyalty points 1000 and tier GOLD, plus stays, total spent, wishlist and unread counts.

Module 36. Admin Dashboard APIs

Day 4

Figure 86. Admin dashboard - GET /dashboard/admin

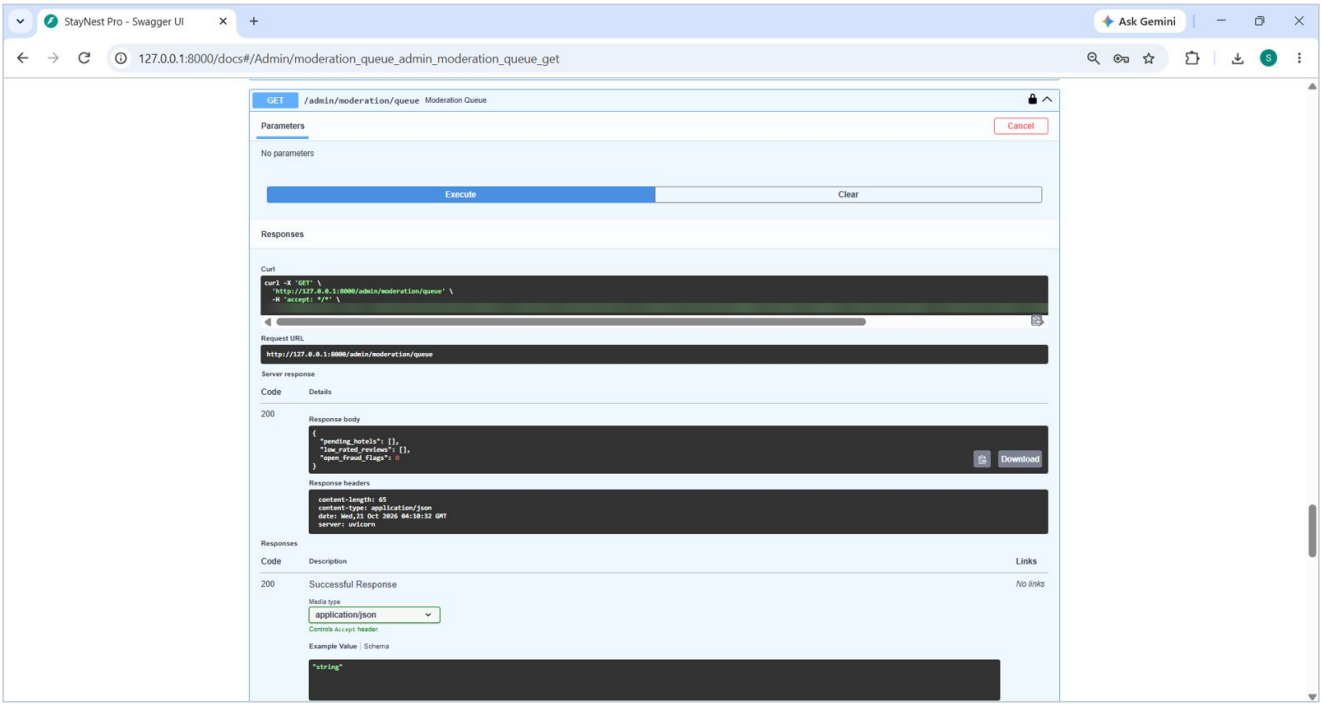


The admin sees the whole platform in one call: 4 customers, 2 hotel owners and 1 admin.
It also shows hotels by status, pending approvals, bookings today, income, subscriptions, fraud flags and failed logins.

Module 37. Hotel Verification and Moderation

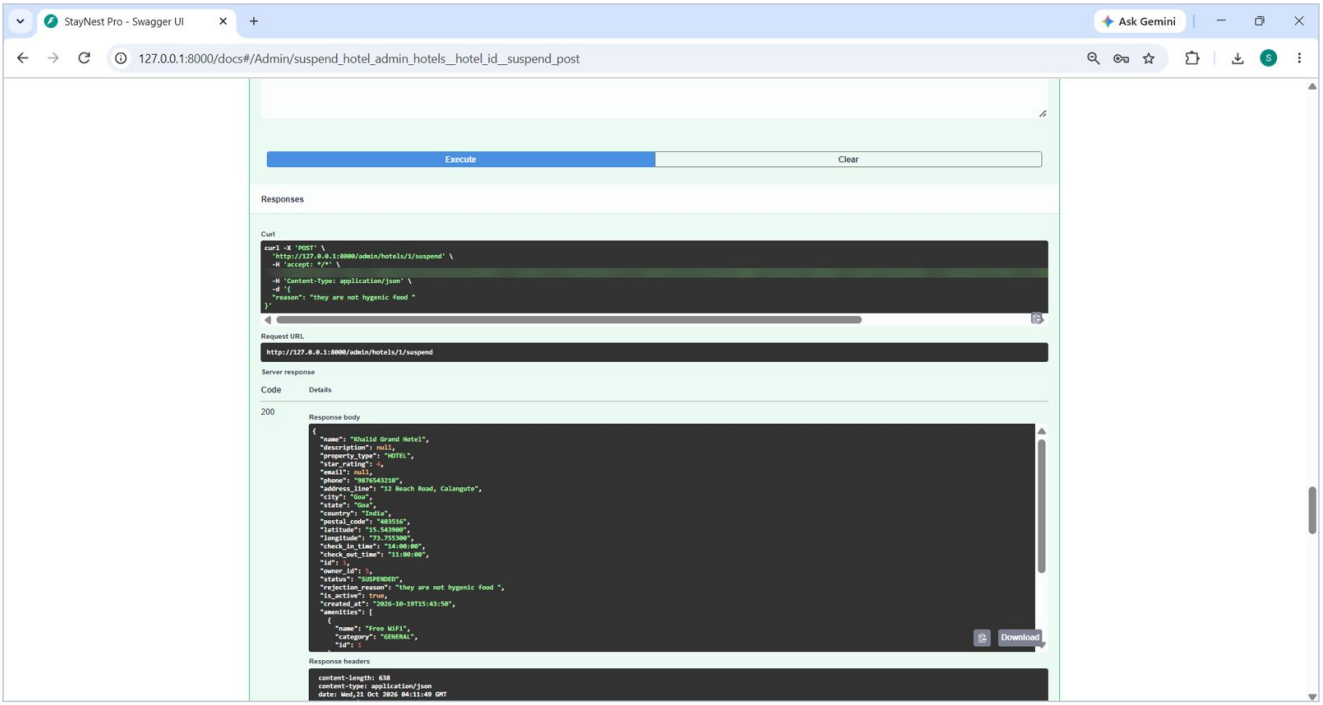
Day 4

Figure 87. Moderation queue - GET /admin/moderation/queue



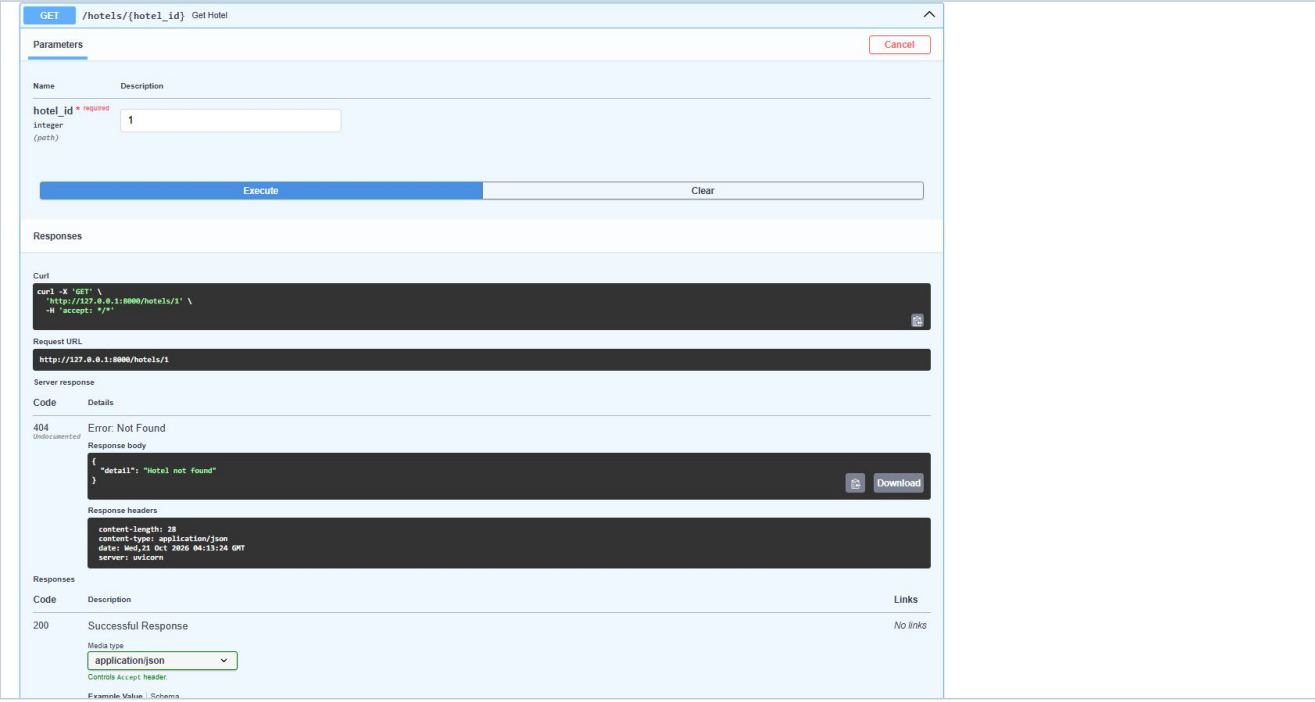
One call shows everything waiting for admin review.
Pending hotels, low-rated reviews and open fraud flags are all empty.

Figure 88. Suspend a hotel - POST /admin/hotels/1/suspend



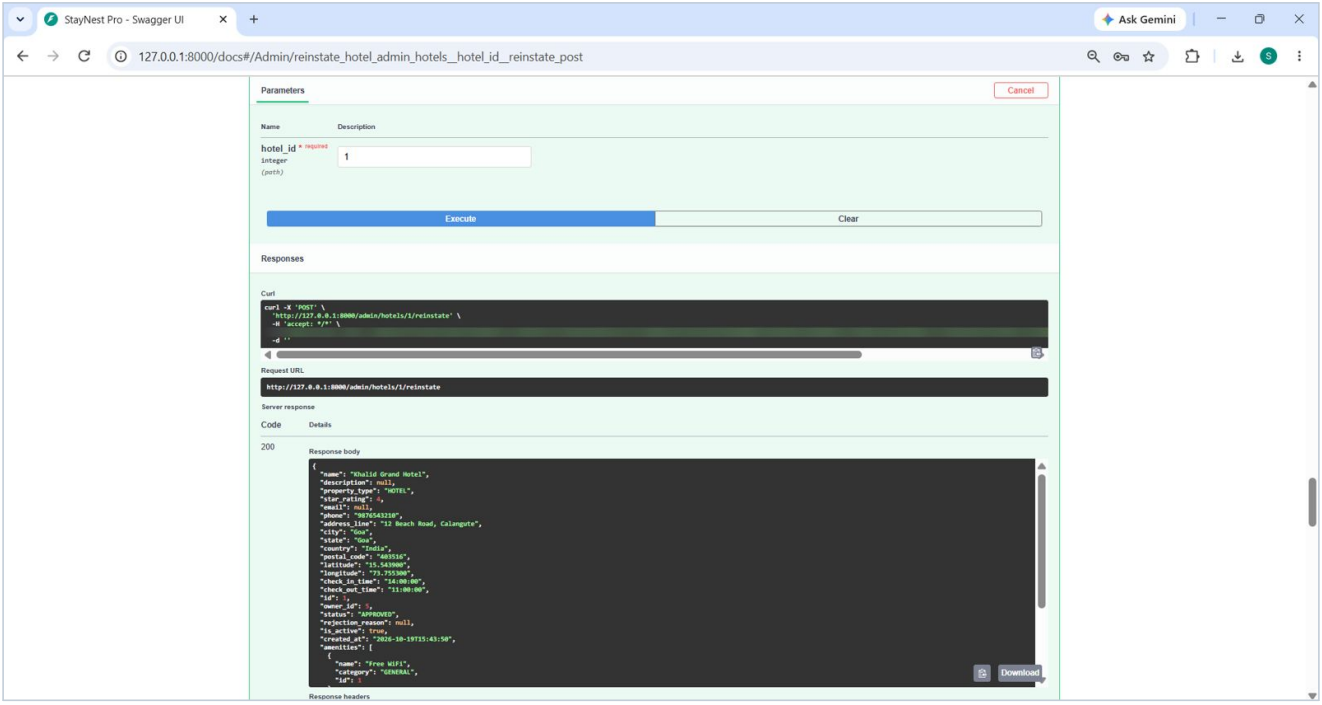
The admin suspends Khalid Grand Hotel with a reason.
The hotel status changes to SUSPENDED and the reason is saved in rejection_reason.

Figure 89. Suspended hotel is hidden - GET /hotels/1



A public request for the suspended hotel is made without logging in.
The API answers 404 Hotel not found, so suspended hotels disappear from customers.

Figure 90. Reinstate a hotel - POST /admin/hotels/1/reinstate

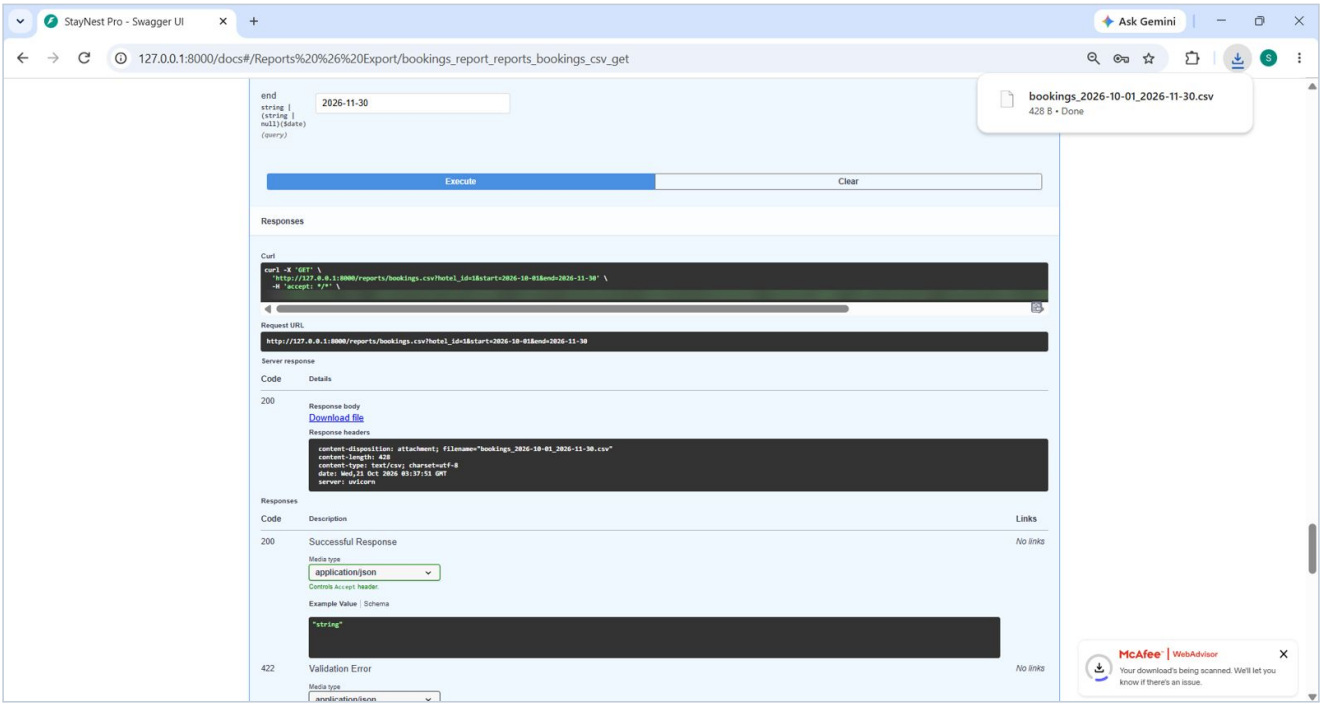


The admin brings the suspended hotel back.
Status returns to APPROVED with rejection_reason null, and the hotel is public again.

Module 38. Reports and Export

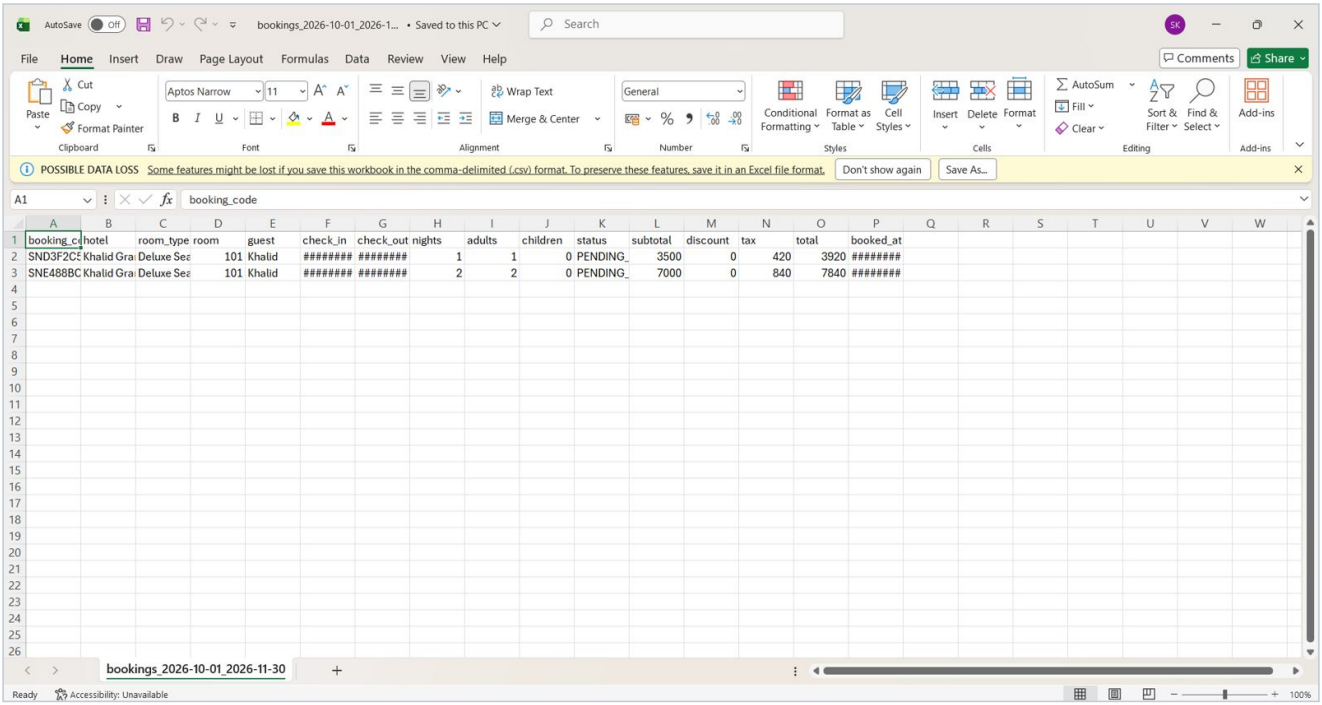
Day 4

Figure 91. Bookings report - GET /reports/bookings.csv



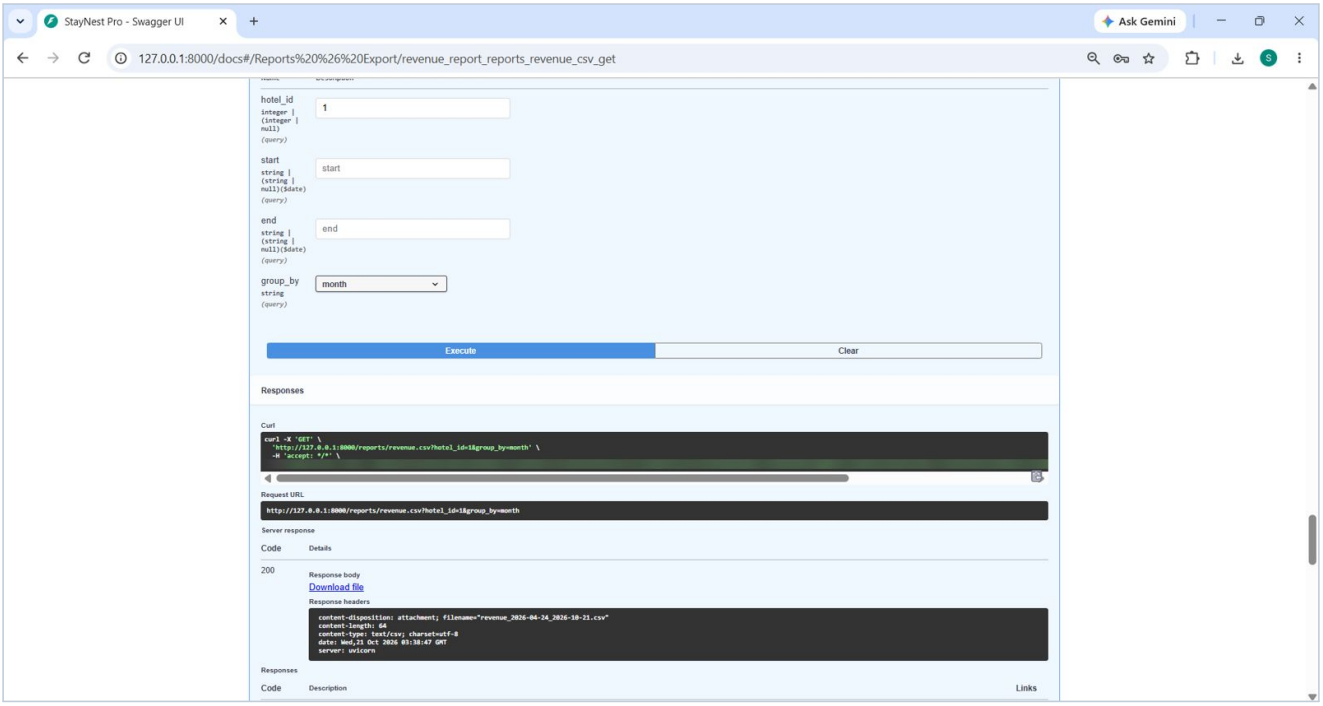
The owner exports bookings of hotel 1 from 1 October to 30 November 2026.
The API returns a CSV file download (428 bytes) named bookings_2026-10-01_2026-11-30.csv.

Figure 92. Bookings CSV opened in Excel



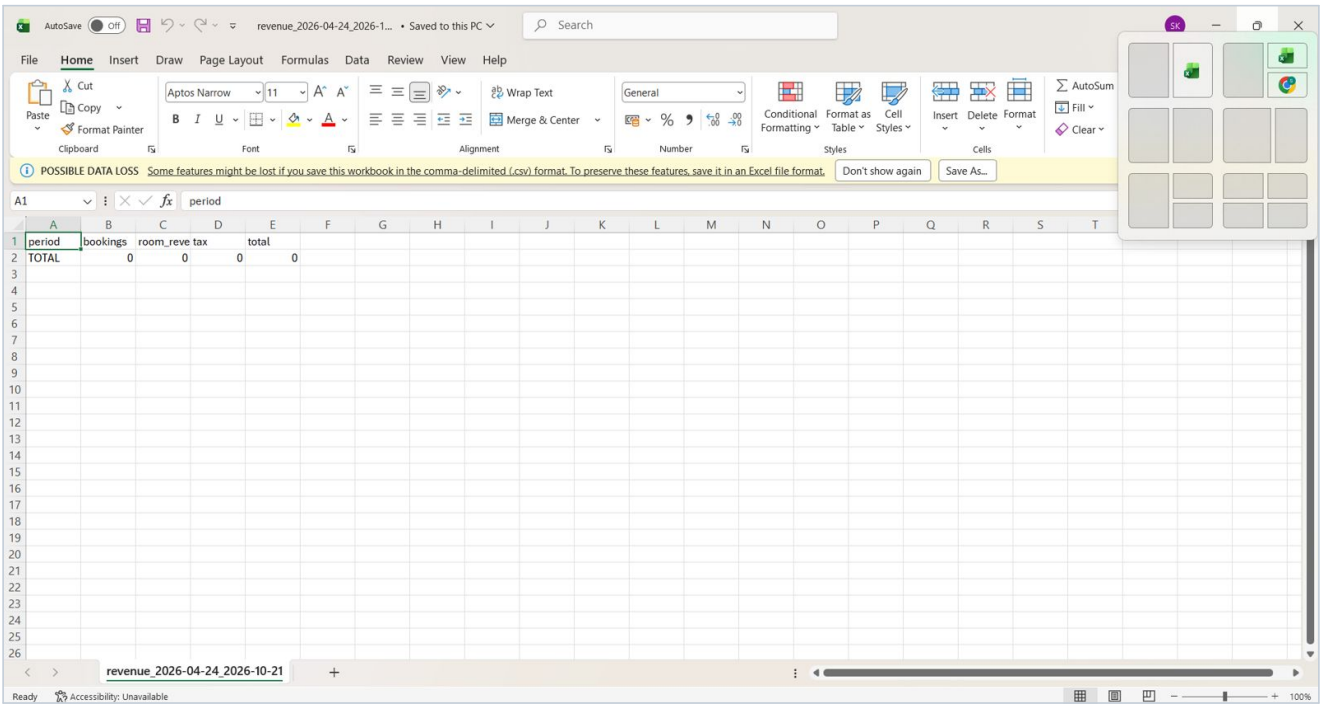
The downloaded file opens in Excel with one row per booking.
Both bookings show code, hotel, room 101, guest, nights, status, subtotal, tax and totals of Rs 3920 and Rs 7840.

Figure 93. Revenue report - GET /reports/revenue.csv



The owner exports revenue for hotel 1 grouped by month.
The API returns a CSV download with Content-Disposition attachment.

Figure 94. Revenue CSV opened in Excel

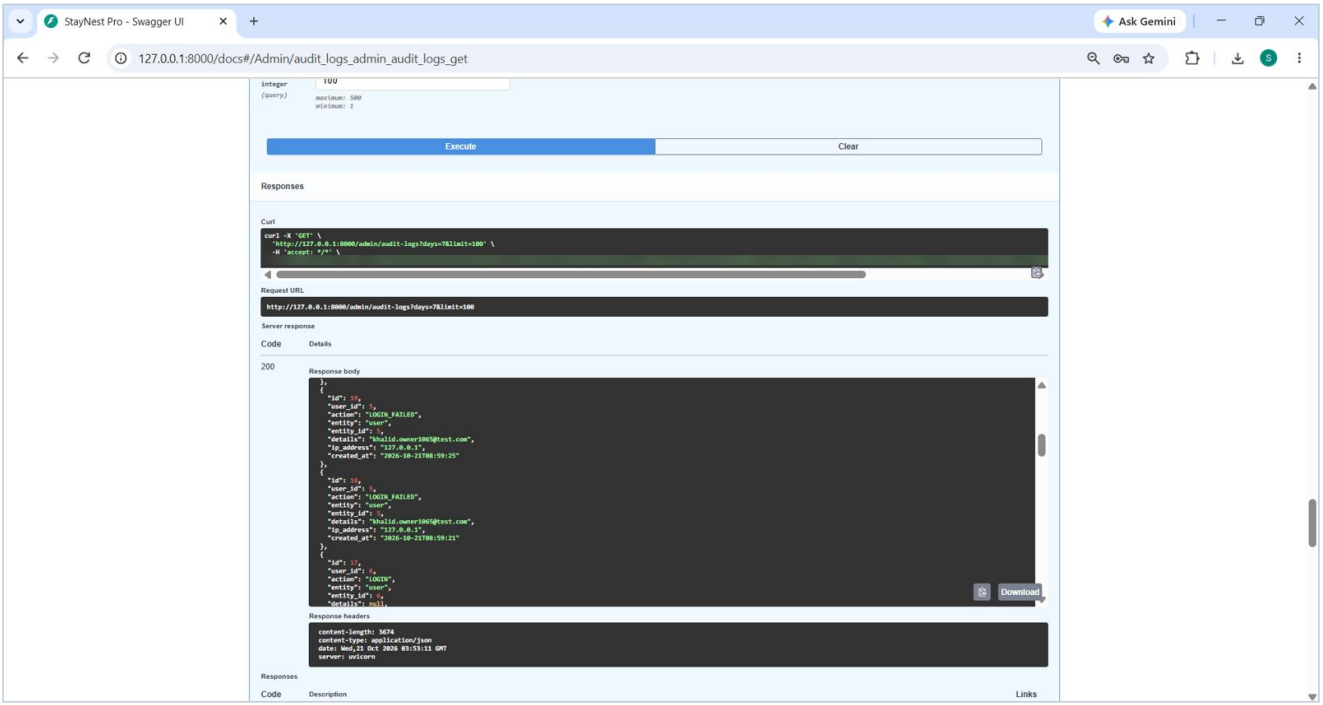


The revenue file opens in Excel with period, bookings, room revenue, tax and total columns.
The TOTAL row is 0 because no booking has been paid yet.

Module 39. Audit Logs and Security Monitoring

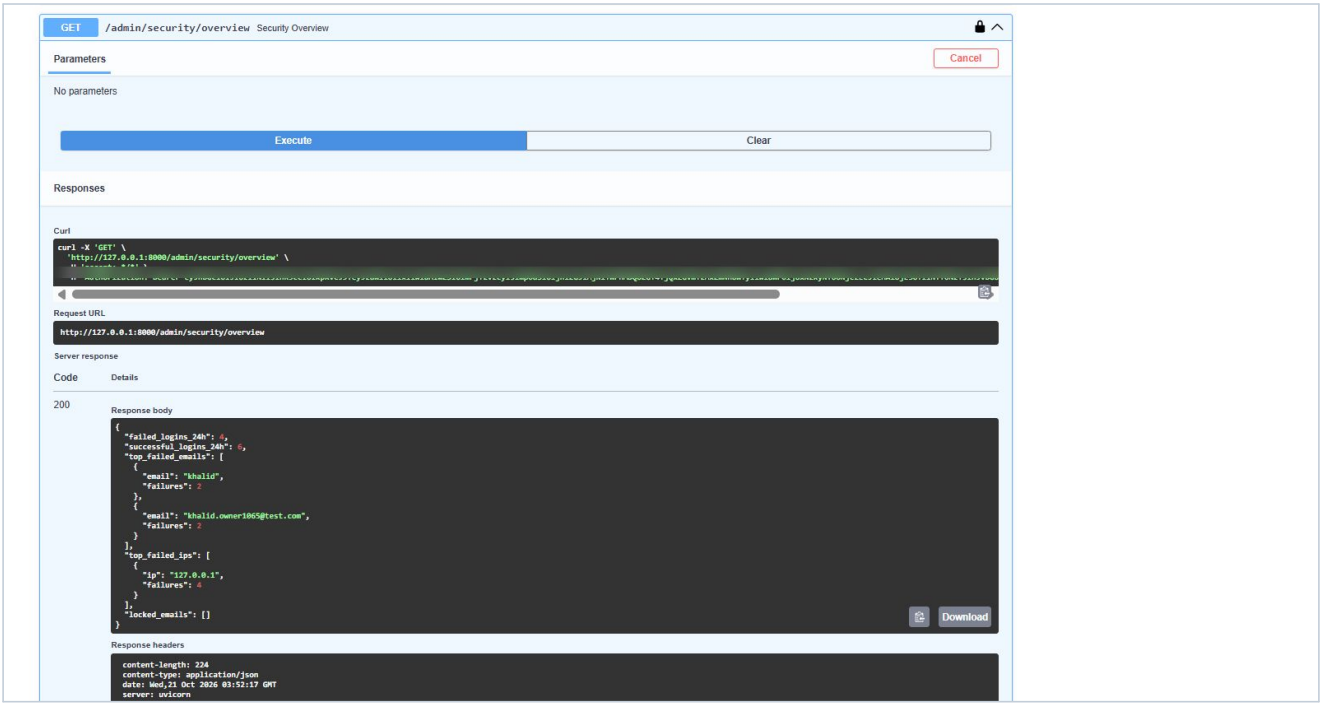
Day 4

Figure 95. Audit logs - GET /admin/audit-logs



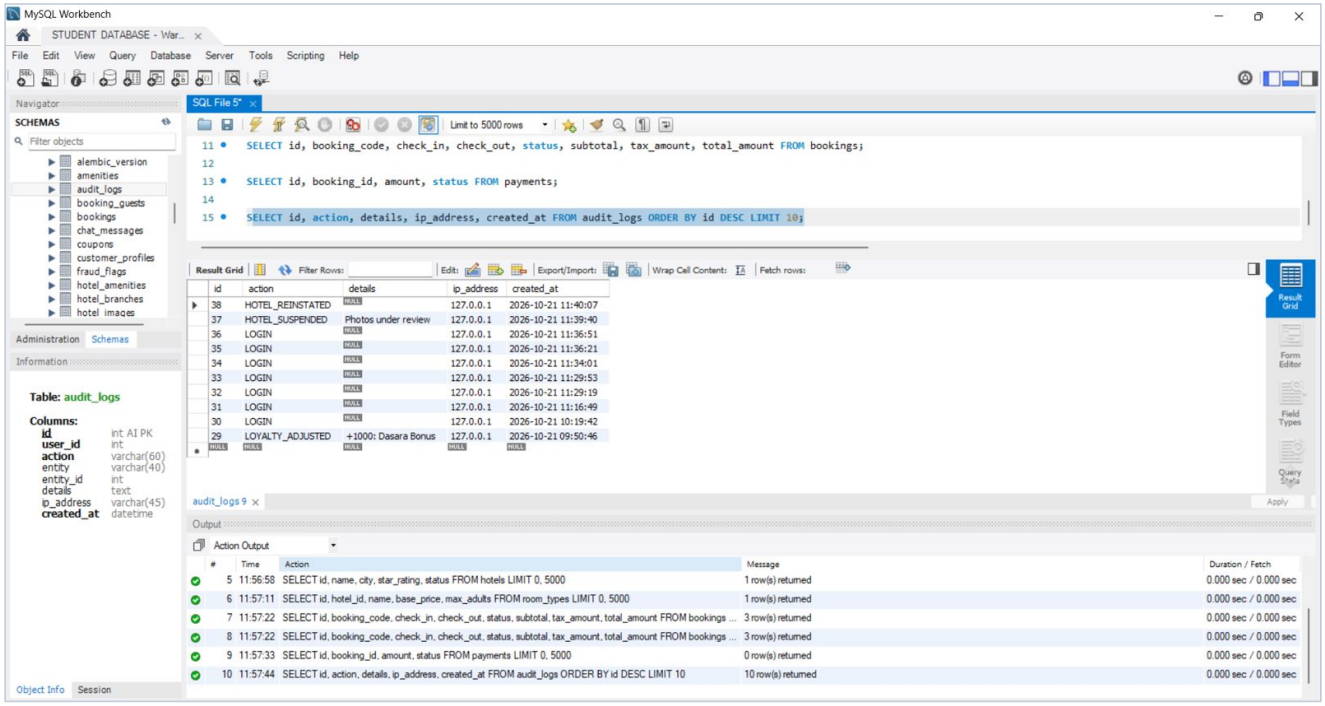
The admin reads the last 7 days of audit logs, up to 100 entries.
Each entry records user, action (LOGIN, LOGIN_FAILED), entity, details, IP address 127.0.0.1 and time.

Figure 96. Security overview - GET /admin/security/overview



A security summary for the last 24 hours.
It shows 4 failed and 6 successful logins, the emails and IPs with most failures, and locked emails.

Figure 97. audit_logs table



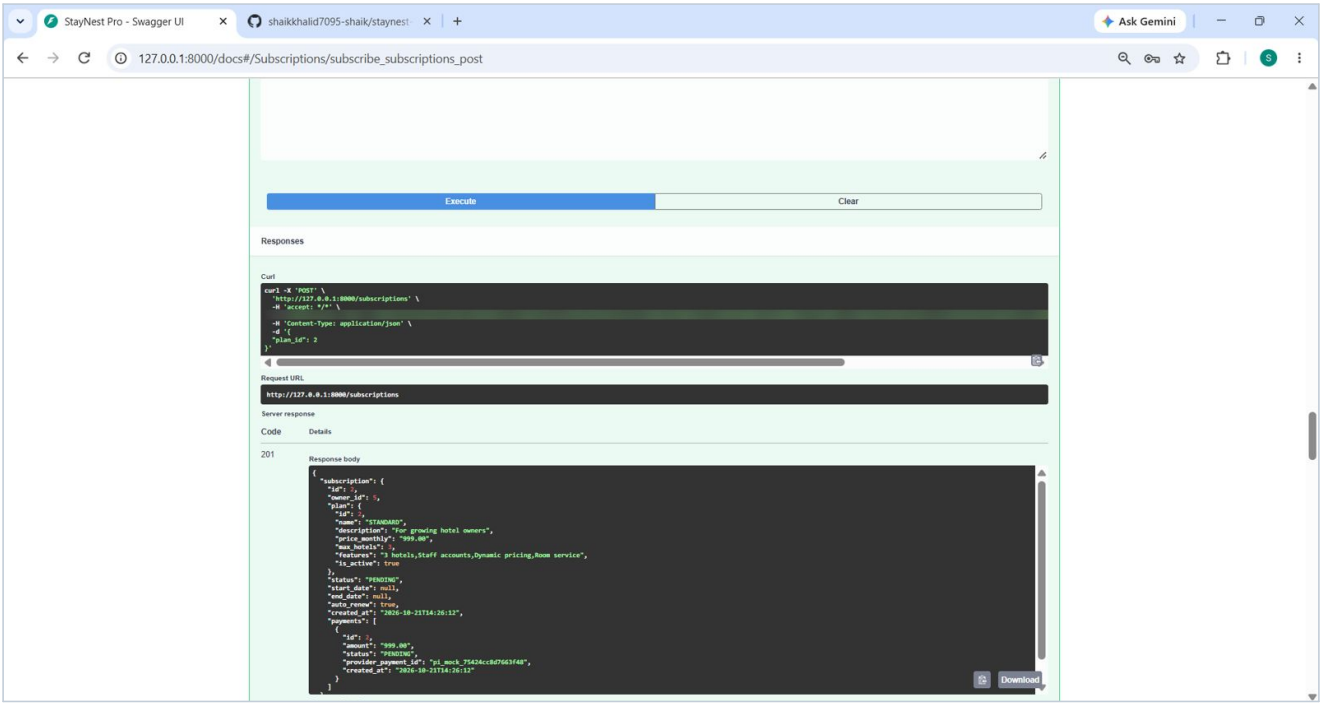
The latest 10 audit entries are read in descending order.
LOGIN, HOTEL_SUSPENDED (Photos under review), HOTEL_REINSTATED and LOYALTY_ADJUSTED are all recorded with IP and time.

Module 40. Hotel Subscription Plans

Day 3

Also shown in Figure 84.

Figure 98. Subscribe to a plan - POST /subscriptions



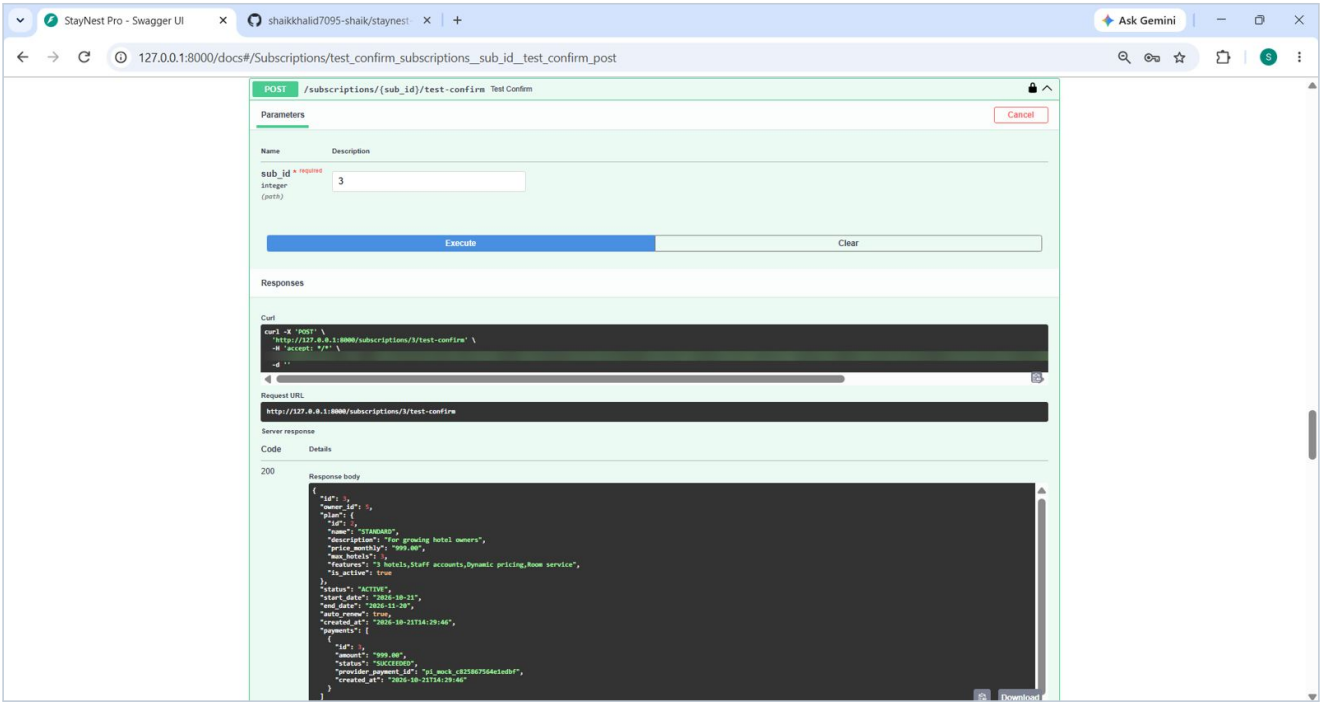
The owner subscribes to the STANDARD plan (Rs 999 per month, up to 3 hotels).
201 Created with status PENDING and a mock Stripe payment waiting to be confirmed.

Module 41. Hotel Subscription Payments

Day 3

Also shown in Figure 98.

Figure 99. Pay the subscription - test-confirm

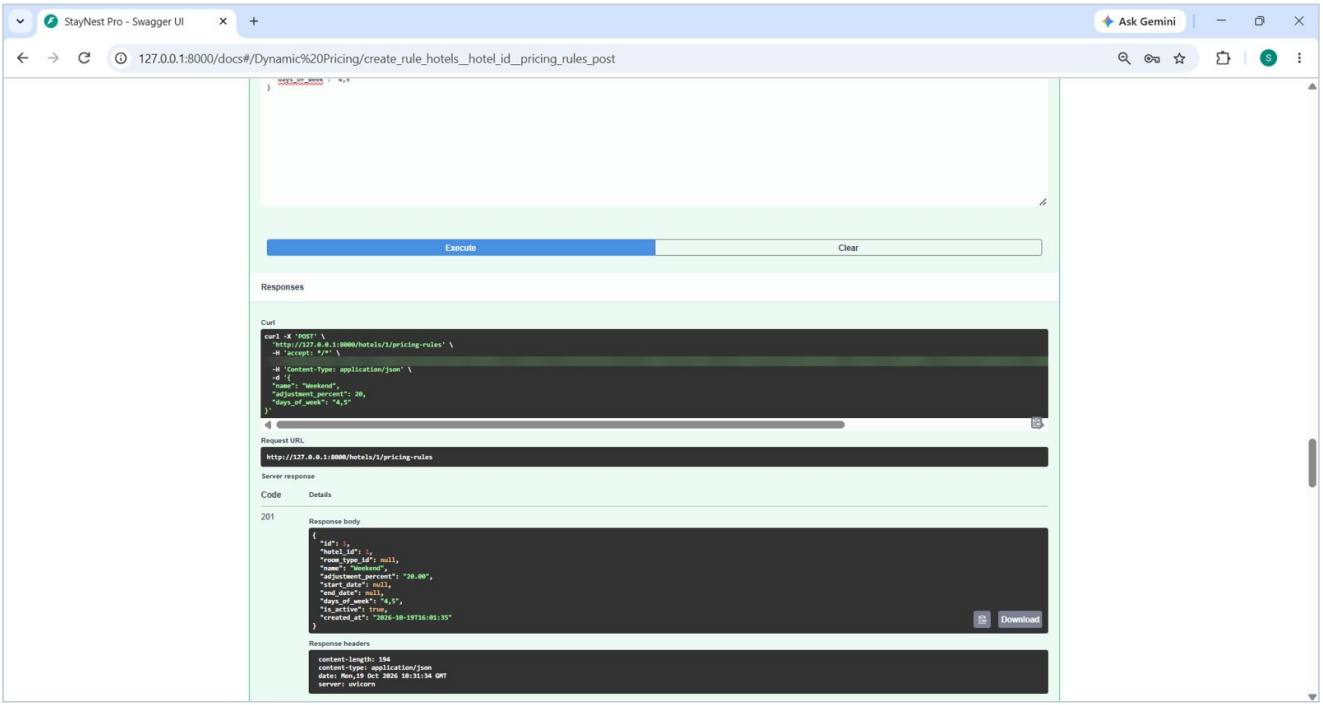


POST /subscriptions/3/test-confirm completes the Rs 999 payment through mock Stripe.
The subscription becomes ACTIVE from 21 October to 20 November 2026 and the payment is SUCCEEDED.

Module 42. Dynamic Pricing Management

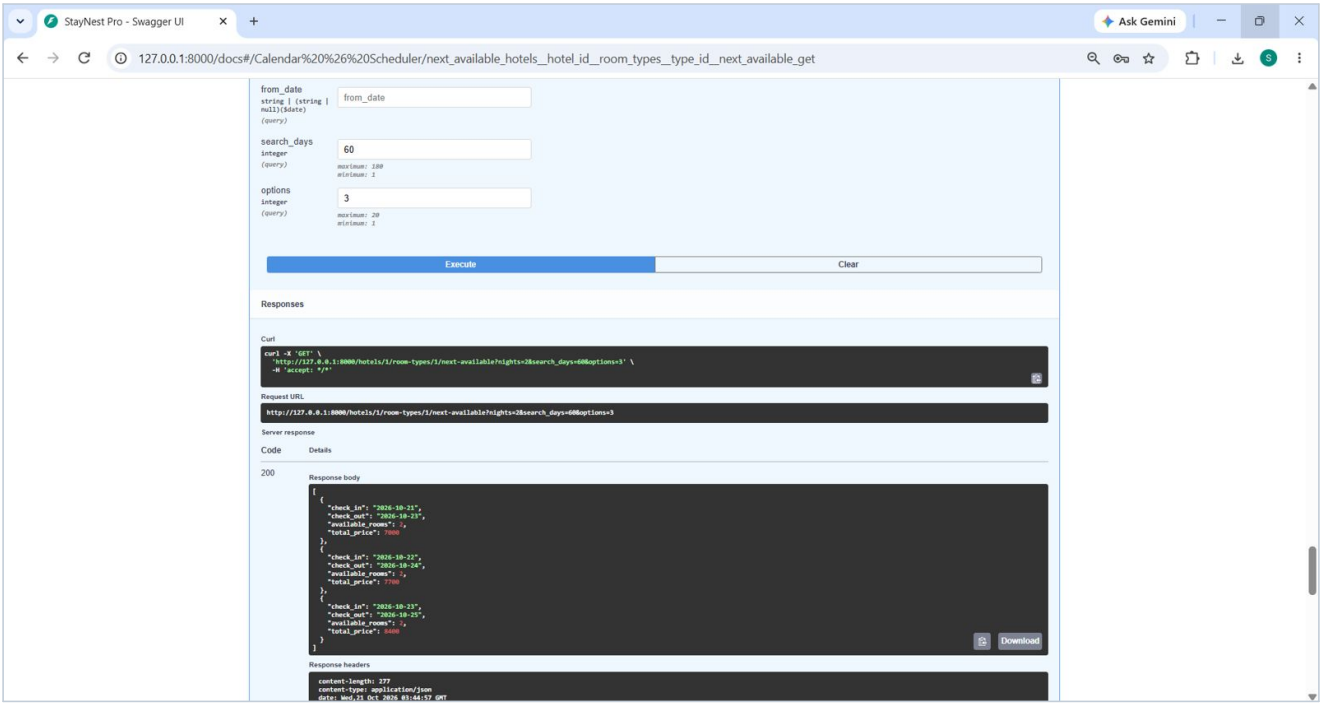
Day 3

Figure 100. Dynamic pricing rule - POST /hotels/1/pricing-rules



The owner creates a Weekend rule that adds 20% on days 4 and 5 of the week.
The API returns 201 Created with rule id 1 for hotel 1, active immediately.

Figure 101. Next available dates - GET /hotels/1/room-types/1/next-available

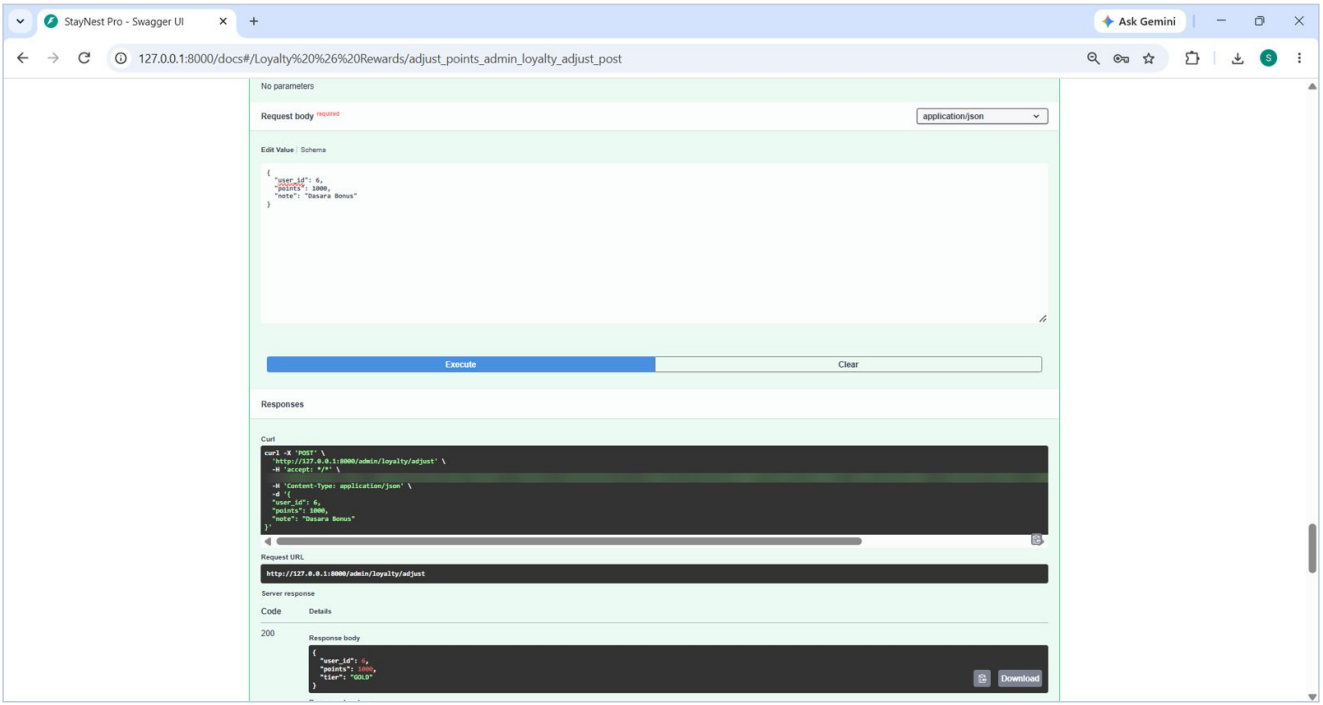


The guest asks for the next 3 free 2-night stays within the coming 60 days.
Options start 21, 22 and 23 October 2026 with 2 rooms each, at Rs 7000, 7700 and 8400 as weekend pricing applies.

Module 43. Loyalty and Rewards

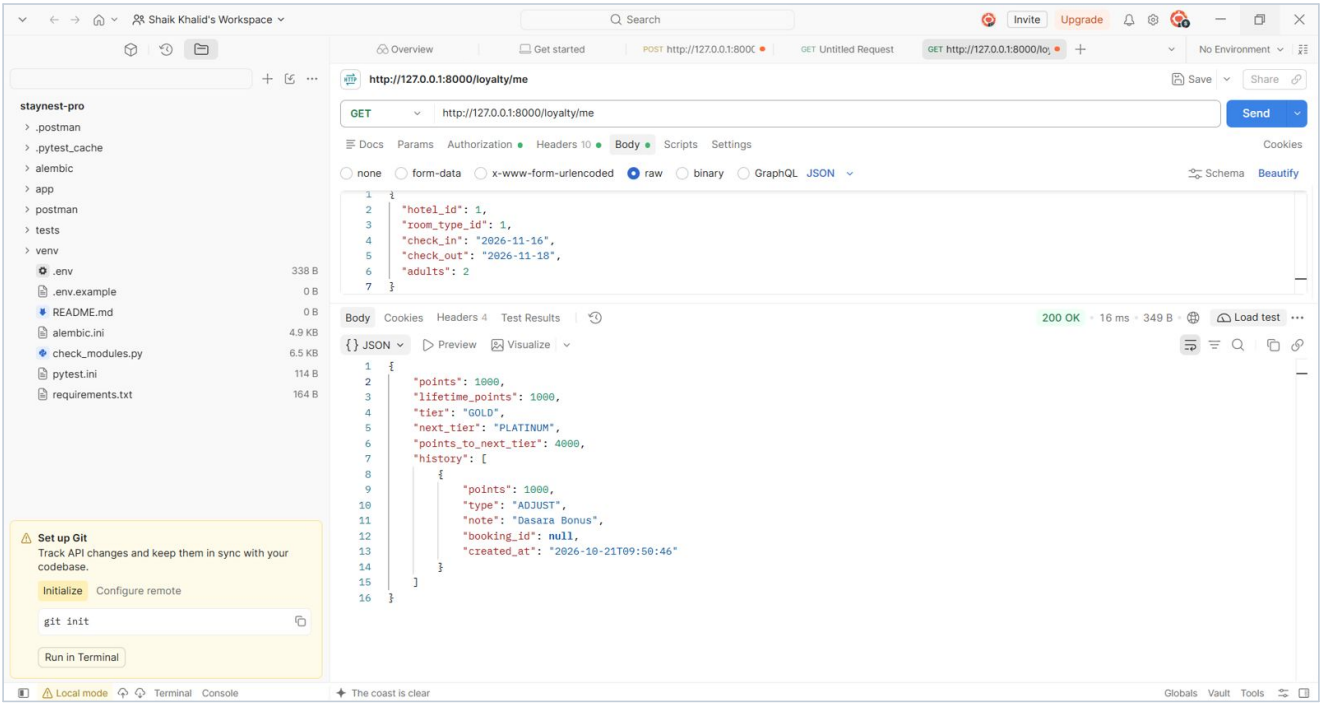
Day 4

Figure 102. Loyalty bonus - POST /admin/loyalty/adjust



The admin gives customer 6 a Dasara Bonus of 1000 points.
The customer now has 1000 points and moves up to the GOLD tier.

Figure 103. Postman - loyalty points (GET /loyalty/me)

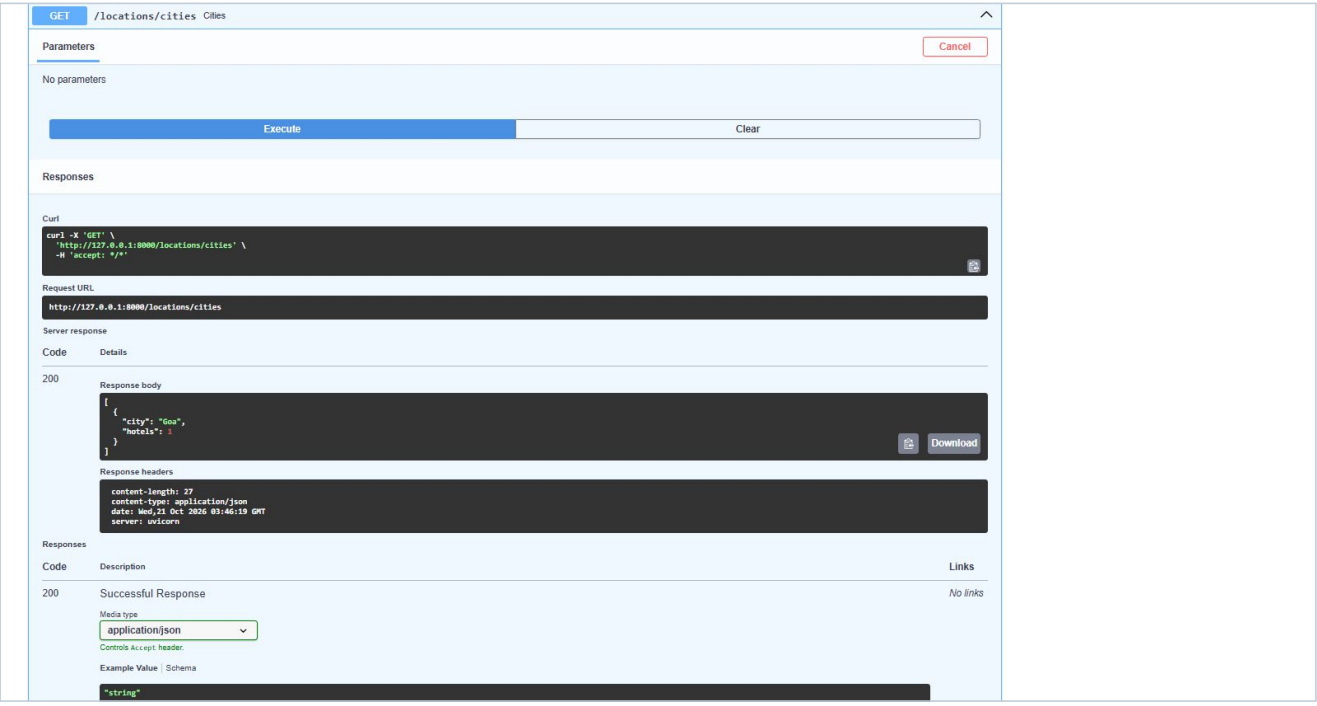


The customer checks their loyalty account.
1000 points, GOLD tier, 4000 points to PLATINUM, with the Dasara Bonus in the history.

Module 44. Location Services

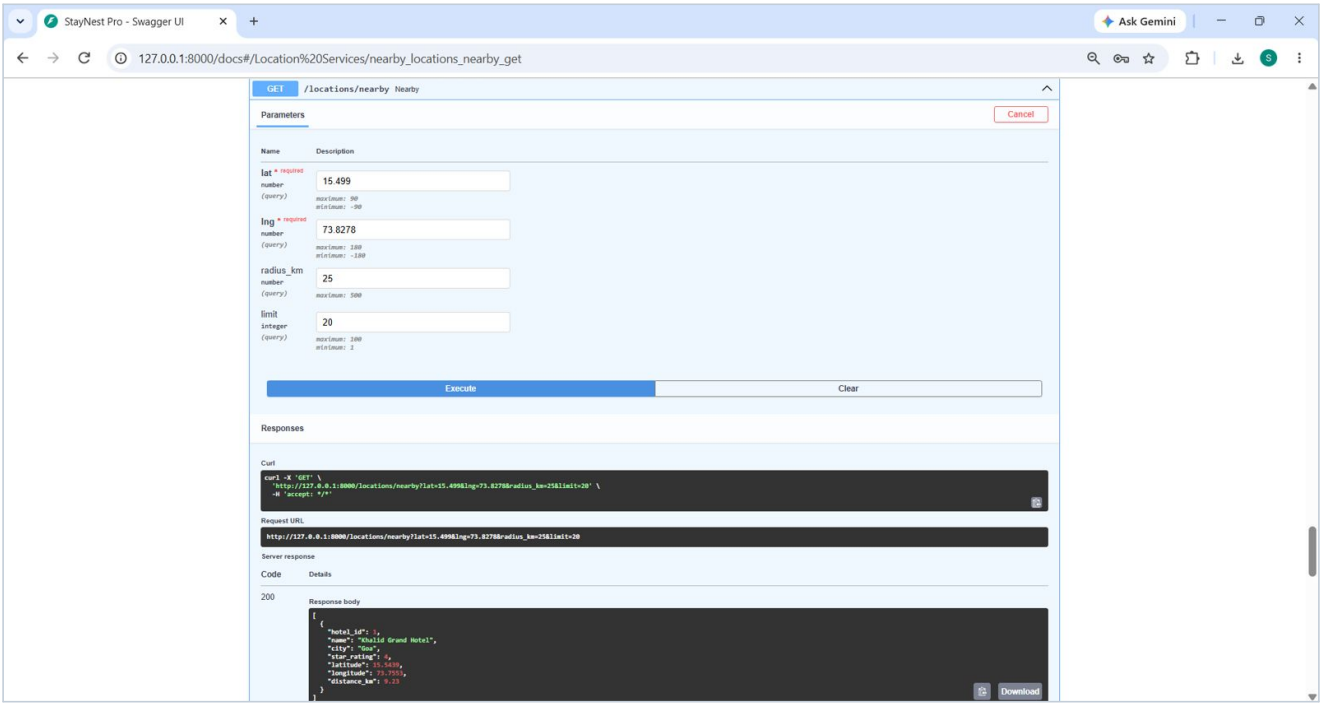
Day 4

Figure 104. Cities - GET /locations/cities



A public list of cities that have approved hotels, used to fill a city dropdown.
Goa is returned with 1 hotel.

Figure 105. Nearby hotels - GET /locations/nearby



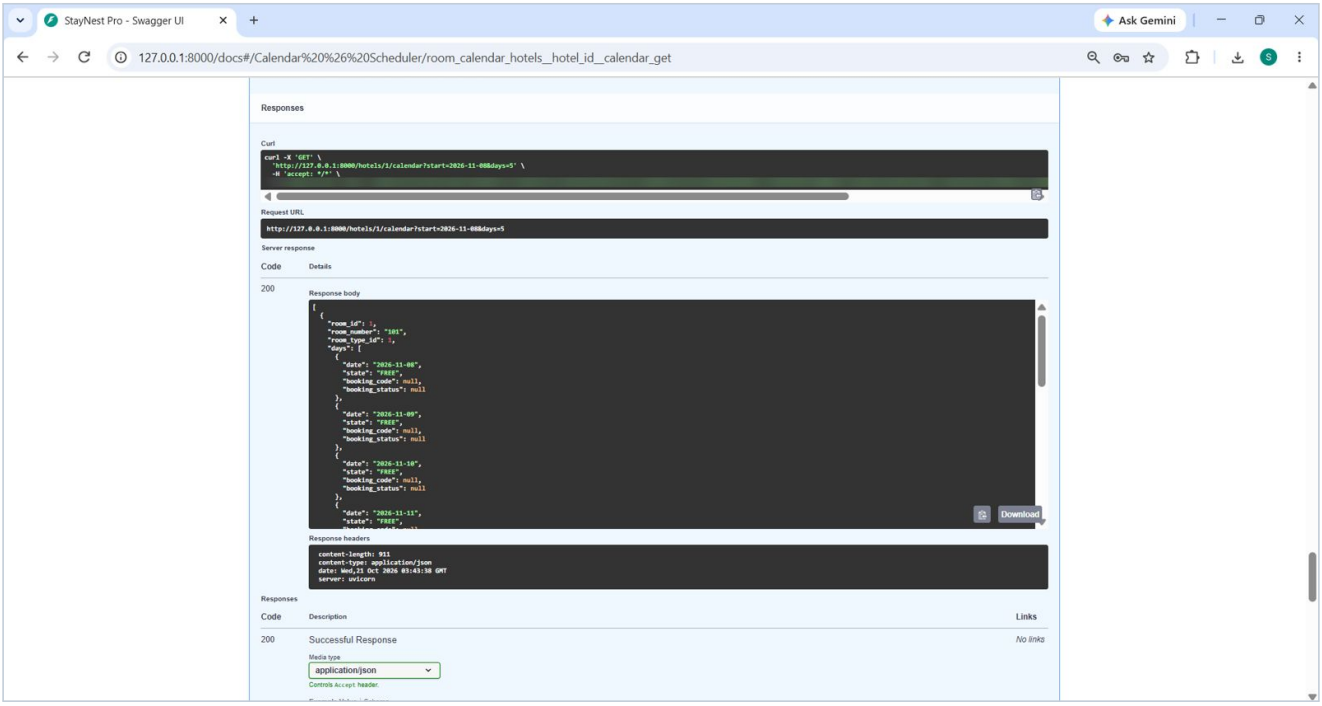
Hotels are searched within 25 km of latitude 15.499, longitude 73.8278.
Khalid Grand Hotel is found 9.23 km away, calculated from its saved map location.

Module 45. Calendar and Booking Scheduler

Day 4

Also shown in Figure 101.

Figure 106. Room calendar - GET /hotels/1/calendar

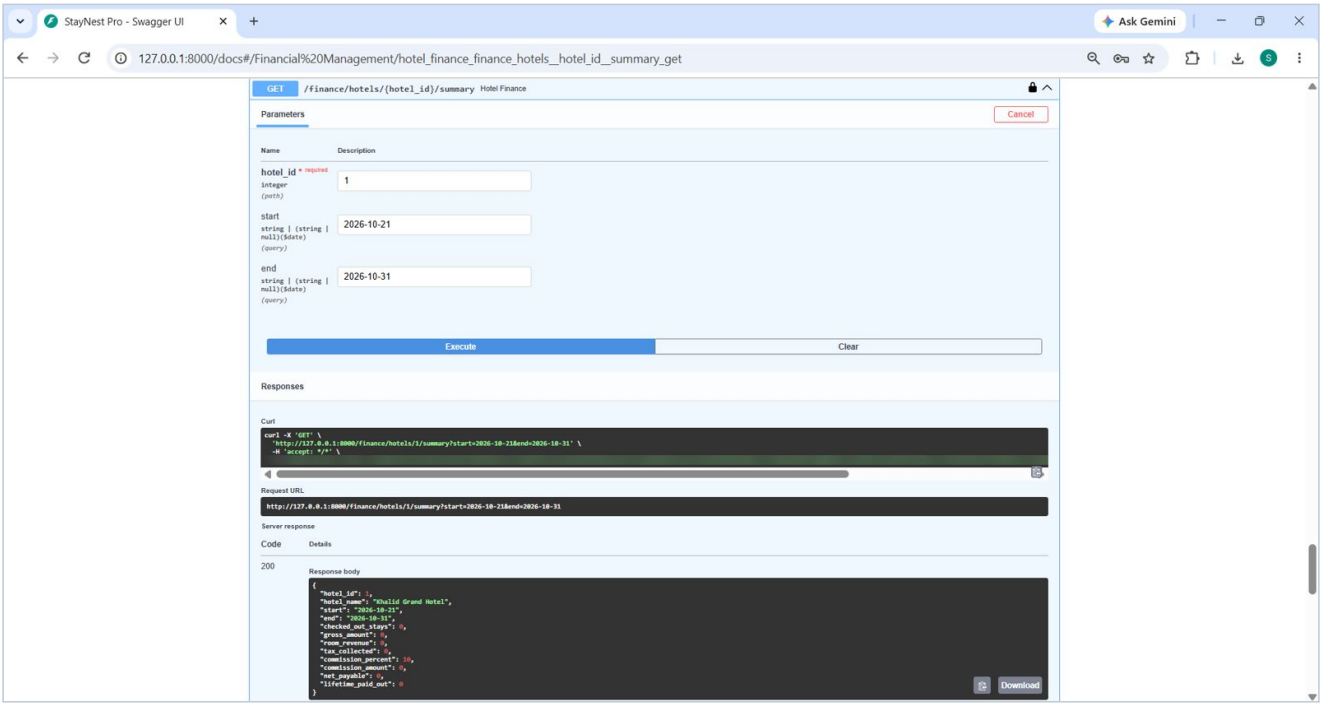


The owner sees each room day by day for 5 days starting 8 November 2026.
Room 101 shows FREE on every date, with booking code and status filled when a day is booked.

Module 46. Financial Management

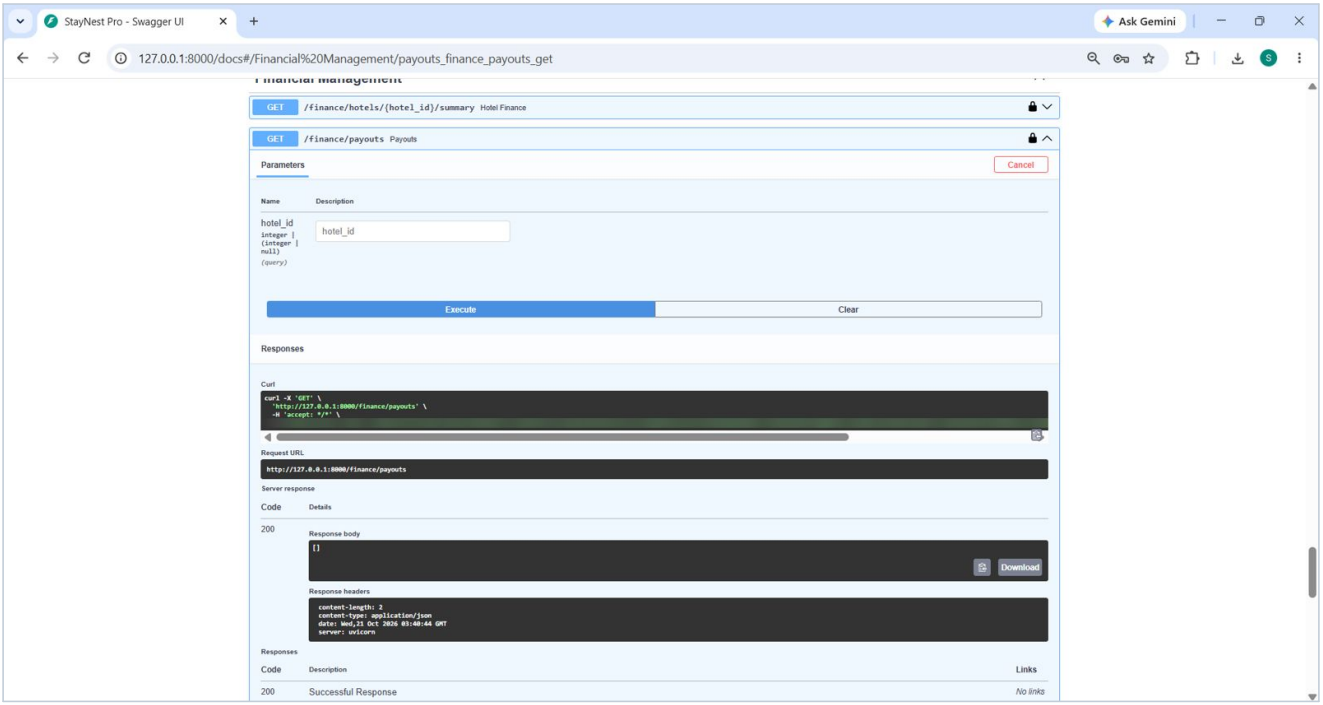
Day 4

Figure 107. Hotel finance summary - GET /finance/hotels/1/summary



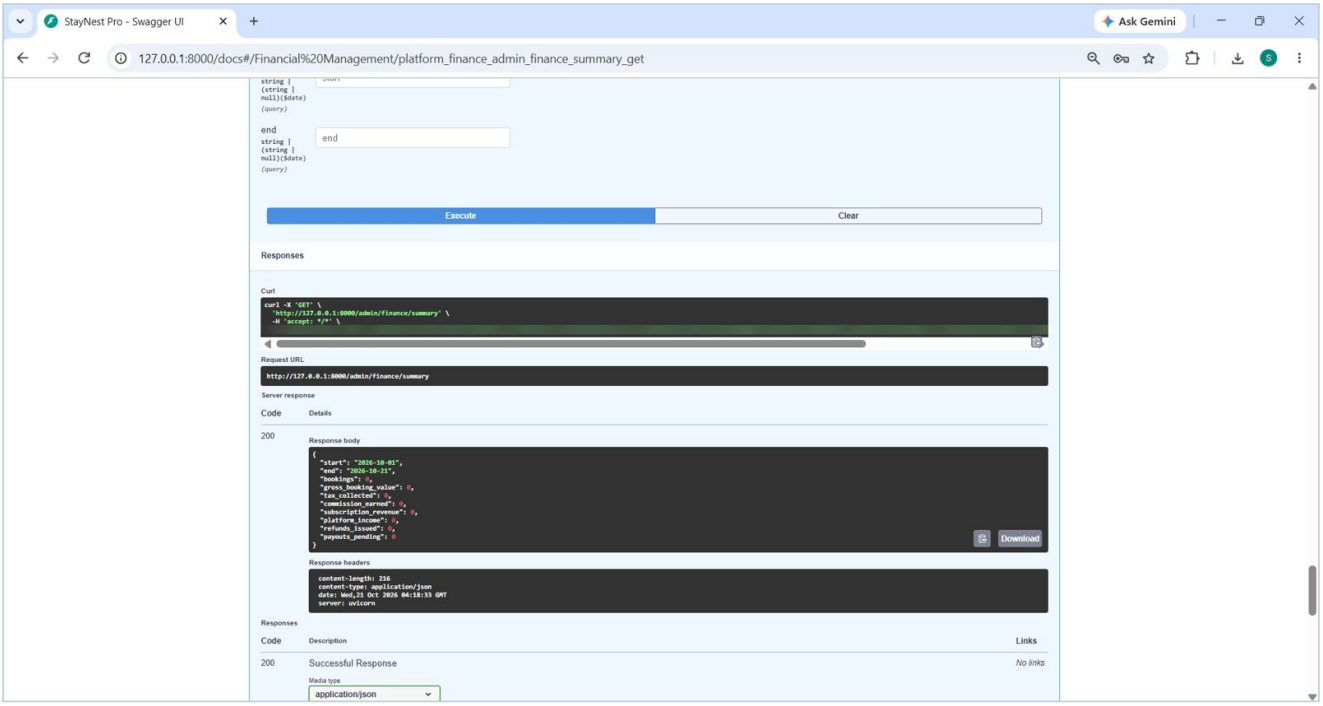
The owner checks finances for Khalid Grand Hotel from 21 to 31 October 2026.
It shows gross amount, room revenue, tax, the 10% platform commission, net payable and lifetime paid out.

Figure 108. Payouts - GET /finance/payouts



The owner lists the payouts the platform has made to them.
The list is empty because no booking has been paid and checked out yet.

Figure 109. Platform finance - GET /admin/finance/summary

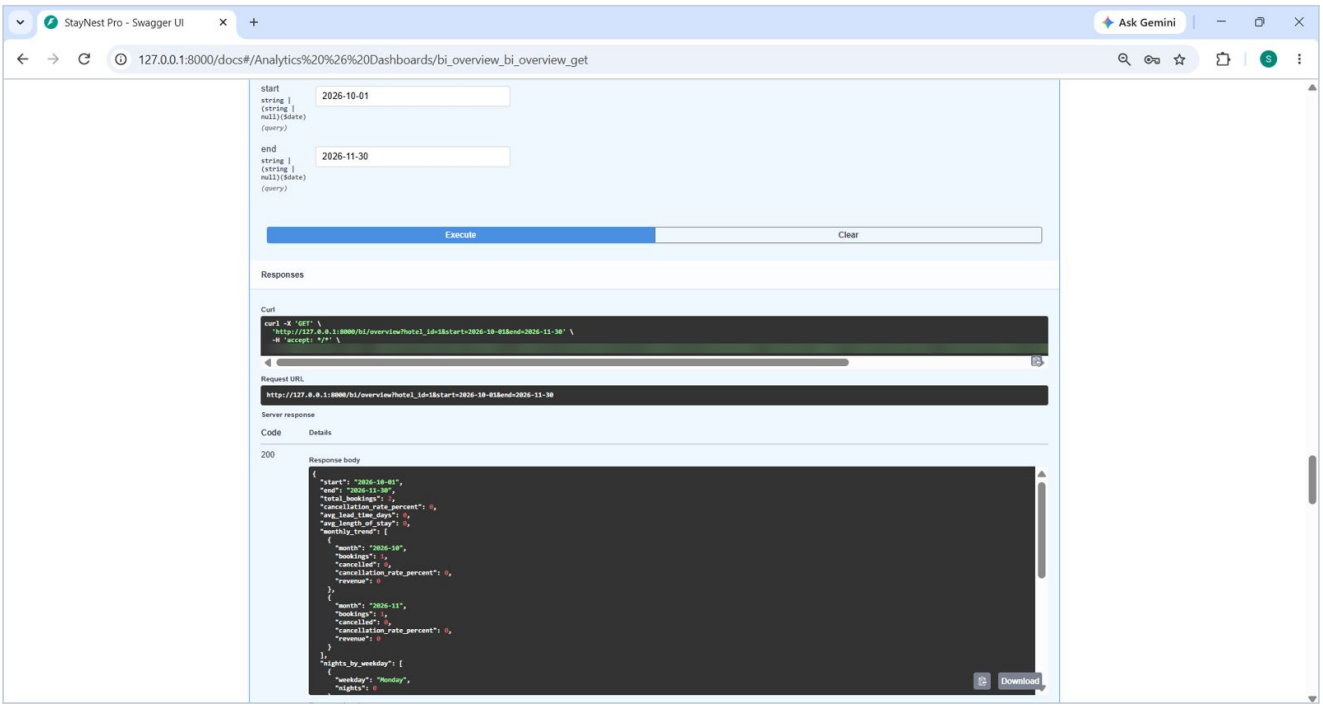


The admin sees platform money for 1 to 21 October 2026.
Bookings, gross value, tax, commission, subscription revenue, platform income, refunds and pending payouts are all shown.

Module 47. Business Intelligence APIs

Day 4

Figure 110. Business intelligence - GET /bi/overview

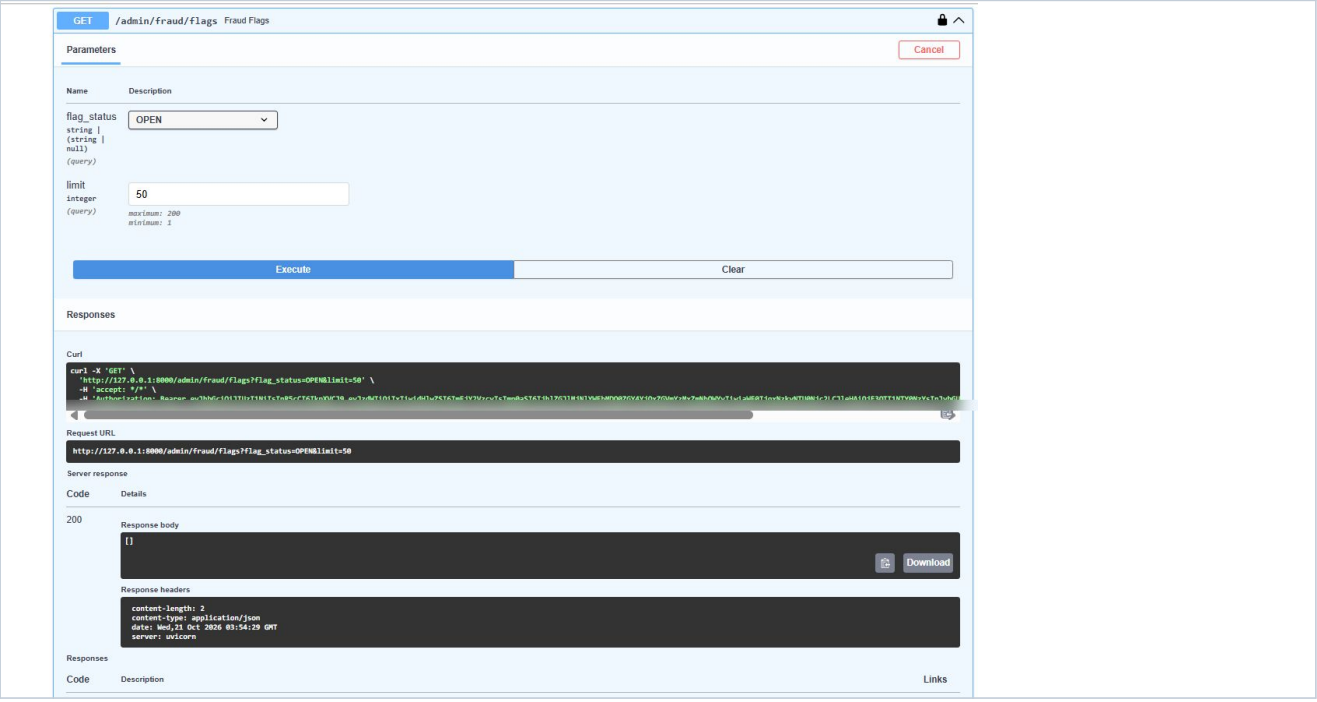


A BI overview for hotel 1 from 1 October to 30 November 2026.
It shows 2 bookings, cancellation rate, lead time, length of stay, a monthly trend and nights by weekday.

Module 48. Fraud Detection and Security

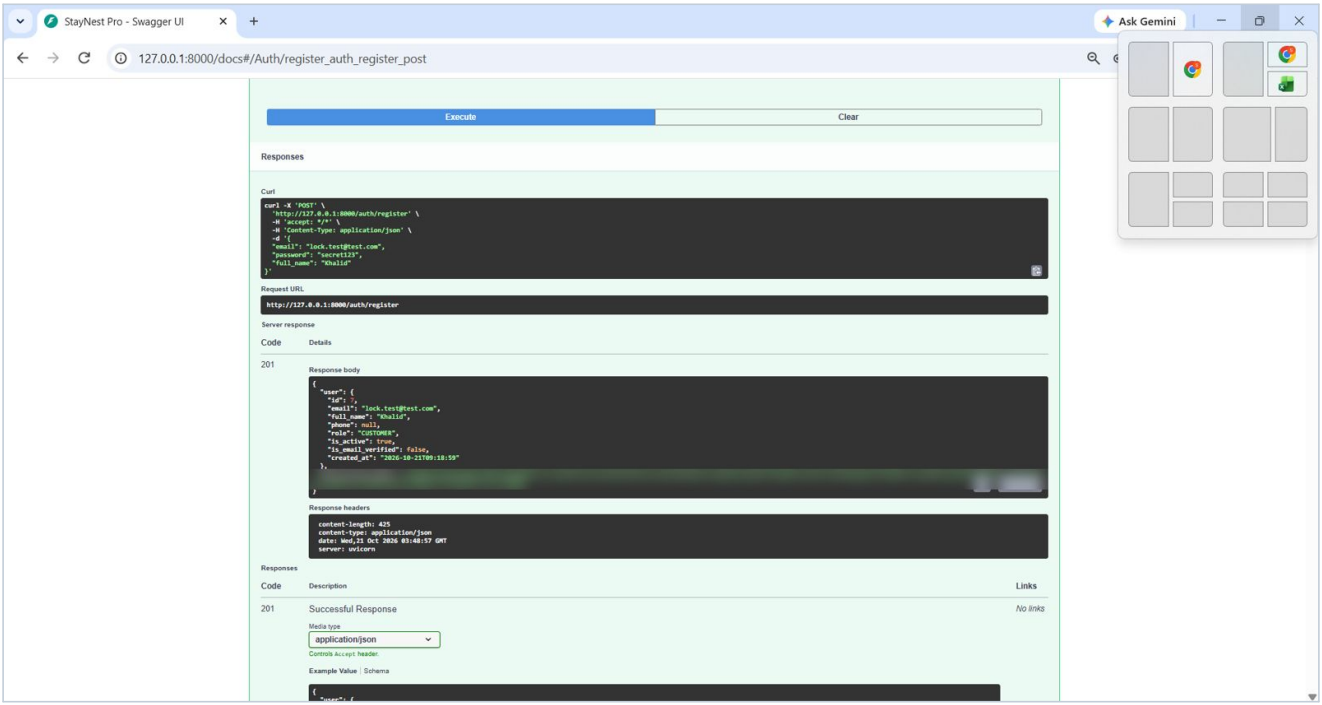
Day 4

Figure 111. Fraud flags - GET /admin/fraud/flags



The admin lists OPEN fraud flags, up to 50.
The list is empty, which means no suspicious bookings or payments were flagged.

Figure 112. Register a lockout test user

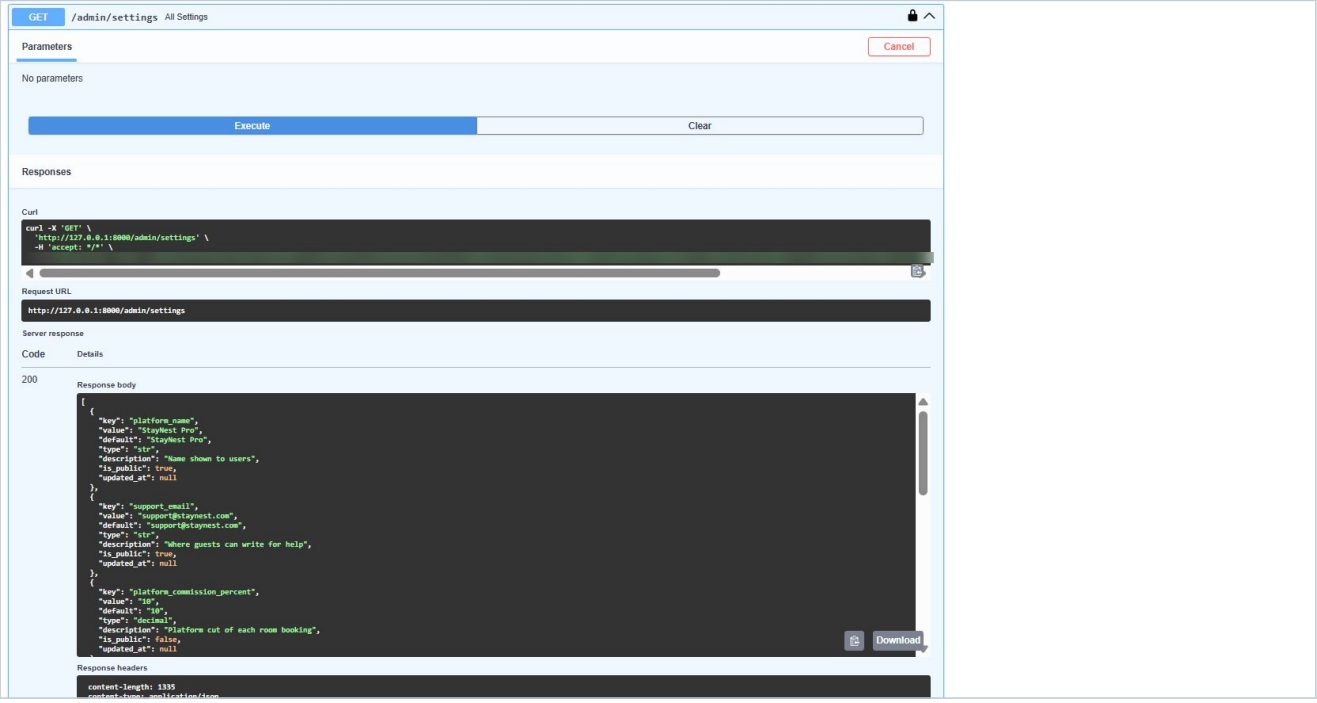


lock.test@test.com is registered to test the account lock after repeated failed logins.
The 201 response returns user id 7 with role CUSTOMER (verification token blurred).

Module 49. System Configuration

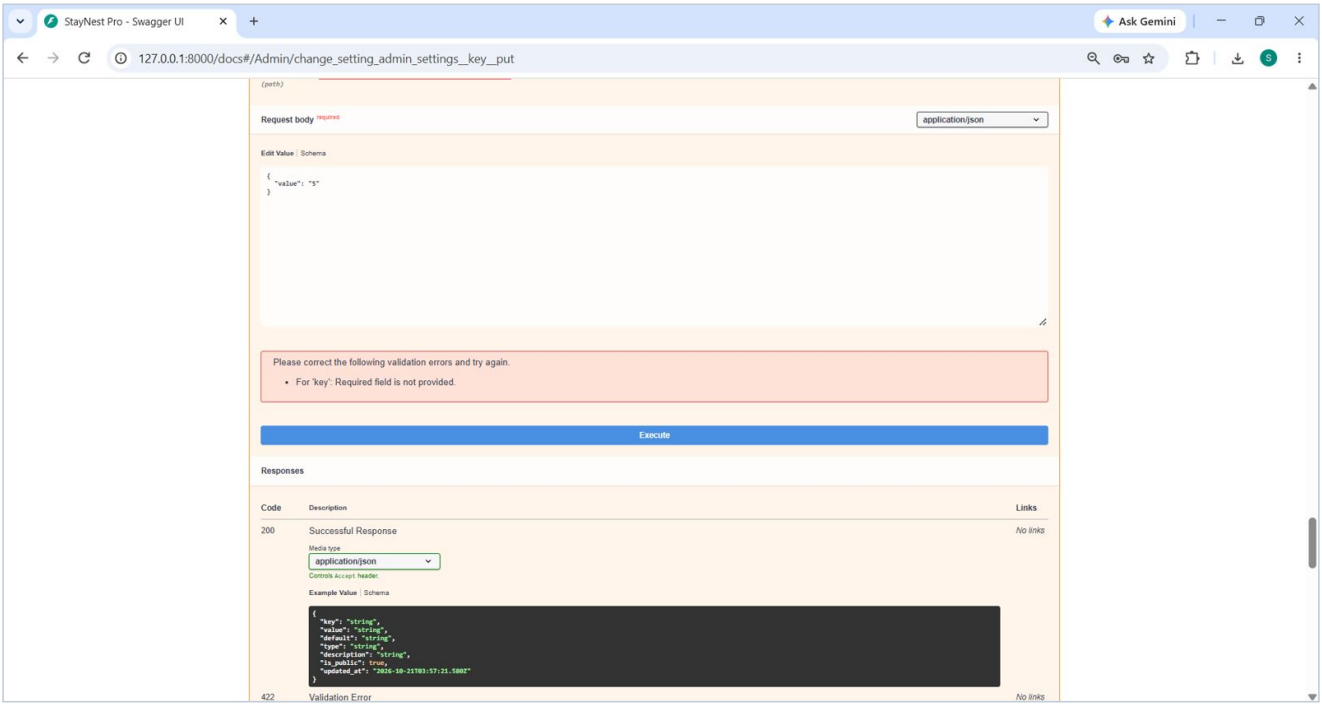
Day 4

Figure 113. Platform settings - GET /admin/settings



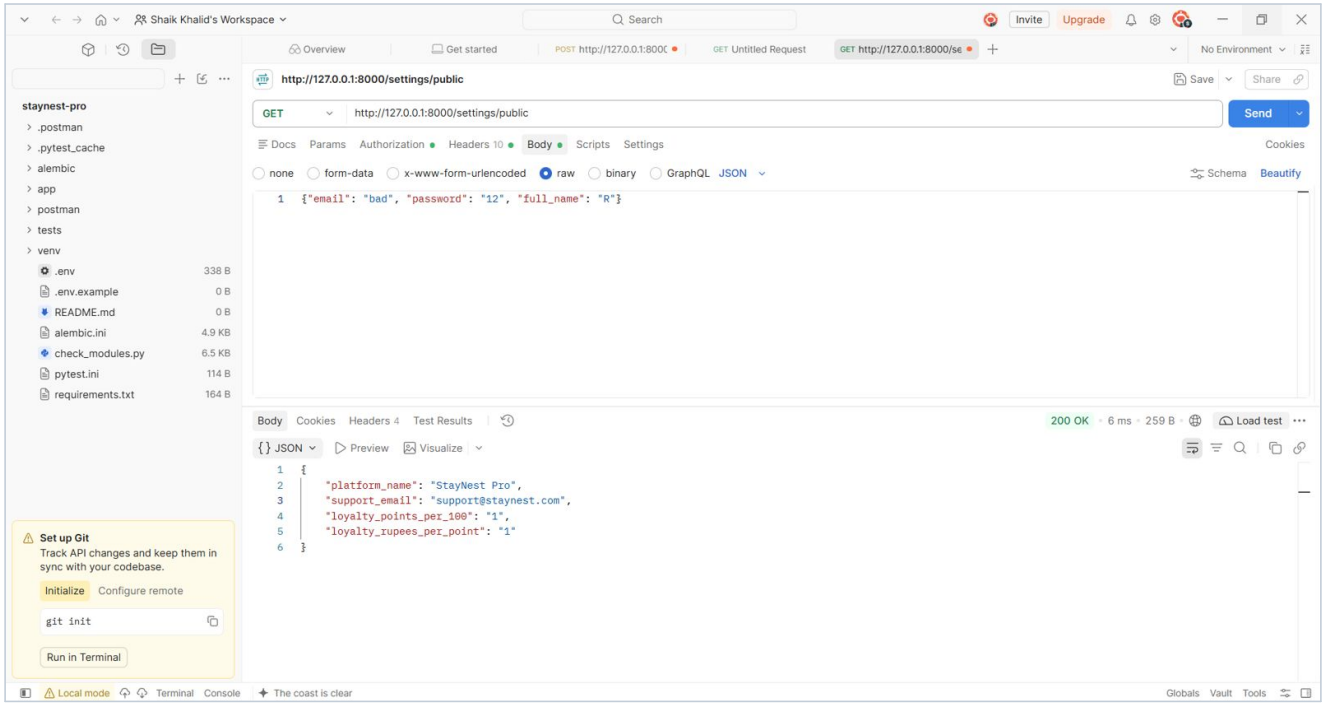
The admin reads all platform settings with their value, default, type and description.
Examples are platform_name StayNest Pro, support_email and platform_commission_percent 10.

Figure 114. Change a setting - validation check



PUT /admin/settings/{key} is tried without the key path parameter.
Swagger stops the request: For 'key': Required field is not provided.

Figure 115. Postman - public settings

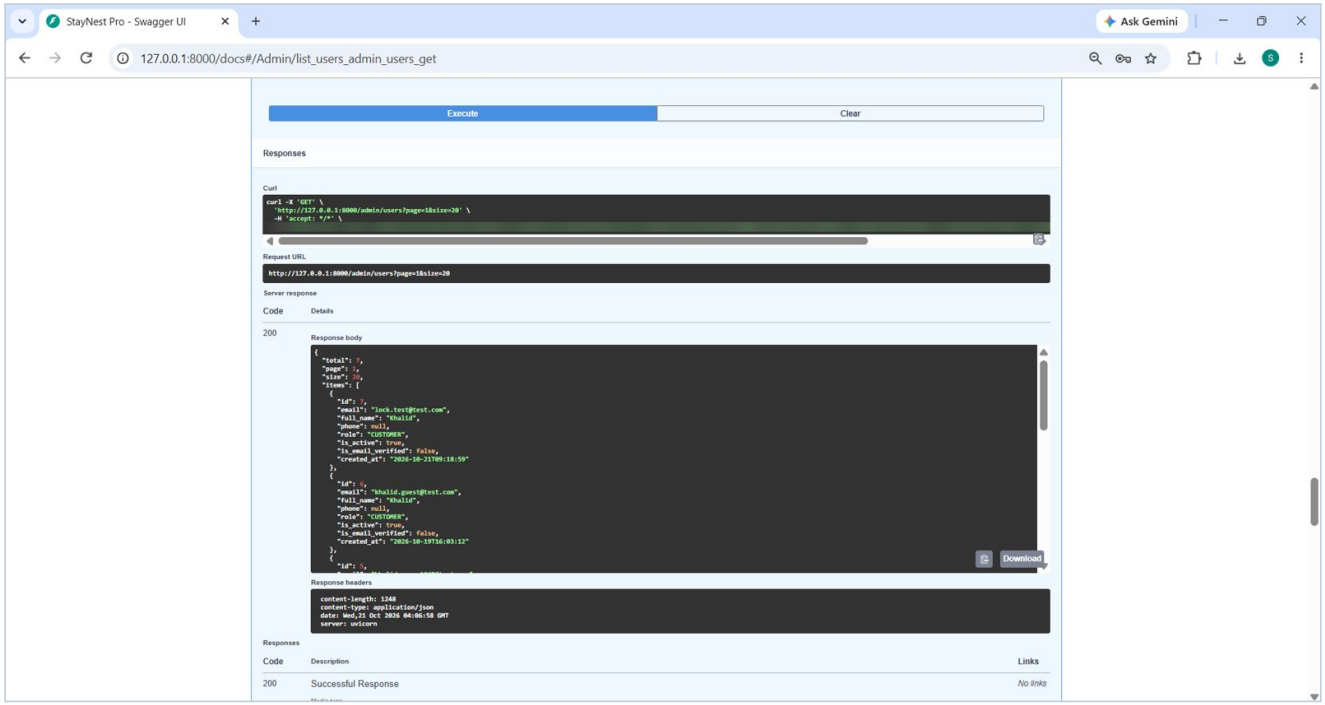


GET /settings/public needs no login.
Returns platform name, support email and loyalty rules (1 point per Rs 100).

Module 50. Advanced Platform Administration

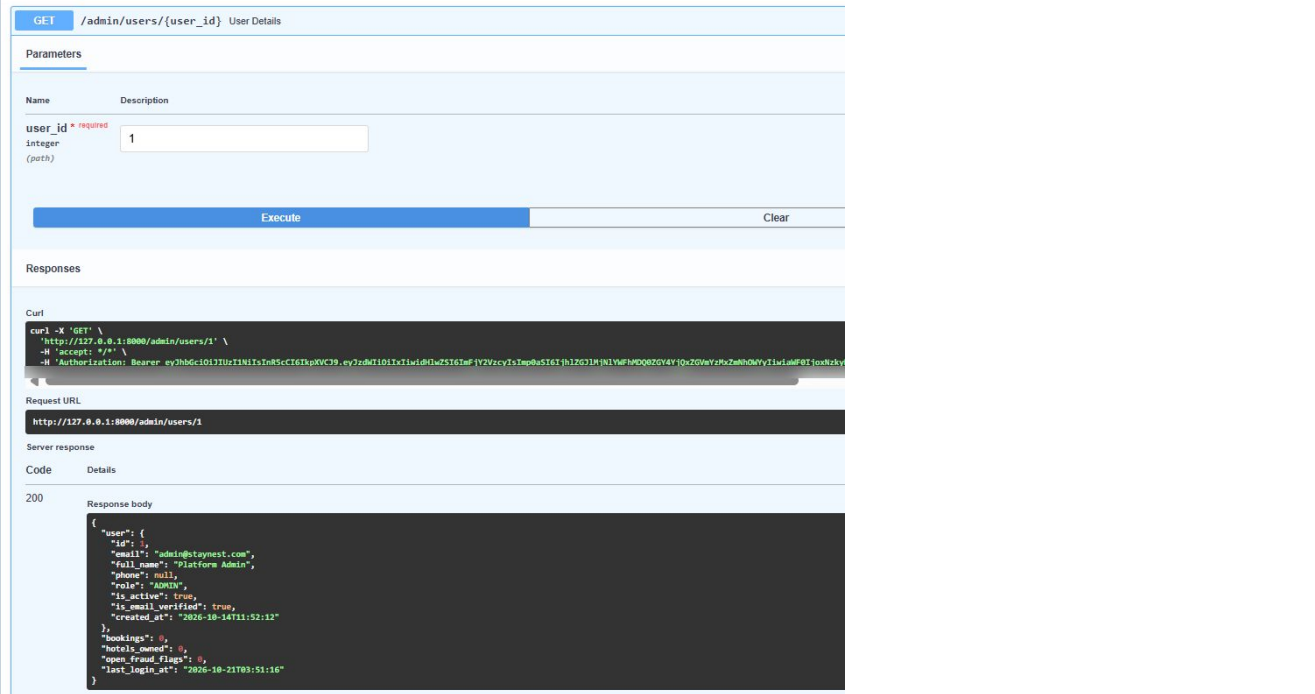
Day 4

Figure 116. List users - GET /admin/users



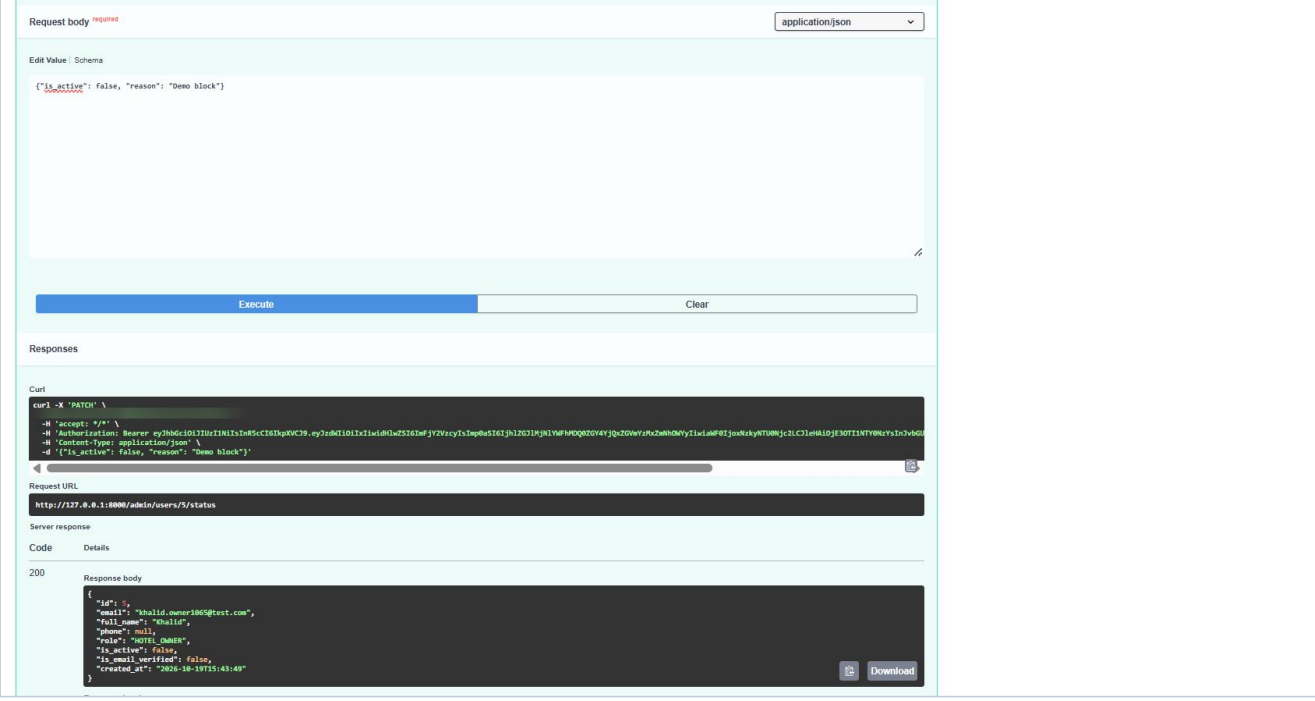
The admin lists all users with paging (page 1, size 20).
7 users are returned with email, role, active and email-verified status.

Figure 117. User details - GET /admin/users/1



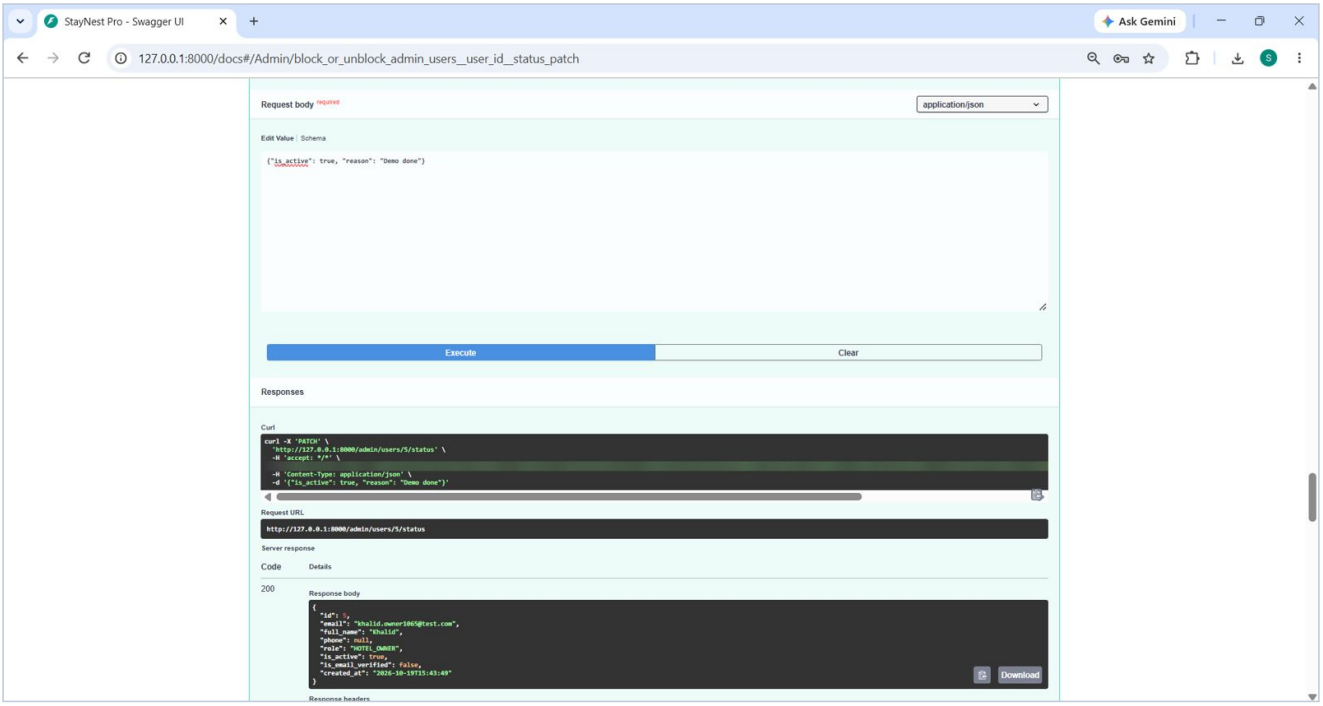
The admin opens one user's full record by id.
Platform Admin is returned with bookings, hotels owned, open fraud flags and last login time.

Figure 118. Block a user - PATCH /admin/users/5/status



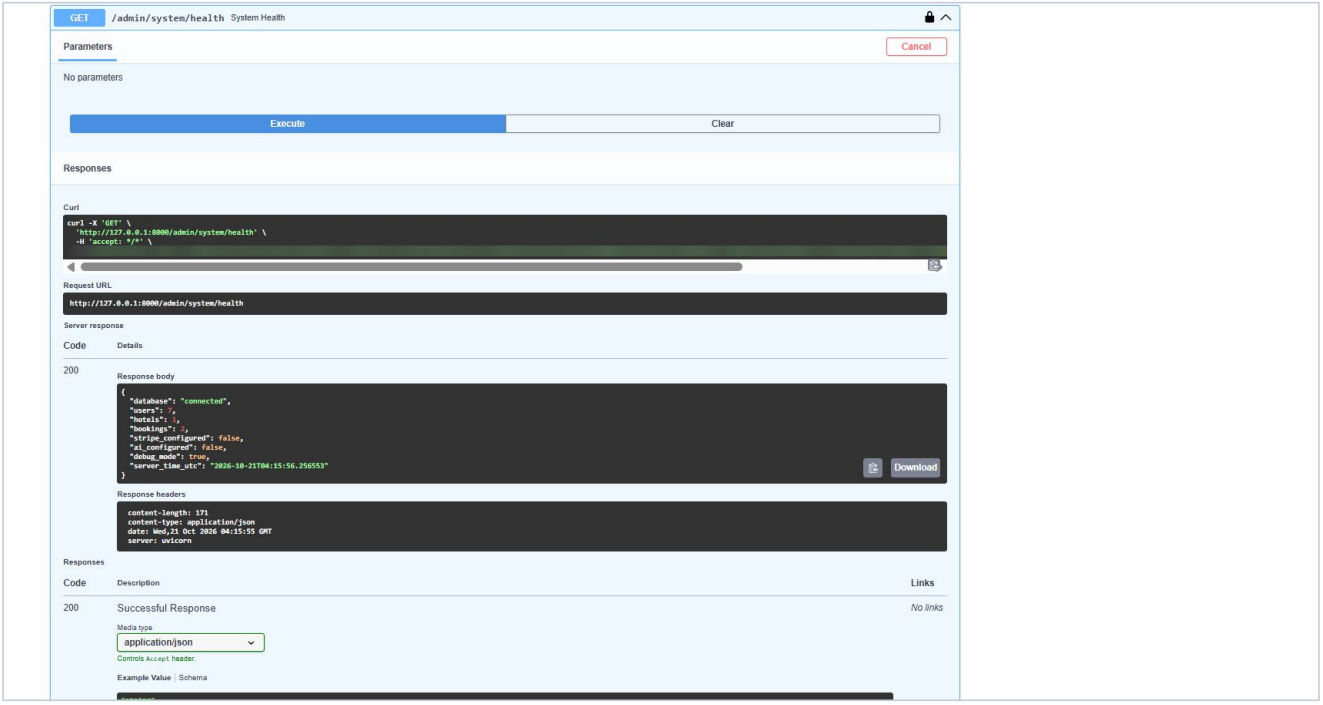
The admin blocks the hotel owner with is_active false and the reason Demo block.
User 5 is returned with is_active false, so this owner can no longer log in.

Figure 119. Unblock a user - PATCH /admin/users/5/status



The same call with is_active true and reason Demo done restores the account.
User 5 (HOTEL_OWNER) is active again.

Figure 120. System health - GET /admin/system/health

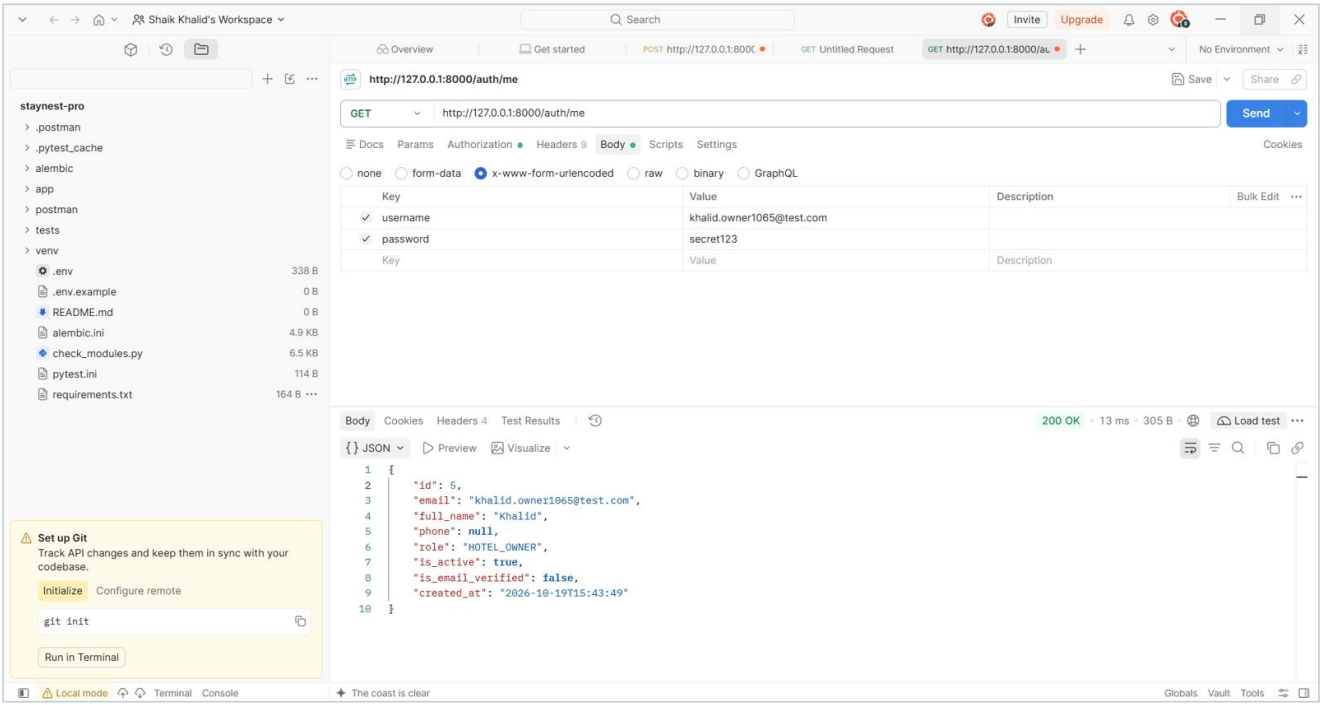


The admin checks that the server and database are working.
Database connected, 7 users, 1 hotel, 2 bookings, debug mode on, and Stripe and AI keys not configured.

Part 4 - Postman API Testing

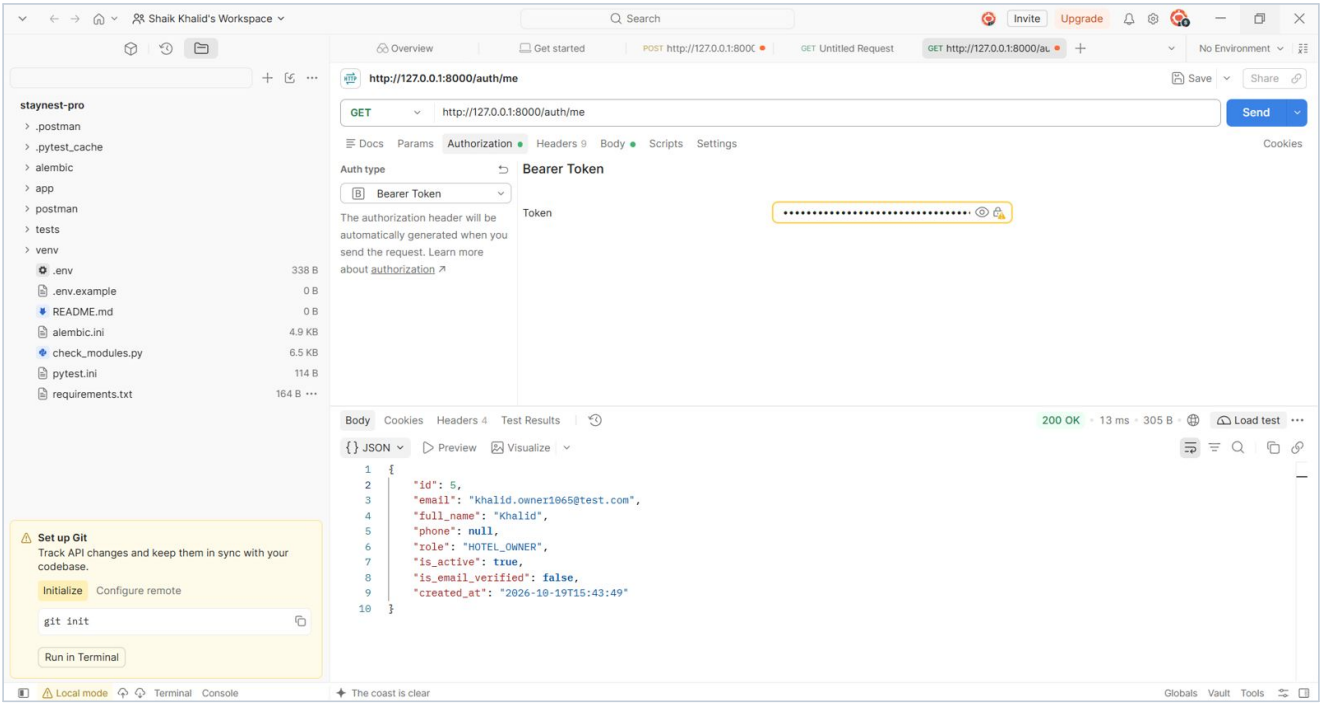
The same APIs were tested in Postman with Bearer tokens, form login and error cases.

Figure 121. Postman - logged-in user (GET /auth/me)



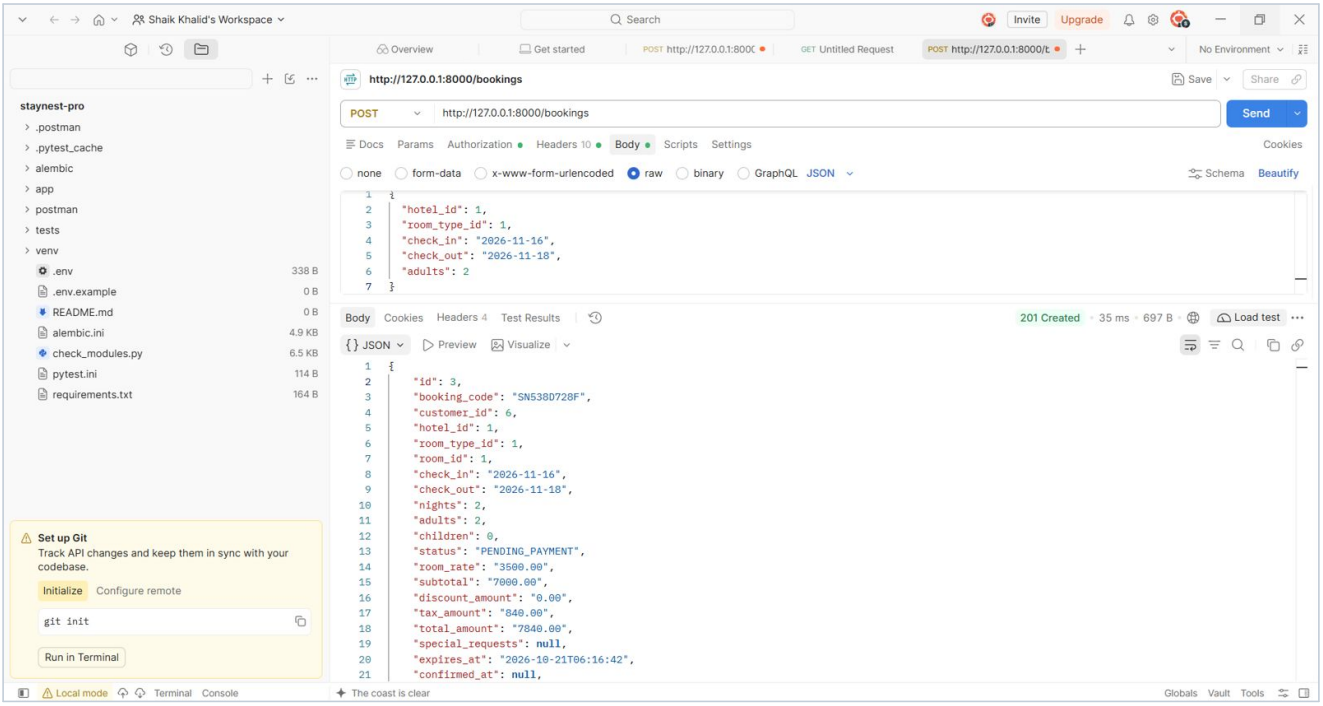
Postman uses the owner token saved under Authorization as a Bearer Token.
The API returns user 5, khalid.owner1065@test.com, with role HOTEL_OWNER, and 200 OK in 13 ms.

Figure 122. Postman - Bearer Token setup



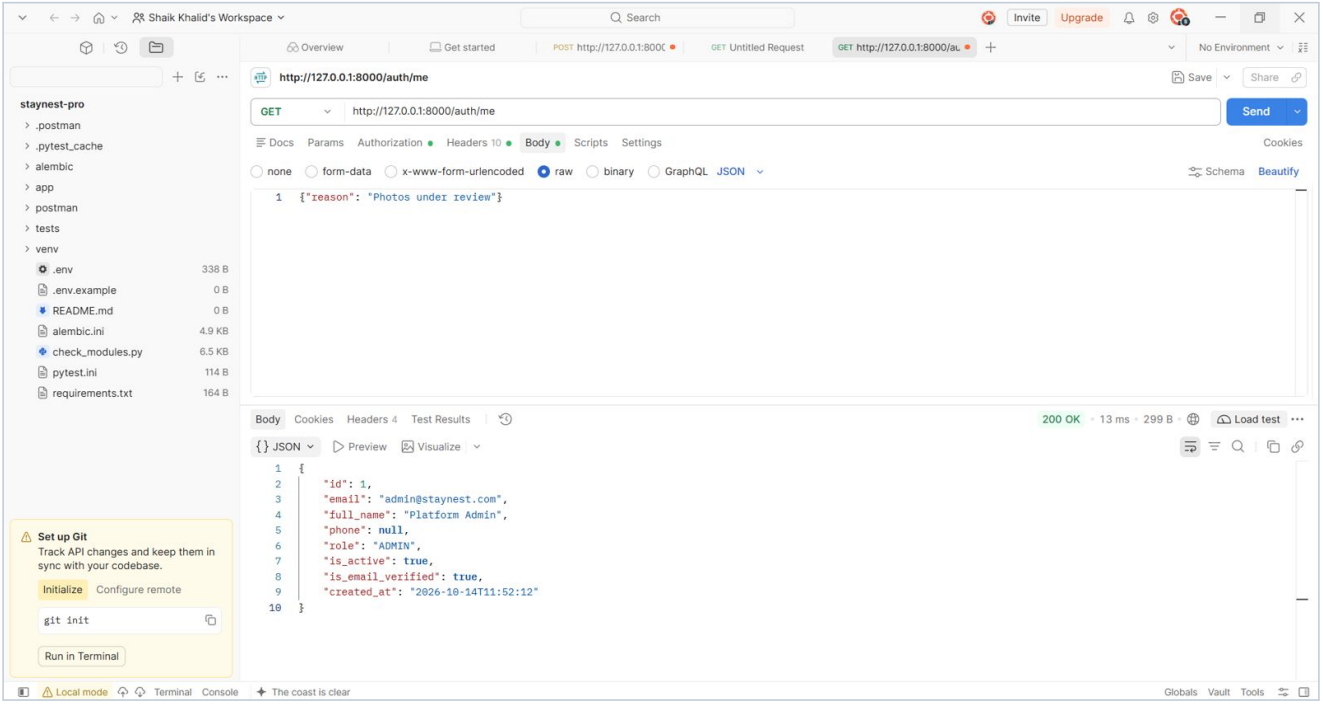
The Authorization tab is set to Bearer Token, and the token is hidden as dots.
Postman adds the Authorization header to every request automatically.

Figure 123. Postman - create a booking (201 Created)



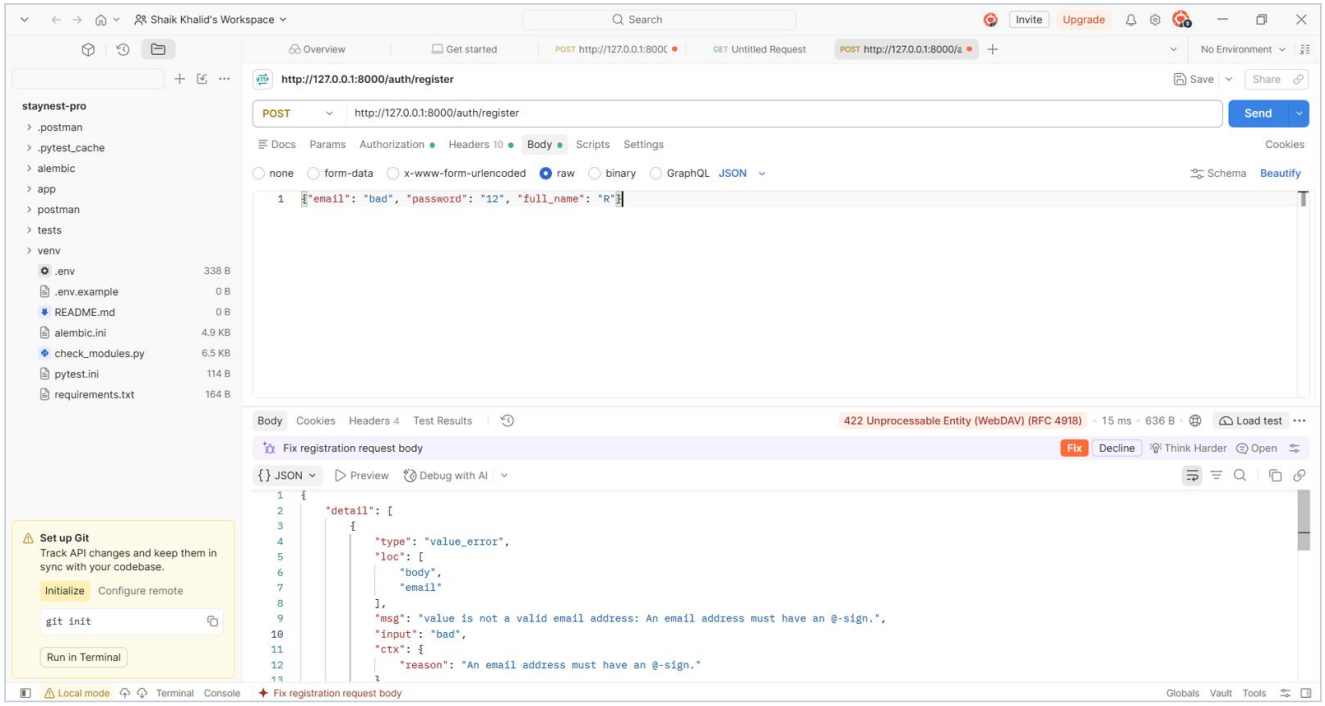
The customer books hotel 1, room type 1, from 16 to 18 November 2026 for 2 adults.
Booking SN538D728F is PENDING_PAYMENT: Rs 7000 + Rs 840 GST = Rs 7840.

Figure 124. Postman - admin profile



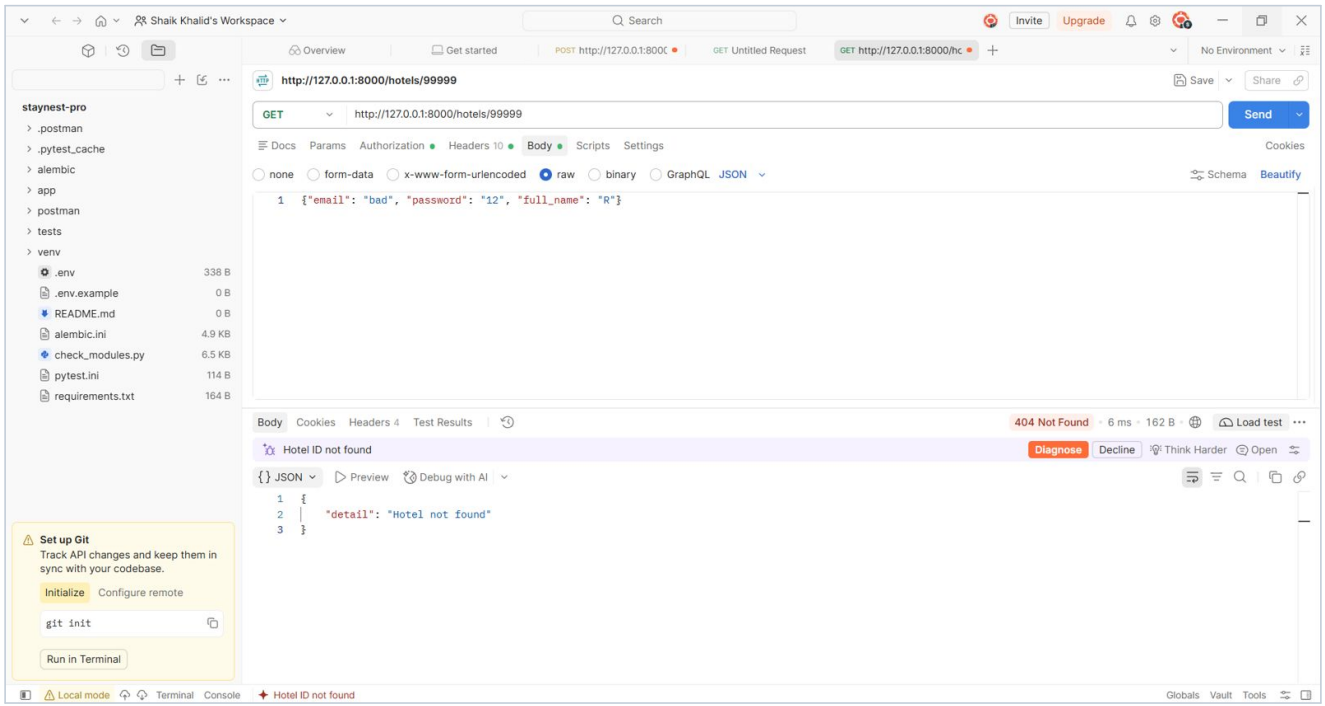
GET /auth/me with the admin token.
admin@staynest.com is returned with role ADMIN and a verified email.

Figure 125. Postman - validation error (422)



Register is sent with a bad email, a 2-character password and a 1-letter name.
Pydantic rejects it with 422: value is not a valid email address.

Figure 126. Postman - not found (404)

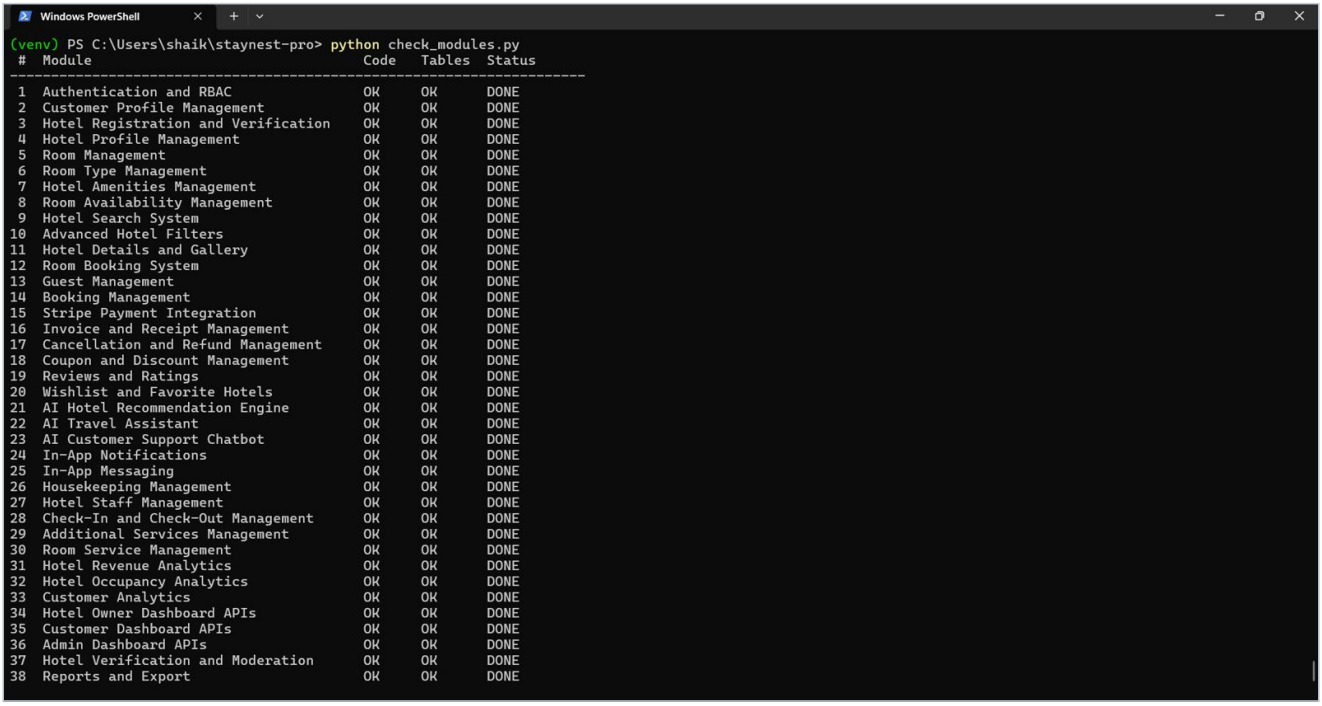


GET /hotels/99999 asks for a hotel that does not exist.
The API answers 404 Hotel not found.

Part 5 - Module Check and Automated Tests

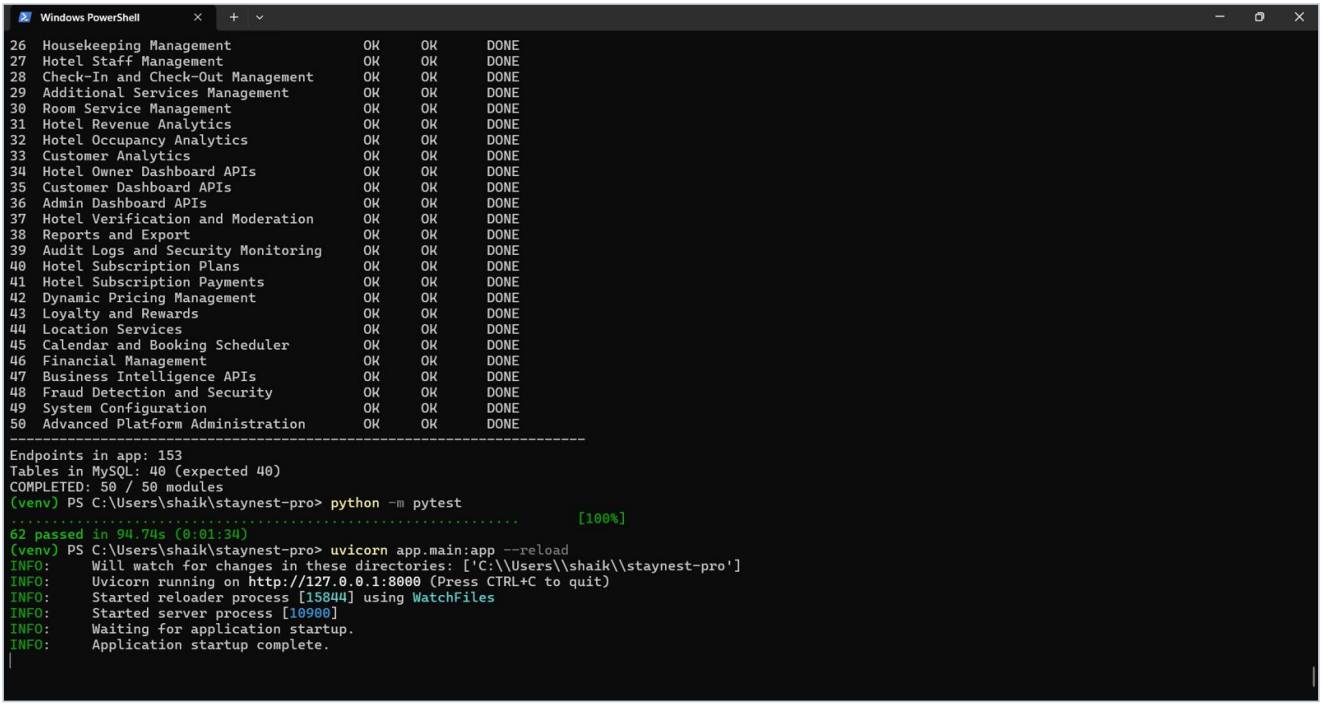
check_modules.py confirms 50 / 50 modules and 40 / 40 tables; pytest runs 62 tests on an in-memory database with mocked Stripe.

Figure 127. check_modules.py - modules 1 to 38



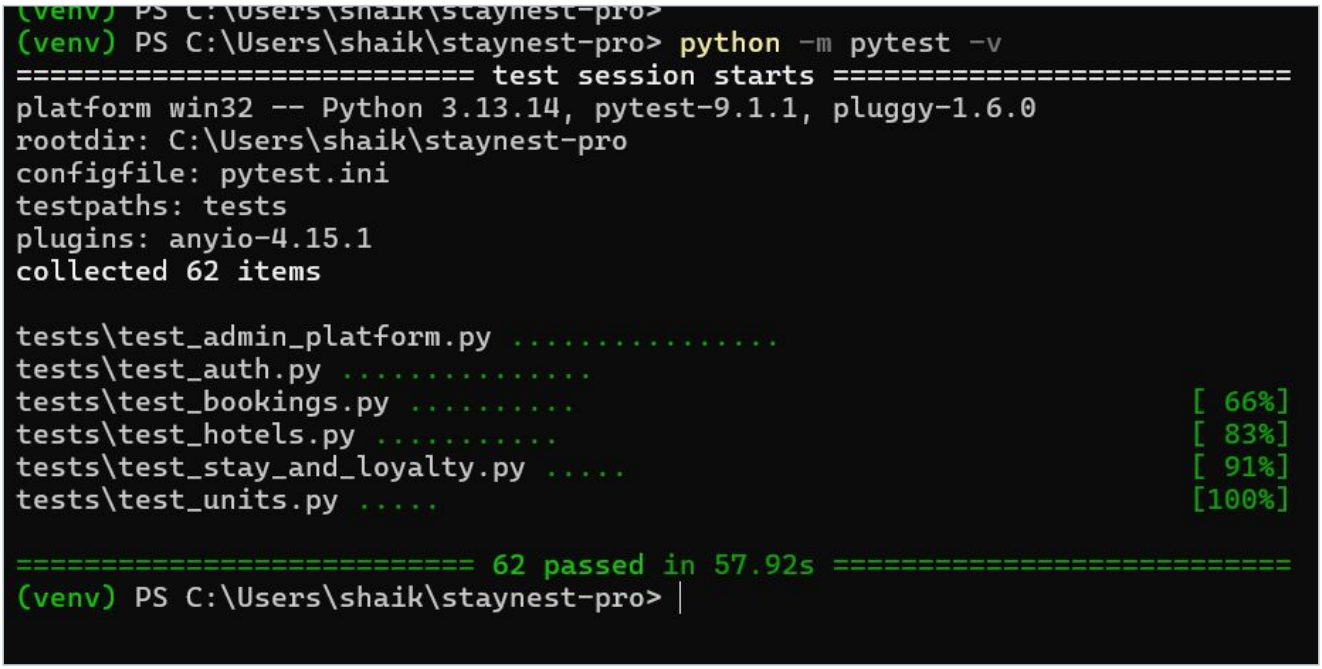
The script checks that each module has its code and its MySQL tables.
Modules 1 to 38, from Authentication and RBAC to Reports and Export, all show OK, OK, DONE.

Figure 128. Modules 39 to 50, pytest and server start



Modules 39 to 50 are DONE: 153 endpoints, 40 of 40 tables, COMPLETED 50 / 50 modules.
pytest reports 62 passed, and uvicorn starts at http://127.0.0.1:8000 with application startup complete.

Figure 129. pytest - 62 passed



python -m pytest -v runs all 6 test files.
All 62 tests passed in 57.92 seconds.